

基于分阶段优化和遗传算法的无人机干扰系统

摘 要

通过对无人机和烟幕干扰弹的精确控制，可以实现在空中预定位置形成烟幕，达到低成本干扰导弹、保护目标的目的。本文针对无人机投放烟幕干扰弹以实现导弹干扰问题，以有效遮蔽时长最大化为目标，基于阴影锥体模型与运动模型等，实现了多场景下的最优无人机投掷策略的制定。

针对问题一，首先构建**运动模型**，确定无人机、导弹、干扰弹和烟幕云团运动状态。其次通过几何分析的方式，构建**阴影锥体模型**，通过在真目标表面采样的方法判断真实目标是否被有效遮挡。若当该时刻所有**采样点**均被遮蔽，则认为该时刻有效遮蔽。随后，将结束遮蔽时刻减去开始遮蔽时刻。最终求得题目给出的无人机投掷策略下的有效遮蔽时长约为 1.4159 秒。

针对问题二，为指定合理的单无人机单干扰弹干扰单导弹策略，我们以有效遮蔽时长为优化目标，构建**优化模型**。针对某一投掷参数的有效遮蔽时间计算，基于问题一构建模型进行有效遮蔽时间的计算。我们首先通过**几何分析**缩小求解范围，随后通过**全局粗略搜索算法**与**局部精细搜索算法**相结合的方式求解，最终得到无人机投掷策略为：无人机 FY1 飞行方向角为 6° 、飞行速度为 138m/s、烟幕干扰弹投放点为(17844.6043, 4.6881, 1800)、烟幕干扰弹起爆点为(17913.2263, 11.9006, 1798.775)，此时总遮蔽时长约为 4.6448s。

针对问题三，考虑到干扰弹数目的增加，首先，实现对多干扰弹位置的描述。其次，针对烟雾云团的遮蔽时间重叠问题，采用**区间合并算法**对重叠遮蔽时间区间进行处理。由于到决策变量维数变高，采用**遗传算法**加快搜索，以确定最优策略。通过 python 程序最终得到的多烟幕干扰弹投掷方案为：无人机 FY1 的方向角为 4.8799° ，飞行速度为 140 m/s；第一枚干扰弹在第 0s 投放，立即引爆；第二枚干扰弹在第 1s 投放，立即引爆；第三枚干扰弹在第 7.0043s 投放，随后 0.2036s 引爆，该投掷策略的总遮蔽时间约为 6.41s。并将相应结果填入文件 result1.xlsx。

针对问题四，需要制定多无人机的投弹策略。首先，描述多架无人机的运动信息。其次，基于问题三对云团重叠问题进行处理。最后，构建优化模型，并以采用**分阶段优化策略**进行搜索，即先选择每架无人机的优秀投弹策略，再对所有它们进行**组合**，最终求得最优策略。经搜索算法求解得到的投掷方案为：无人机 FY1 的方向角度为 5.625° ，飞行速度为 139.75 m/s，投放时间为 0.45s，起爆延迟为 0.4s；无人机 FY2 的方向角度为 293° ，飞行速度为 134 m/s，投放时间为 5.75s，起爆延迟为 5.25s；无人机 FY2 的方向角度为 74° ，飞行速度为 133 m/s，投放时间为 23.75s，起爆延迟为 0.4s，该投掷策略的总遮蔽时间约为 11.82s；并将结果填写到文件 result2.xlsx。

针对问题五，需要制定多无人机多导弹的投弹策略，建立**层次化优化模型**。首先基于问题二粗略搜索单无人机单烟幕弹的优秀参数。其次，根据参数以**方向-速度组合**建立单机多弹优化模型，对每机挑选出前 100 方案。最后，以组合方式，辅以**多目标权衡评价体系**，选择出最佳策略。策略的遮蔽总时长约为 21.11s。

关键词：运动模型 阴影锥体模型 优化模型 区间合并算法 遗传算法

一、问题重述

1.1 问题背景

为对抗导弹探测，提高战场中人员存活率和保护装备，烟幕干扰弹因其低成本、高效费比等优点，成为各国军事发展重点^[1]。烟幕干扰是一种广泛应用的光电无源干扰方式，通过向空中施放大量气溶胶微粒，可以有效掩盖要保护的目标^[2]。由于该干扰技术迅速发展，现在可以实现在抛撒烟幕干扰弹前精确控制烟幕干扰弹到达预定位置并控制起爆时间，提升干扰效率。现代战争形态正加速向智能化、无人化演进^[3]，运用无人机完成烟幕干扰弹的投放策略，对保护目标进行有效掩盖，成为对抗导弹探测的关键议题。

1.2 问题提出

携带烟幕干扰弹的无人机以 $70\sim 140\text{ m/s}$ 的速度水平匀速直线飞行，其飞行方向和速度确定后不可变。无人机投放的干扰弹在起爆后形成球状云团，该云团以 3 m/s 的速度匀速下沉，且距云团中心 10 m 内的烟幕浓度可在 20 s 内提供有效遮蔽。来袭导弹速度以 300 m/s 的速度飞向假目标。为尽量避免导弹发现真目标，实现高效的干扰效果，设计有效的无人机投放烟幕干扰弹策略，本文基于上述数据，通过运动学分析及建立数学模型，分析以下问题：

问题 1：无人机 FY1 位于 $(17800, 0, 1800)$ ，并以 120 m/s 的速度飞向假目标。导弹 M1 位于 $(20000, 0, 2000)$ ，现在派遣无人机 FY1 以指定方式投放并引爆 1 枚烟幕干扰弹对导弹 M1 进行干扰，计算烟幕干扰弹对导弹的有效遮蔽时长。

问题 2：该问是在问题 1 的基础上进行推广，由确定的无人机飞行方式、投弹方式推广至未知的飞行方式、投弹方式，制定无人机 FY1 飞行策略和 1 枚烟雾干扰弹的投放策略，使得有效遮蔽时间最优。

问题 3：该问基于问题 2 对投弹数量上进行了拓展，现在无人机可投放 3 枚干扰弹对导弹进行干扰，设计合理的烟幕干扰弹无人机 FY1 飞行策略和烟雾干扰弹的投放策略，使得烟幕对导弹视线的阻断时间尽可能长，并将结果填入文件 `result1.xlsx`。

问题 4：该问基于问题 2 对无人机数量上进行了拓展，现在 3 架不同位置的无人机各可投掷 1 枚烟幕弹对导弹进行干扰，设计合理的飞行策略和烟幕干扰弹的投放策略，并将结果填入文件 `result2.xlsx`。

问题 5：现在利用所有无人机对导弹 M1、M2、M3 实施干扰，每架无人机有 3 枚烟幕干扰弹，设计合理的飞行策略和烟幕干扰弹的投放策略，并将结果填入文件 `result3.xlsx`。

二、问题分析

2.1 问题一的分析

问题一在已知无人机 FY1 飞行方式、干扰弹投放和起爆策略的条件下，要求得到真实目标被有效遮蔽时间。首先，依据题目条件构建运动学方程，用位置向量表示出无人机、导弹、干扰弹和云团的位置信息。然后构建以导弹到云团球体的切线为母线，导弹到云团中心的连线为旋转轴的阴影锥体模型，对真目标是否被完全遮蔽进行判断。对于真实目标是否在锥体内部的问题，通过在真实目标表面采样的方式，判断采样点是否位于锥体内部。通过分析导弹到采样点的方向向量和导弹到云团中心的方向向量等，运用几何分析的方式得到判定公式。当该时刻所有采样点均判定为被遮蔽，则认为该时刻有效遮蔽真实目标，并在总有效时长中累加有效遮蔽的时刻。

2.2 问题二的分析

问题二要求得到单导弹单无人机单干扰弹最佳投掷策略，即在导弹行为已知的情况下，求得使得真实目标遮蔽时间最长的无人机投掷参数。问题分析过程如下图所示

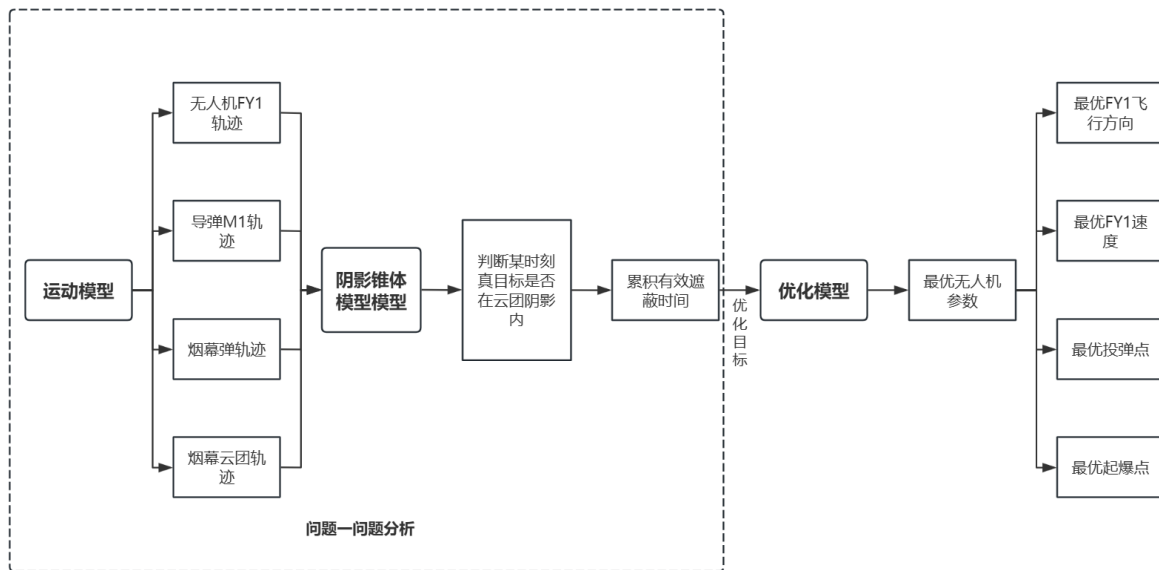


图 1 问题二流程图

2.3 问题三的分析

问题三要求利用无人机 FY1 投放 3 枚烟幕干扰弹对导弹 M1 实施干扰，即求得单导弹单无人机多干扰弹最优投弹策略设计。该问题在问题二的基础上，增加了无人机的投弹数目，增加了优化模型的参数维度。由于多枚干扰弹的遮蔽区域在可能存在重叠，故需要对重叠遮蔽时间段进行处理，计算出总有效遮蔽时间。最后，构建以总遮蔽时间最大化为目标的优化模型，来求解投放策略参数。由于到决策变量维数较高，采用遗传算法进行搜索，以确定最优的多弹投放策略。

2.4 问题四的分析

问题四基于问题二对无人机数量进行了拓展，要求无人机 FY1、FY2、FY3 各投放 1 枚烟幕干扰弹对导弹 M1 实施干扰。该问题需进行单导弹多无人机单干扰弹最优投弹策略设计。首先，对运动模型进行拓展，确定每架无人机位置、对应投放的干扰弹的运动及烟幕云团的运动过程。其次，3 架无人机引爆干扰弹形成的烟幕云团可能形成的重叠遮蔽区域，即在遮蔽时间发生重叠现象。因此，采用时间区间合并算法来处理不同云团遮蔽时段的重叠问题。接着，以总遮蔽时间最大化为目标，构建优化模型。在求解过程中，采用分阶段优化策略降低复杂度。先筛选出独立的每架无人机的投放优秀策略作为候选策略，再对所有候选策略进行组合，计算每种组合策略的总遮蔽时间，最终求得最优策略。

2.5 问题五的分析

问题五要求得到五架无人机 FY1 至 FY5 各投至多 3 枚烟雾弹对三枚导弹 M1 至 M3 实施干扰的最优投弹策略。由于每架无人机都有运动方向、速度，每枚烟雾弹都有投放时间与起爆时间，可变参数空间过于庞大，因此我们采用了多层次优化架构，对每架无人机分而治之。首先，对每个无人机-导弹对，优化单个烟幕弹的参数；其次，根据该结果进行每架无人机导弹拦截倾向决策，确定无人机最终的方向和速度；最后，组合所有无人机的投弹候选策略，评估得出最优组合

三、模型假设

1. **无人机匀速水平直线飞行：**假设无人机在飞行过程中保持匀速直线飞行，且飞行高度不变，并且飞行方向和速度在确定后不可以改变。
2. **烟幕干扰弹运动理想化：**烟幕干扰弹投放后做平抛运动，忽略空气阻力等外部因素对干扰弹的影响。
3. **云团形态规则且下沉匀速稳定：**干扰弹起爆后形成正球体状云团，且云团始终以 3m/s 的速度匀速下沉，不受风速等外部因素影响。
4. **烟幕云团遮蔽效果即时生效：**云团在起爆后瞬间形成，并在 20 秒内于有效遮蔽半径 10 米范围内保持浓度，遮蔽作用立即生效。
5. **导弹匀速直线飞行：**来袭导弹以 300m/s 的速度匀速飞向假目标，其运动轨迹为直线，不考虑导弹的机动变向。

四、符号说明

符号	说明	单位
t	导弹飞行时长	s
α	阴影锥体的锥角角度	度
R_C	烟幕云团有效遮蔽半径	m
P	真目标表面采样点	/
$P_{F,i}(t)$	t 时刻无人机 FYi 的位置向量	/
$\theta_{F,i}$	无人机的飞行方向与 x 轴方向的夹角	度

五、模型的建立与求解

5.1 问题一模型的建立与求解

5.1.1 模型的建立

根据题目中已知的真、假目标坐标，导弹、无人机运动速度和方向，烟幕干扰弹投放、爆炸时间以及烟幕云团特性，要求解对应有效遮蔽总时长。首先，可依据运动学方程建立运动模型分别表示出导弹 M1、无人机 FY1、无人机 FY1 投放的烟幕干扰弹和干扰弹爆炸后产生的烟幕云团中心点在不同时刻的位置。随后，依据云团与导弹的位置关系，建立一个阴影锥体模型，该锥体侧面由导弹与云团有效遮蔽球体的切线构成。在有效遮蔽时效内，当真目标位于该锥体内部且云团位于真目标与导弹之间，或导弹在云团内部，则判断导弹被有效遮蔽。通过对真目标进行采样的方式，判断每个时刻真目标是否被遮蔽。最后，对遮蔽时刻进行累加，得到指定干扰策略下的有效遮蔽时长。

5.1.2 运动模型

● 导弹 M1 运动方程

依据题目条件，依据运动学公式，导弹 M1 的位置函数为

$$P_M(t) = P_{M0} + \frac{P_{FT} - P_{M0}}{|P_{FT} - P_{M0}|} \cdot v_M \cdot t \quad (1)$$

其中，t 为导弹飞行时长， $P_M(t)$ 为 t 时刻导弹 M1 的位置向量， P_{M0} 为导弹 M1 的初始位置， P_{FT} 为假目标位置， v_M 为导弹 M1 的恒定飞行速度大小。

- 无人机 FY1 运动方程

依据题目条件，无人机 FY1 位置函数为

$$\mathbf{P}_F(t) = \mathbf{P}_{F0} + v_F \cdot t \cdot (\cos\theta_F, \sin\theta_F, 0) \quad (2)$$

其中， $\mathbf{P}_F(t)$ 为t时刻无人机 FY1 的位置向量， $\mathbf{P}_{F0}$ 为无人机 FY1 的初始位置， v_F 为无人机 FY1 速度大小， θ_F 为在 xy 面上无人机 FY1 的飞行方向于 x 轴方向的夹角，即无人机 FY1 的飞行方向角，故 $(\cos\theta_F, \sin\theta_F, 0)$ 是飞行器飞行的方向向量。

- 烟幕干扰弹运动方程

依据题目条件和运动学公式，烟幕干扰弹在投放后做平抛运动，干扰弹在被投放后的位置函数为

$$\mathbf{P}_S(t) = \mathbf{P}_F(t_r) + v_F \cdot (t - t_r) \cdot (\cos\theta, \sin\theta, 0) + \left(0, 0, -\frac{1}{2}g(t - t_r)^2\right), \quad t \geq t_r \quad (3)$$

其中， $\mathbf{P}_S(t)$ 为烟雾弹在t时刻的位置， t_r 为烟雾弹的投放时刻， $\mathbf{P}_F(t_r)$ 为烟幕干扰弹刚被投放时的位置， g 为重力加速度。

- 烟幕云团运动方程

依据运动学方程和题目描述，起爆后的云团中心位置函数为

$$\mathbf{P}_C(t) = \mathbf{P}_S(t_d) + (0, 0, -v_D \cdot (t - t_d)), \quad t \geq t_d \quad (4)$$

其中， $\mathbf{P}_C(t)$ 为t时刻云团中心的位置， t_d 为起爆时刻，且 $t_d = t_r + \Delta t_d$ ， Δt_d 为起爆延迟， $\mathbf{P}_S(t_d)$ 为起爆时刻云团中心的位置， v_D 烟幕云团下沉速度大小。

5.1.3 阴影锥体模型建立

建立阴影锥体模型，通过判断真实目标是否完全位于锥体内部，判断是否有效遮蔽并将遮蔽时间进行累加计算总有效遮蔽时间，所建立的阴影锥体如下图所示

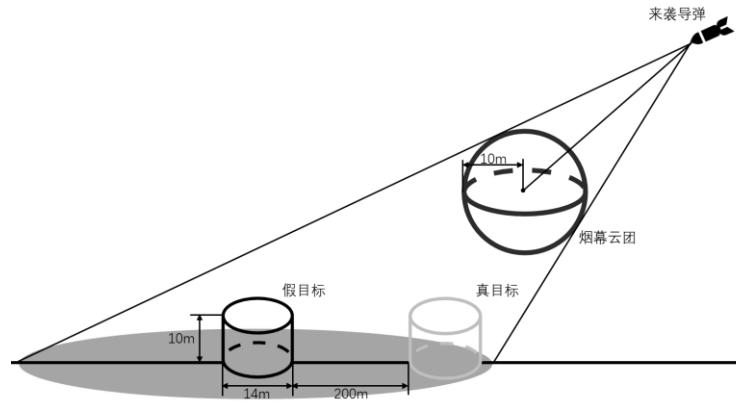


图 2 阴影锥体模型示意图

若目标被遮蔽，则真实目标上的所有点都位于阴影锥体内部，即应满足阴影锥体锥角 α 大于真实目标上所有点到导弹的连线与云团中心到导弹的连线间的夹角。

首先，计算锥体的锥角，t时刻导弹到烟幕云团的方向向量为 $\mathbf{d} = \mathbf{P}_C(t) - \mathbf{P}_M(t)$ ，故由几何关系可知，锥体模型锥角的三角函数值如下：

$$\begin{aligned} \sin \alpha &= \min\left(1.0, \frac{R_C}{|\mathbf{d}|}\right) \\ \cos \alpha &= \sqrt{1 - \sin^2 \alpha} \end{aligned} \quad (5)$$

其中，阴影锥体的锥角为 α ， R_C 为烟幕云团有效遮蔽半径。

其次，为提高计算效率，在对目标遮蔽判断时采用采样方式。若真目标圆柱体的表面的点位于阴影锥体中，则真实目标位于阴影锥体中，故在在真实目标圆柱体底面和顶面的圆周上均匀选取 n 个采样点。

接着，比较采样点是否位于锥体内部，依据几何关系，若采样点位于锥体内部，则导弹到采样点的向量与导弹到云团的方向向量的夹角应小于锥体锥角。设采样点为 $\mathbf{P}$ ，则导弹到采样点的向量为 $\mathbf{v} = \mathbf{P} - \mathbf{P}_M(t)$ ，则向量 $\mathbf{v}$ 与方向向量 $\mathbf{d}$ 的夹角 β 的余弦值如下：

$$\cos\beta = \frac{\mathbf{v} \cdot \mathbf{d}}{|\mathbf{v}||\mathbf{d}|} \quad (6)$$

若要使 $\alpha > \beta$ ，仅需使得 $|\cos\beta| \geq |\cos\alpha|$ 。因此，当所有采样点均满足 $|\cos\beta| \geq |\cos\alpha|$ ，则认为目标完全被遮蔽；反之，则认为目标未被有效遮蔽。

最后，对于有效遮蔽时长计算，通过对每个时刻进行目标遮蔽判断，对被遮蔽的时刻进行累加得到总有效遮蔽时长。

5.1.4 模型的求解

在 $[0s, 15s]$ 的时间区间内，我们将其均匀离散为 100,000 个等间隔的采样时刻，对于每一时刻 t ，均按如下步骤通过采样的方式判断遮蔽是否有效并计算总遮蔽时间：

首先，对通过时间判断烟幕是否有效，若 $t < t_d$ 或 $t > t_d + T$ ，则遮蔽无效，其中 t 为当前时间， t_d 为云团形成时间， T 为烟幕有效持续时间。其次，计算导弹到云团中心的距离 $|\mathbf{d}|$ 并与 R_c 比较，若 $|\mathbf{d}|$ 小于有效半径 R_c ，则导弹在云团内部，当前时刻遮蔽有效。接着，计算导弹到烟幕云团的方向向量为 $\mathbf{d}$ 和锥体锥角 θ 的三角函数值，并在目标圆柱体底面和顶面圆周上生成 12 个采样点。此外，对每个采样点 $\mathbf{P}$ ，判断其是否满足 $\cos\alpha < \cos\theta$ ，若存在点不满足，则该时刻遮蔽无效；否则，该时刻遮蔽有效。最后，若判断该时刻有效则将在该时刻前的时间间隔累加到总有效遮蔽时间中。

根据以上过程，得到按照题目给定投掷方案，烟幕干扰弹在时间区间 $[8.03213s, 9.44799s]$ 烟幕干扰弹对 M1 的有效遮蔽时长约为 1.4159 秒，精确到小数点后 4 位。

5.2 问题二模型的建立与求解

5.2.1 模型的建立

为使得有效遮蔽时间最长，即使得真实目标在阴影锥体内部时间最长，需找到最优的无人机飞行、投弹参数。首先，依据问题一建立的运动模型，得到导弹 M1、无人机 FY1、烟幕干扰弹和爆炸云团中心位置轨迹。接着，依据问题一建立的阴影锥体模型。最后，建立优化模型，求解出使得有效遮蔽时间最大化的无人机 FY1 参数，得到最优 FY1 的飞行方向、速度、烟幕干扰弹投放点以及起爆点。

5.2.2 优化模型

题目要求，设计 FY1 的飞行方向 θ_F 、飞行速度 v_F 、烟幕干扰弹投放点 $\mathbf{P}_F(t_r)$ 、烟幕干扰弹起爆点 $\mathbf{P}_S(t_d)$ ，使得有效遮蔽总时长 T_E 最长。若要得到烟幕干扰弹投放点 $\mathbf{P}_F(t_r)$ 和烟幕干扰弹起爆点 $\mathbf{P}_S(t_d)$ ，只需得到投放时刻 t_r 和起爆时刻 t_d ，又因 $t_d = t_r + \Delta t_d$ ，故只需得到投放时刻 t_r 和起爆延迟 Δt_d 。

● 目标函数

优化的目标是找到最优参数组合 $(\theta_F^*, v_F^*, t_r^*, \Delta t_d^*)$ ，使有效遮蔽总时长 T_E 最长，故优化目标函数为

$$\max T_E = T_E(\theta_F, v_F, t_r, \Delta t_d) \quad (7)$$

● 约束条件

$$s.t. \begin{cases} \theta_F \in [0^\circ, 360^\circ) \\ v_F \in [70 \text{ m/s}, 140 \text{ m/s}] \\ t_r \geq 0 \\ \Delta t_d \geq 0 \end{cases} \quad (8)$$

关于约束条件的说明：

依据题目描述，无人机 FY1 以 70~140 m/s 的速度飞行，故飞行速度 v_F 在 $[70 \text{ m/s}, 140 \text{ m/s}]$ 的区间内。

5.2.3 模型的求解

由于问题的复杂性，我们采用数值优化方法求解，包括全局搜索和局部精细化搜索两个阶段。

首先，为了简化问题，减小搜索范围，观察无人机、导弹、真目标和假目标四者在 xy 平面上的投影分析飞行角度与遮蔽效果的关系，如下图所示

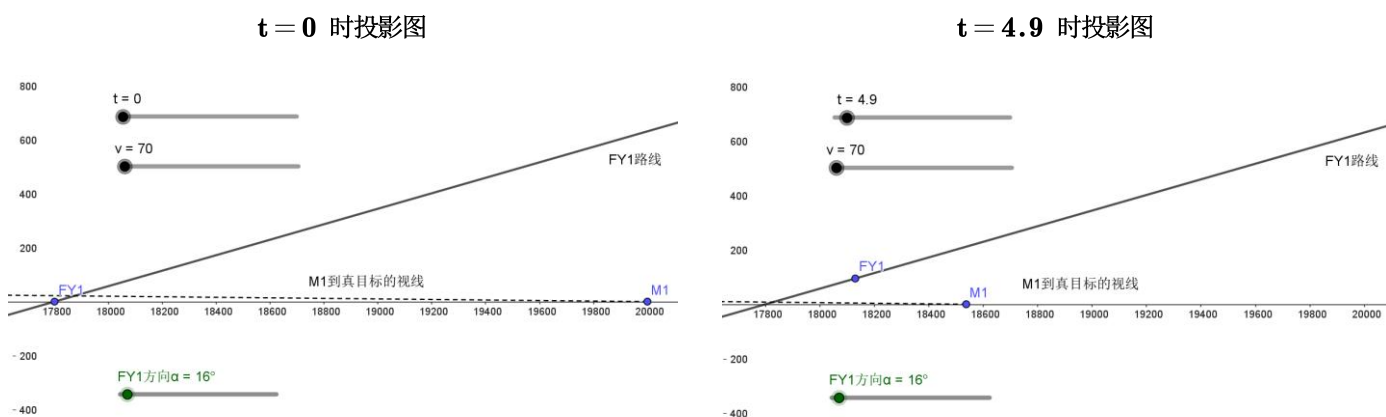


图 4 无人机、导弹、真目标和假目标在 xy 平面上投影图

显然，若烟幕在 xy 平面的投影无法遮蔽导弹视线，则在三维空间中也不可以有效遮蔽。由图中无人机 FY1 和导弹 M1 两者运动轨迹的位置关系可知，只有当无人机方向角在区间 $[0, 180^\circ]$ 时才可能产生有效遮蔽，且角度偏向于 x 轴时效果更好，因此，对于无人机方向角 θ_F ，优先考虑 $0-30^\circ$ 和 $150-180^\circ$ 这两个区间。下图分析了在这两个区间内的不同方向组中飞行速度对最大有效遮蔽时长的影响

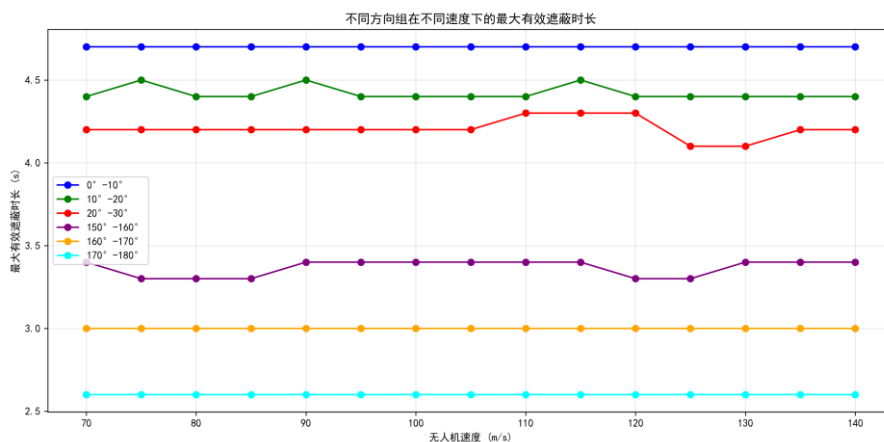


图 3 不同方向组下速度对最大有效遮蔽时长的影响

由图中分析可知，获得最大遮蔽时长需选择 $0-10^\circ$ 方向组。

下图展示了在 $0-10^\circ$ 方向区间内，方向、速度与目标有效遮蔽时长的函数关系。

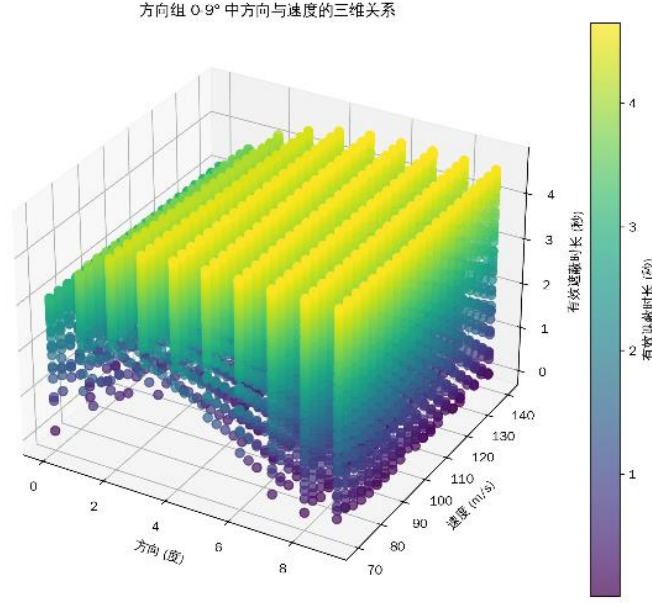


图 4-9° 方向区间有效遮蔽时长与无人机方向与速度相关三维关系图

由图中分析可知，应将无人机方向角 θ_F 搜索范围限制在 $3-9^\circ$ ，将无人机速度 v_F 搜索范围限制在 $90-140\text{m/s}$ 。

其次，基于以上对参数的研究和题目性质，对于参数 $\theta_F, v_F, t_r, \Delta t_d$ ，搜索方式如下：对于无人机方向角 θ_F ，其搜索范围为 $3-9^\circ$ ，搜索步长为 0.15° ；对于无人机速度 v_F ，在区间 $90-140\text{ m/s}$ 内搜索，搜索步长为 2m/s ；对于干扰弹投放时间 t_r ，其搜索范围为 $0-1.5\text{s}$ ，搜索步长为 0.15s ；对起爆延迟 Δt_d ，其搜索范围为 $0-1\text{s}$ ，搜索步长为 0.1s 。

● 全局搜索算法

我们对参数组合 $(\theta_F, v_F, t_r, \Delta t_d)$ 按以上搜索方式进行搜索，并在每一次搜索时进行如下操作：

Step1: 确定无人机和烟幕干扰弹的运动状态

由运动模型，依据确切的参数值得到无人机轨迹 $\mathbf{P}_F(t)$ 、烟幕干扰弹投放点 $\mathbf{P}_F(t_r)$ 和烟幕干扰弹起爆点 $\mathbf{P}_S(t_d)$ 。

Step2: 模拟导弹飞行过程

对于飞行时间 t ，以 0.01s 为步长，对每个时间步进行计算和判断，模拟其飞行过程。计算该时刻导弹位置 $\mathbf{P}_M(t)$ 和烟幕云团中心位置 $\mathbf{P}_C(t)$ ，并依据阴影锥体模型判断该时刻是否形成有效遮蔽。

Step3: 累计计算有效遮蔽总时长

在模拟飞行过程中，若判断该时间步可以形成有效遮挡，则在有效遮蔽总时长 T_E 中对该时间步进行累加。

经过全局搜索后，选择前 10 个使有效遮蔽总时长 T_E 最佳的参数组合 $(\theta_F^*, v_F^*, t_r^*, \Delta t_d^*)$ ，进一步进行局部精细化所搜。

全局搜索部分结果如下表所示：

表 1 前 10 个全局搜索最佳结果表

无人机方向角 θ	无人机速度 v_F	烟幕干扰弹投放时间 t_r	起爆延迟 Δt_d	有效遮蔽总时长 T_E
6	136.5	0.3	0.5	4.65
7	140	0	0.7	4.65
5	119	1.05	0	4.65
5	77	1.05	0.4	4.64
5	80.5	1.05	0.4	4.64
5	84	0.9	0.4	4.64
5	87.5	1.05	0.3	4.64
5	91	0.75	0.5	4.64
5	91	1.05	0.2	4.64
5	94.5	1.2	0.1	4.64

故经全局搜索后的最佳结果为当无人机 FY1 的飞行方向角为 6° 、飞行速度为 136.5m/s、烟幕干扰弹投放点为 (17840.7256716153, 4.28044057081041, 1800)及烟幕干扰弹起爆点为(17908.6017909742, 11.4145081888278, 1798.775)，此时有效遮蔽总时长 T_E 最长，为 4.65s。

● 局部精细化搜索

为了在全局最优解附近进行更精细的搜索，以获取更精确的烟幕干扰弹投放参数，我们采用局部参数空间精细化策略。基于已获得前 10 个的全局最优解，在其邻域进行更精细的所搜，其步骤如下：

Step1: 确定精细搜索空间

依据全局搜索步长和全局搜索所得结果，确定精细搜索范围：无人机方向角 θ_F 在 $\theta_F^* \pm 0.075^\circ$ 内取 5 个均匀分布的值；无人机速度 v_F 在 $v_F^* \pm 0.5$ m/s 内取 5 个均匀分布的值；干扰弹投放时间 t_r 在 $t_r^* \pm 0.25$ s 内取 5 个均匀分布的值；起爆延迟 Δt_d 在 $\Delta t_d^* \pm 0.25$ s 内取 5 个均匀分布的值。

Step2: 累计计算有效遮蔽总时长

将时间片步长精细为 0.0001s，重复全局搜索算法的 Step2 和 Step3，计算出不同参数下的有效遮蔽总时长并比较。

经局部精细化搜索后，其搜索结果存于 all_fine_tuned_solutions.xlsx。当无人机 FY1 的飞行方向角为 6° 、飞行速度为 138m/s、烟幕干扰弹投放点为(17844.6043, 4.6881, 1800)及烟幕干扰弹起爆点为(17913.2263, 11.9006, 1798.775)，此时有效遮蔽总时长最长，为 4.6448s。

5.3 问题三模型的建立与求解

5.3.1 模型的建立

现在，烟幕干扰弹数量增加到 3 枚，需要确定无人机 FY1 的飞行方向、飞行速度以及 3 枚烟幕干扰弹的投放时间和起爆延迟，使得烟幕对导弹视线的阻断时间尽可能长。通过遗传算法求解，寻找最优参数向量，使得有效遮蔽总时长 T_E 最大化。

5.3.2 优化模型

设参数向量为 $\mathbf{I}_1 = (\theta_F, v_F, t_{r1}, \Delta t_{d1}, t_{r2}, \Delta t_{d2}, t_{r3}, \Delta t_{d3})$ ，其中 θ_F 为无人机飞行方向角度， v_F 为无人机飞行速度， t_{ri} 为第 i 枚烟幕干扰弹的投放时间， Δt_{di} 为第 i 枚烟幕干扰弹的起爆延迟。

- 目标函数

优化目标为使遮蔽时长最长，则使遮蔽总时长 T_E 最大化可表示为

$$\max T_E(\mathbf{I}_1) \quad (9)$$

- 约束条件

约束条件可表述为

$$s.t. \begin{cases} 0^\circ \leq \theta_F < 360^\circ \\ 70 \text{ m/s} \leq v_F \leq 140 \text{ m/s} \\ 0 \leq t_{r1} < t_{r2} < t_{r3} \\ 0 \leq \Delta t_{di} \leq 2.0 \text{ s}, i = 1, 2, 3 \\ t_{r3} + \Delta t_{d3} \leq 12.0 \text{ s} \end{cases} \quad (10)$$

5.3.3 重叠时间处理

本题中无人机可以投放多个干扰弹，故每个干扰弹形成的阴影锥体可能存在重叠区域。在问题一、二的遮蔽时间计算过程中，未对该情况进行处理，会出现对在重叠区域的时长进行重复累加的情况，故需要对重叠问题进行处理，更改遮蔽总时长计算方法。

依据区间合并算法对遮蔽时间进行处理，对三个干扰弹的所有遮蔽时间区间进行遍历，并对重叠区间进行合并，最终累加每个区间的长度，计算出有效遮蔽总时间 T_E 。基于区间合并的重叠时间处理算法的伪代码如下

重叠时间处理伪代码

```
DATA = {[a1, b1], [a2, b2], ..., [an, bn]} //收集所有烟幕云团的遮蔽时间区间
DATA ← sort(I, key = ai) //按开始时间升序排序
M ← ∅ //初始化合并区间列表
M ← M ∪ [a1, b1] //将第一个区间加入合并列表
for i ← 2 to n
    if ai ≤ 当前合并区间结束时间 then
        bcurrent ← max(bcurrent, bi) //更新当前合并区间结束时间
    else M ← M ∪ [ai, bi] //将当前区间加入合并列表
    end if
end
TE ← ∑j=1|M| (bj - aj) //计算总时长
```

5.3.4 遗传算法

遗传算法是一种基于自然选择和遗传机制的全局搜索算法，适用于处理像本例中高维、非线性、多模态的优化问题。遗传算法包括以下关键组件：

- 1) 参数向量 $\mathbf{I}_1 = (\theta_F, v_F, t_{r1}, \Delta t_{d1}, t_{r2}, \Delta t_{d2}, t_{r3}, \Delta t_{d3})$ 使用实数编码表示，避免编解码的转换过程，便于施加约束；
- 2) 算法的初始种群大小为 80，即最优投放策略参数向量 $\mathbf{I}_1 = (\theta_F, v_F, t_{r1}, \Delta t_{d1}, t_{r2}, \Delta t_{d2}, t_{r3}, \Delta t_{d3})$ 的初始候选解有 80 个，其中大多数随机

生成，部分向量基于问题二的最优解扩展生成，这可以注入先验知识极大地加快收敛速度，避免算法在初期在低质量的投放策略参数向量的解空间中进行大量搜索；

- 3) 算法的适应度函数为有效遮蔽总时长 T_E ，我们通过阴影锥体模型，并通过对重叠时间的处理计算有效遮蔽时间。若3枚干扰弹有效遮蔽总时长 T_E 越长，其对应的投放策略参数向量适应度越高。
- 4) 算法使用锦标赛选择，即模拟优胜劣汰，在候选解向量中适应度高的投放策略参数向量越有可能被选作父代。
- 5) 算法用两个父代参数向量以0.8的交叉概率和均匀交叉的方式生成一个子代解向量的值，即子代参数向量有80%的概率是由均匀交叉的方式生成，即向量中的每个参数都以相等的概率随机来自两个父代对应参数中的某一个。子代向量有20%的概率直接复制某一个父代向量。
- 6) 算法会有变异操作，即子代向量的某些参数会有小概率发生合理扰动。对无人机方向参数进行 $\theta_F \pm 2^\circ$ 的扰动，对无人机速度进行 $v_F \pm 5 \text{ m/s}$ 的扰动，时间进行 $t \pm 0.3 \text{ s}$ 的扰动。
- 7) 对有效遮蔽总时长 T_E 最长的8个参数向量不会发生交叉或变异，而是直接复制为子代解向量。

5.3.5 模型的求解

● 参数约束处理

在遗传算法中，新一代投放策略参数向量可能违反约束条件，我们采用以下处理方式：对于无人机方向参数 θ_F ，进行求余处理，即 $\theta_F = \theta_F \bmod 360^\circ$ ；对于无人机速度参数 v ，进行范围约束，若超过速度边界则按边界速度处理，即 $v_F = \max(70, \min(140, v_F))$ ；对时间参数 t_{r1}, t_{r2}, t_{r3} ，保证其投放时间顺序，即 $t_{r1} = \max(0, t_{r1})$ ， $t_{r2} = \max(t_{r1} + 1, t_{r2})$ ， $t_{r3} = \max(t_{r2} + 1, t_{r3})$ ；对于总体时间，若出现 $t_{r3} + \Delta t_{d3} > 12.0$ 的情况，则按比例缩放所有投放时间使得 $t_{r3} + \Delta t_{d3} \leq 12.0$ 。

● 初始种群构建

遗传算法的初始种群，即初代80个投放参数向量包括以下四个类型：

- 1) 随机参数向量：该向量内各参数均为在约束范围内随机生成
- 2) 问题二扩展向量：基于最优解 $[6.0, 138, 0.325, 0.5, 2.5, 0.5, 4.7, 0.6]$
- 3) 均匀间隔策略向量：即 $[5.0, 135.0, 0.2, 0.3, 2.7, 0.4, 5.2, 0.5]$
- 4) 差异化时序向量：包括早期遮蔽优先和后期遮蔽优先两种策略

● 遗传算法搜索最优解

对初代解向量进行迭代，对每个参数向量进行遮蔽总时间的计算，找出本代的精英，选择父代解向量，并通过交叉操作、变异操作和精英保留方式生成下一代，当进行到最后一代时，即第40代时，结束搜索操作。

经遗传算法搜索后，第1代至第40代每一代的最优解如下表所示

表 2 遗传算法中每一代的最优解

代数	遮蔽总 时间	无人机 方向角	无人机速 度	t_{r1}	Δt_{d1}	t_{r2}	Δt_{d2}	t_{r3}	Δt_{d3}
1	4.65	6	138	0.325	0.5	2.5	0.5	4.7	0.6
2	4.79	5.2476	100.564	0.4225	0.5199	1.4397	0.2196	6.5641	0.2279
3	4.79	5.2476	100.564	0.4225	0.5199	1.4397	0.2196	6.5641	0.2279
4	5.48	5.2476	100.564	0	0.5	1.5232	0.2196	3	0.6744
5	5.6	2.691	100.564	0.3759	0.0533	1.4397	0.2196	7.4689	0.9254
6	5.6	3.7734	100.564	0.4225	0.0049	1.4397	0.2196	6.6469	0.6
7	5.66	5.2476	138	0	0.5	1.2169	0.2196	3	0.6
8	5.94	3.8761	138	0	0.5	1.2169	0	3	0.6
9	6.07	3.8761	138	0	0.2417	1.2169	0	6.5108	0.6
10	6.17	5.2476	100.564	0	0.0049	1.2169	0	4.7648	0.2279
11	6.19	3.771	138	0	0.0049	1.2169	0	6.6469	0.576
12	6.19	3.771	138	0	0.0049	1.2169	0	6.6469	0.576
13	6.2	3.7734	138	0	0.0049	1.2063	0	6.5108	0.6362
14	6.22	3.771	136.4354	0.1135	0	1.136	0	6.6469	0.7444
15	6.36	6.4405	136.4354	0	0.0131	1	0	6.5182	0.7444
16	6.36	6.4405	136.4354	0	0.0131	1	0	6.5182	0.7444
17	6.36	6.4405	136.4354	0	0.0131	1	0	6.5182	0.7444
18	6.36	6.4405	136.4354	0	0.0131	1	0	6.5182	0.7444
19	6.41	5.1207	136.4354	0	0	1	0	6.6999	0.7444
20	6.41	5.1207	136.4354	0	0	1	0	6.6999	0.7444
21	6.41	5.1207	136.4354	0	0	1	0	6.6999	0.7444
22	6.41	5.1207	136.4354	0	0	1	0	6.6999	0.7444
23	6.41	5.1207	136.4354	0	0	1	0	6.6999	0.4189
24	6.41	5.2466	131.8495	0	0	1	0	6.9865	0.7444
25	6.41	6.1096	136.4354	0	0	1	0	6.5645	0.7444
26	6.41	5.1207	136.4354	0	0	1	0	6.7312	0.7444
27	6.41	5.1207	135.0931	0	0	1	0	6.9865	0.5134
28	6.41	5.1207	136.7405	0	0	1	0	6.8293	0.7444
29	6.41	4.5235	136.4986	0	0.0049	1	0	6.5182	0.7444
30	6.41	5.2466	136.5841	0	0.0061	1	0	6.9785	0.7444
31	6.41	5.9073	139.3289	0	0	1	0	6.6163	0.3236
32	6.41	6.0921	131.8495	0	0	1	0	6.7273	0.5213
33	6.41	5.2466	126.4814	0	0	1	0	6.697	0.7952
34	6.41	5.1207	136.4354	0	0.0049	1	0	6.8831	0.3478
35	6.41	5.1207	132.0696	0	0	1	0	6.7606	0.1983
36	6.41	4.3357	139.3289	0	0	1	0	5.3162	0.5022
37	6.41	4.3357	139.3289	0	0	1	0	6.9333	0.3137
38	6.41	6.3181	128.5903	0	0	1	0	5.1311	0.6095
39	6.41	5.011	139.3289	0	0.0049	1	0	6.5182	0.4068
40	6.41	4.8799	140	0	0	1	0	7.0043	0.2036

选择第 40 代的搜索最优结果，作为烟幕干扰弹投放策略，并将策略保存于文件 result1.xlsx 中。故烟幕干扰弹投放策略为：无人机 FY1 的方向角为 4.8799° ，飞行速度为 140 m/s；第一枚干扰弹在第 0s 投放，投放的同时引爆；第二枚干扰弹在第 1s 投放，投放后立即引爆；第三枚干扰弹在第 7.0043s 投放，随后 0.2036 引爆。此时干扰弹对导

弹 M1 的总干扰时长为 6.41s。

5.4 问题四模型的建立与求解

5.4.1 模型的建立

该问题基于问题二对无人机的数目进行了增加,求解在多无人机的前提下实现对真实目标进行最佳保护的策略。首先,对运动模型进行了扩展,使其实现对多无人机投掷干扰弹活动的过程描述。其次,运用阴影锥形模型,用以计算在不同投放参数下每架无人机投放干扰弹对导弹 M1 的遮蔽时长。最后,建立优化模型,以 3 枚干扰弹对导弹 M1 实现最长时间的干扰为目标,查找实现该目标的无人机投放策略。

5.4.2 运动模型扩展

本题中需要对 3 架无人机 FY1、FY2、FY3 及其对应的干扰弹和引爆后的云团进行描述,故拓展无人机运动方程、烟幕弹方程和烟幕云团运动方程如下。

● 无人机运动方程

基于物理学,各无人机位置函数为

$$\mathbf{P}_{F,i}(t) = \mathbf{P}_{F0,i} + v_{F,i} \cdot t \cdot (\cos\theta_{F,i}, \sin\theta_{F,i}, 0) \quad (11)$$

其中, $\mathbf{P}_{F,i}(t)$ 为 t 时刻无人机 FY i 的位置向量, $\mathbf{P}_{F0,i}$ 为第 i 架无人机的初始位置, $v_{F,i}$ 为对应无人机速度大小, $\theta_{F,i}$ 为无人机的飞行方向与 x 轴方向的夹角,即 $(\cos\theta_{F,i}, \sin\theta_{F,i}, 0)$ 是飞行器飞行的方向向量。

● 烟幕干扰弹运动方程

依据运动学公式,干扰弹位置函数为

$$\mathbf{P}_{S,i}(t) = \mathbf{P}_{F,i}(t_{r,i}) + v_{F,i} \cdot (t - t_{r,i}) \cdot (\cos\theta_{F,i}, \sin\theta_{F,i}, 0) + \left(0, 0, -\frac{1}{2}g(t - t_{r,i})^2\right), t \geq t_{r,i} \quad (12)$$

其中, $\mathbf{P}_{S,i}(t)$ 为第 i 架无人机投放的烟雾弹在 t 时刻的位置, $t_{r,i}$ 为该烟雾弹的投放时刻,故 $\mathbf{P}_{F,i}(t_{r,i})$ 为烟幕干扰弹刚被投放时的位置。

● 烟幕云团运动方程

起爆后各云团中心位置函数为

$$\mathbf{P}_{C,i}(t) = \mathbf{P}_{S,i}(t_{r,i} + \Delta t_{d,i}) + (0, 0, -v_D \cdot (t - (t_{r,i} + \Delta t_{d,i}))) \quad (13)$$

其中, $\mathbf{P}_{C,i}(t)$ 为 t 时刻第 i 架无人机投放的干扰弹引爆后产生的云团的中心位置, $\Delta t_{d,i}$ 为对应起爆延迟,即该干扰弹投放到引爆的时间间隔,故 $\mathbf{P}_{S,i}(t_{r,i} + \Delta t_{d,i})$ 为起爆时刻云团中心的位置。

5.4.3 优化模型

无人机投放烟幕干扰弹策略的目标是使对导弹 M1 的有效遮蔽时间最长,故优化模型的目标函数可表示如下

$$\max T_E(\theta_{F,1}, v_{F,1}, t_{r,1}, \Delta t_{d,1}, \theta_{F,2}, v_{F,2}, t_{r,2}, \Delta t_{d,2}, \theta_{F,3}, v_{F,3}, t_{r,3}, \Delta t_{d,3}) \quad (14)$$

其中, T_E 为三架无人机对真实目标有效遮蔽的总时长。

依据题目描述,优化模型的约束条件如下

$$s.t. \begin{cases} 0^\circ \leq \theta_i < 360^\circ \\ 70 \text{ m/s} \leq v_i \leq 140 \text{ m/s} \\ t_{r,i} \geq 0 \\ 0 \leq \Delta t_{d,i} \leq 2.0 \text{ s} \\ t_{r,i} + \Delta t_{d,i} \leq 15.0 \text{ s} \end{cases}, i = 1, 2, 3 \quad (15)$$

5.4.4 策略搜索算法

基于问题的复杂性和参数空间维度，我们采用分阶段优化策略。首先，进行单无人机最佳方案筛选。接着，对筛选后的单无人机方案进行组合评估。利用遮蔽时间区间合并算法对每种组合的总有效遮蔽时间进行计算，并经排序，选择最佳方案。

● 无人机最佳方案筛选

使用问题二的模型，计算无人机 FY1、FY2 和 FY3 各方案的有效遮蔽时长和遮蔽时间区间，并以遮蔽时长为关键词降序排列，选择每架无人机前 100 个方案。

● 时间区间合并算法

对时间区间的合并算法处理同问题二中重叠问题的处理，通过合并有重叠部分的区间，消除重叠时间的影响。通过将合并后处理后的遮蔽时间区间长度累加，计算出总有效遮蔽时长。

● 多无人机组组合评估

对经无人机最佳方案筛选后得到的三架无人机方案进行排列组合，共产生 100×100×100 种可能组合。基于时间区间合并算法对各组合的总遮蔽时长进行计算，并按总遮蔽时长降序排列，选择遮蔽时长最大的组合作为三架无人机投掷策略。

5.4.5 模型的求解

首先，依据改进后的运动模型和问题二的单无人机单干扰弹最佳遮蔽策略求解模型，分别对位于(17800,0,1800)的无人机 FY1、位于(12000,1400,1400)的无人机 FY2 和位于(6000,-3000,700)的无人机 FY3 干扰导弹 M1 的问题进行求解。得到每架无人机按有效遮蔽时间降序排列的前 100 个方案。部分方案如下图所示

表 3 无人机 FY1 前 10 个最佳方案

无人机 FY1 方向角	无人机 FY1 速度	干扰弹投放时间	干扰弹起爆延迟	有效遮蔽时间
5.625	139.75	0.45	0.4	4.645
5.475	139.875	0.575	0.275	4.644
5.5125	139.625	0.575	0.275	4.644
5.5125	139.75	0.575	0.275	4.644
5.5125	140	0.575	0.275	4.644
5.55	139.5	0.575	0.275	4.644
5.55	139.875	0.575	0.275	4.644
5.55	140	0.575	0.275	4.644
5.5875	139.625	0.45	0.4	4.644

表 4 无人机 FY2 前 10 个最佳方案

无人机 FY2 方向角	无人机 FY2 速度	干扰弹投放时间	干扰弹起爆延迟	有效遮蔽时间
293	134	5.75	5.25	4
288	124	6	5.5	3.99
291	138.5	5.25	5.25	3.99
292	136	5.5	5.25	3.99
287	129	5.5	5.5	3.98
270	138	3.5	6.25	3.97
290	138	5.25	5.25	3.97
292	139.5	5.25	5.25	3.97
292	120	7	5.25	3.96
293	137	5.5	5.25	3.96

表 5 无人机 FY3 前 10 个最佳方案

无人机 FY3 方向角	无人机 FY3 速度	干扰弹投放时间	干扰弹起爆延迟	有效遮蔽时间
74	133	23.75	0.4	3.17
76.5	131	22.5	1.75	3.16
74	131	24	0.5	3.16
74.5	128	24.25	0.75	3.15
75.5	140	22	0.775	3.14
75	133	23.5	0.5	3.14
74	122	25.5	0.775	3.14
74.5	130	24.5	0.1	3.14
74	134	23.75	0.2	3.14
80	138	20.25	2.5	3.13

接着，得到的三架无人机的方案进行排列组合。使用问题三中的重叠时间处理模型，对三架无人机的重叠遮挡进行处理，计算出每种组合的总有效遮蔽时长，并依据其进行降序排列，得到最佳方案：无人机 FY1 的方向角度为 5.625° ，飞行速度为 139.75 m/s，投放时间为 0.45s，起爆延迟为 0.4s；无人机 FY2 的方向角度为 293° ，飞行速度为 134 m/s，投放时间为 5.75s，起爆延迟为 5.25s；无人机 FY2 的方向角度为 74° ，飞行速度为 133 m/s，投放时间为 23.75s，起爆延迟为 0.4s；此时的总有效遮蔽时间为 11.82s。同时将结果填写到文件 result2.xlsx。

5.5 问题五模型的建立与求解

5.5.1 模型的建立

本题需制定多无人机投掷多烟幕干扰弹策略对多导弹进行干扰，首先，需要对运动模型进行扩展使其满足对多无人机多导弹多干扰弹情形的描述。随后，建立总遮蔽时间最大化为目标的优化模型，并对决策变量进行搜索，基于阴影遮蔽模型和重叠遮蔽模型可对对应决策变量的总遮蔽时间进行计算。该搜索过程可分成三个优化所搜阶段。首先，进行单烟幕弹参数优化，基于问题二粗略搜索每个无人机投放 1 枚烟雾弹分别对 3 枚导弹的干扰方案参数，找到这 15 种组合（FY1 对 M1 的干扰投放为其中一种组合）的部分优秀解决方案。随后，进行单无人机多烟幕弹策略优化，现基于上一步的优秀解决方案，建立优化模型，搜索出每个无人机可投放 3 枚烟雾弹时对 3 枚导弹遮蔽效果较好的策略。最后，进行全局防御策略优化，基于上一步，建立全局优化模型，将单无人机扩展到 3 架无人机，最终搜索出全局最优解。

5.5.2 单烟幕弹参数优化

本阶段建立以总遮蔽时间最大化为优化目标的单无人机单干扰弹遮蔽一枚导弹的优化模型。优化目标如下

$$\max_{(\theta, v, t_{release}, t_{delay})} T_{ji}(\theta, v, t_{release}, t_{delay}) \quad (16)$$

其中， T_{ji} 为第 j 个无人机对第 i 个导弹总遮蔽时间， θ 为无人机方向角， v 为无人机速度， $t_{release}$ 为投放时间， t_{delay} 为起爆延迟。

对于每种无人机-导弹组合(j,i)，优秀方案搜索方式为：无人机方向角搜索区间为 $[0^\circ, 360^\circ]$ ，步长为 9° ；无人机速度搜索区间为 $[70 \text{ m/s}, 140 \text{ m/s}]$ ，步长约 2.7m/s；投放时间搜索区间为 $[0, 40\text{s}]$ ，步长 4 秒；起爆延迟搜索区间为 $[0, 10\text{s}]$ ，步长 1 秒。

5.5.3 层次化优化算法

本阶段建立以总遮蔽时间最大化为优化目标的单无人机多干扰弹遮蔽一枚导弹的优化模型。优化目标如下

$$\max_{(\theta_j, v_j, \mathcal{B}_j)} f_j(\theta_j, v_j, \mathcal{B}_j) = \sum_{i=1}^3 T_{ji}(\theta_j, v_j, \mathcal{B}_j) \quad (17)$$

其中， $\mathcal{B}_j = \{(t_{release}^k, t_{delay}^k, m_{target}^k)\}_{k=1}^{n_j}$ 表示无人j的烟幕弹参数集合。

约束条件如下：

$$s.t. \begin{cases} 1 \leq n_j \leq 3 \\ \forall k \in \{1, 2, \dots, n_j - 1\}, t_{release}^{k+1} - t_{release}^k \geq 1s \end{cases} \quad (18)$$

首先，由于无人机速度方向一经确认无法改变，故同一无人机的所有烟幕干扰弹必须具有相同的飞行方向和速度，我们首先构建方向-速度候选集 D_j

$$D_j = \{(\theta, v) \mid \exists s \in \bigcup_{i=1}^3 S_{ji}, s.\theta = \theta \wedge s.v = v\} \quad (19)$$

其中 S_{ji} 是第一层优化得到的无人机j对导弹i的解决方案集合。

通过这种方式，我们将原本四维的参数空间 $\Theta \times V \times T_{release} \times T_{delay}$ 降维为由离散点组成的集合 D_j ，减小了搜索空间。

其次，对 D_j 中的方向-速度组合进行优化，步骤如下：

Step1: 找每个导弹出在给定方向和速度下的最佳参数

$$p_{ji}^*(\theta, v) = \arg \max_{s \in S_{ji}, |s.\theta - \theta| < \epsilon, |s.v - v| < \epsilon} s.T \quad (20)$$

其中 ϵ 是容差值， $s.T$ 是对应解决方案的有效遮蔽时长。

由此得到最佳参数映射 $M_j(\theta, v) = \{(i, p_{ji}^*(\theta, v)) \mid i \in \{1, 2, 3\}, p_{ji}^*(\theta, v) \text{ 存在}\}$

Step2: 枚举所有可能的目标导弹组合 $C \subseteq \{i \mid (i, p) \in M_j(\theta, v)\}$ 且 $1 \leq |C| \leq 3$ ，并对其构建烟幕弹参数集合 $\mathcal{B}_j(C, \theta, v) = \{(p_{ji}^*(\theta, v).t_{release}, p_{ji}^*(\theta, v).t_{delay}, i) \mid i \in C\}$ 。随后条序投放时间，使其满足干扰弹投放间隔不小于 1s，现将集合 $\mathcal{B}_j(C, \theta, v)$ 按投放时间升序排序，若排序后的相邻两烟幕弹投放间隔时间小于 1s，对后一个烟幕弹投放时间进行如下调整

$$t_{release}^{k+1} = \max(t_{release}^{k+1}, t_{release}^k + 1) \quad (21)$$

我们计算每个烟雾弹的遮蔽区间集合 I_{ji} ，将合并同一导弹区间，计算其有效遮蔽时长 T_{ji} ，故总效果为 $f_j(\theta, v, \mathcal{B}_j(C, \theta, v)) = \sum_{i \in C} T_{ji}$ 。并将生成的组合作为一个候选策略加

入到无人机 j 的策略集合 P_j 中，即 $P_j = \{(\theta, v, \mathcal{B}_j(C, \theta, v)) \mid (\theta, v) \in D_j, C \text{ 是有效组合}\}$ 。

5.5.4 全局防御策略优化

本阶段考虑所有无人机对三个导弹共同作用的遮蔽效果，构建全局优化模型，其优化目标如下

$$\max_{(p_1, p_2, p_3, p_4, p_5) \in P_1 \times P_2 \times P_3 \times P_4 \times P_5} F(p_1, p_2, p_3, p_4, p_5) = \sum_{i=1}^3 T_i(p_1, p_2, p_3, p_4, p_5) \quad (22)$$

约束条件为

$$\forall i \in \{1, 2, 3\}, T_i(p_1, p_2, p_3, p_4, p_5) > 0 \quad (23)$$

其中，其中 $p_j \in P_j$ 表示无人机 j 的策略， T_i 表示导弹 i 的总有效遮蔽时长。

为计算多烟幕弹对同一导弹的总有效遮蔽时长，需要正确进行重叠区间的处理，我们基于问题三重叠时间处理模型对其进行处理。设所有无人机对导弹 M_i 的遮蔽区间集

合为 $I_i = \bigcup_{j=1}^5 \bigcup_{k: m_{target}^{jk} = i} I_{jk}$ ，其中 I_{jk} 是无人机 j 的第 k 个烟幕弹对导弹 i 的遮蔽区间集合。

经区间合并得到区间集合 $I_i' = \{[a_1', b_1'], [a_2', b_2'], \dots, [a_{m_i}', b_{m_i}']\}$ 。故导弹 i 的总有效遮蔽

时长为 $T_i = \sum_{l=1}^{m_i} (b_l' - a_l')$ ，全局防御效果即所有无人机对三个导弹的总遮蔽时长如下

$$F(p_1, p_2, p_3, p_4, p_5) = \sum_{i=1}^3 T_i \quad (24)$$

5.5.5 最优策略选择

在实际应用中，策略选择除了关注总遮蔽时长最大化外，还需关均衡性。因此，在选择最优策略时，我们采用以下多目标权衡方法

设定平衡指标为 $CV = \frac{\sigma}{\mu}$ ，其中 $\sigma = \sqrt{\frac{1}{3} \sum_{i=1}^3 (T_i - \mu)^2}$ ， $\mu = \frac{1}{3} \sum_{i=1}^3 T_i$ ，平衡因子为

$B = \frac{1}{1 + CV} \in [0, 1]$ ，平衡因子越接近 1，说明该投掷策略越均衡。设定平衡遮蔽时间

最大化和均衡性的综合得分为 $S = F \times (1 + \alpha \times B)$ ，其中 α 为平衡性权重，通常采用 0.5。

对于多目标选择权重策略的选择，首选筛选出满足 $T_i > 0$ 的策略，在选择其中综合得分 S 最高的策略为最佳策略。

5.5.6 模型的求解

经上述分层分析求解后得到的最优烟幕干扰弹的投放策略为：第一枚干扰弹于 (17800, 0, 1800) 起爆，第二枚干扰弹于 (17939.49, 11.91, 0, 1800) 起爆，第三枚干扰弹于 (12098.61, 429.20, 1349.82) 起爆，第四枚干扰弹于 (6388.23, -128.57, 684.12) 起爆，第五枚干扰弹于 (11733.02, 398.94, 1310) 起爆，第六枚干扰弹于 (12482.04, 32.03, 1255.9) 起爆。遮蔽总时长为 21.11。（精确到小数点后第二位）

六、模型的分析与检验

针对问题二中求出的无人机FY2对导弹M1投放干扰弹的最优参数(方向角 136.5° , 速度 0.3 m/s , 投放时间 0.5 s , 起爆延迟 0.54 s)，我们固定其他参数不变，仅改变其中一个参数，观察其对有效遮蔽时长的影响。结果如下：

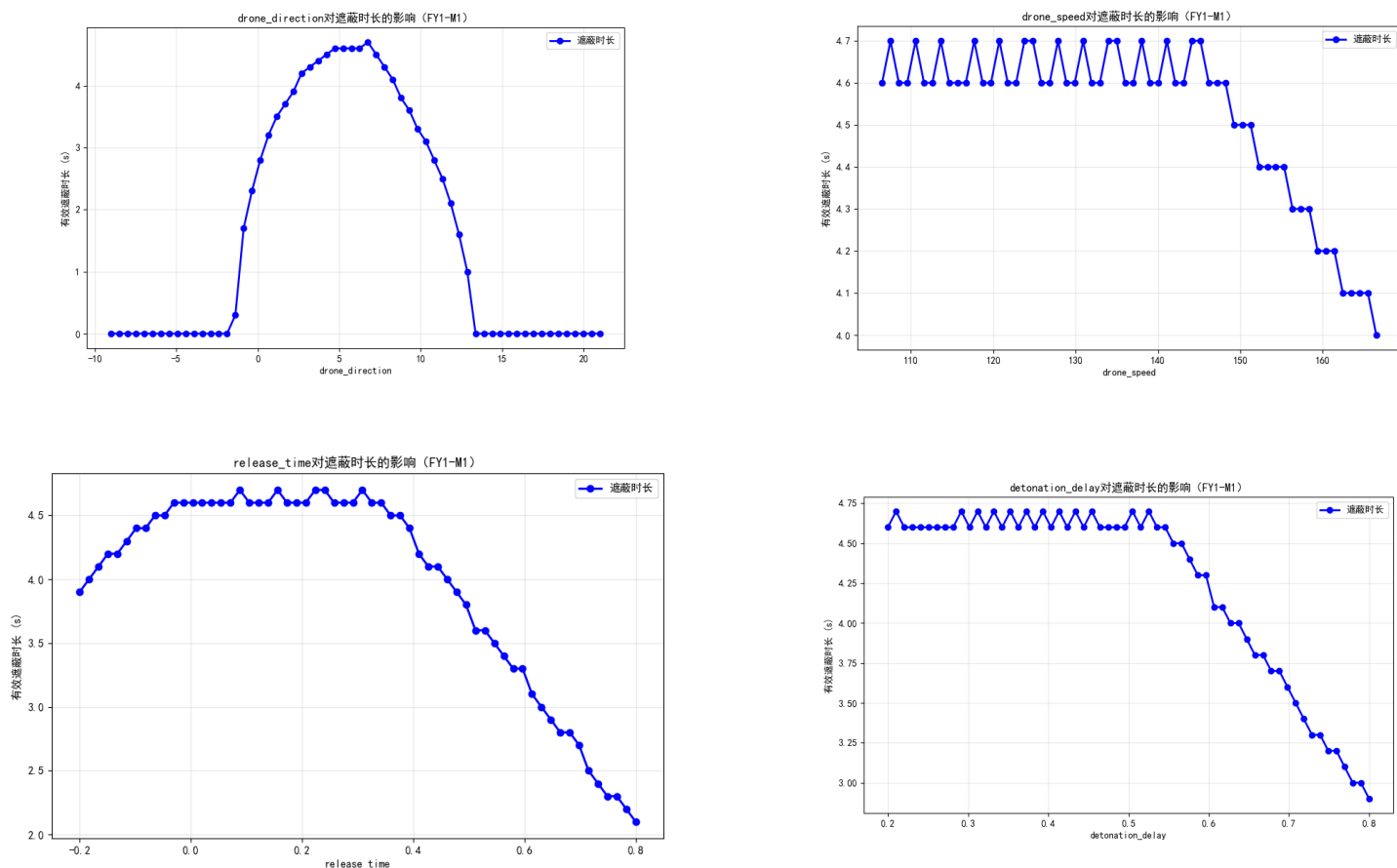


图 5 不同参数对有效遮蔽时间的影响

从上述单参数分析的结果可以看出来，在最优值附近，无人机角度对遮蔽时间的影响显著，而飞行速度、释放时间与爆炸延时对遮蔽时间的影响较弱。

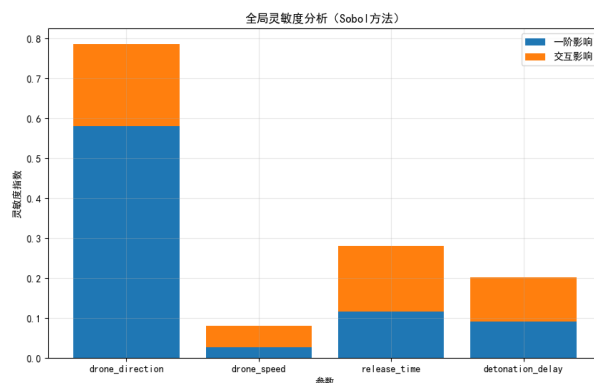


图 6 全局灵敏度分析图

同时，我们也对各参数进行了全局灵敏度分析。通过单参数与全局灵敏度分析发现，

无人机飞行方向对遮蔽时长的影响最大（一阶指数 0.58，总阶指数 0.79），而速度的影响可忽略（总阶指数 0.08）。投放时间与起爆延迟的交互作用显著（交互贡献率分别为 16.4%和 11.1%），需在策略优化中联合调整。

七、模型的评价、改进与推广

7.1 模型的优点

1. 我们的模型可拓展性强，适应范围广，针对更复杂的情况，如更多无人机、更多干扰弹、更多导弹，只需要简单的调整参数即可。
2. 采用全局搜索加局部精细化搜索的分阶段优化策略，显著降低了高维参数空间的复杂度。
3. 基于搜索空间全局遍历的策略预选种子，极大的加快了遗传算法收敛的速度，因此模型计算高效。
4. 对无效参数做了谨慎处理，模型的鲁棒性高

7.2 模型的缺点

1. 本模型基于遗传算法，无法做到百分百的复现性。
2. 模型的及时性较差，无法做到战场实时响应

7.3 模型的改进

1. 可以引入神经网络替代部分耗时的遍历操作。
2. 采用自适应遗传算法，摆脱对变异率、交叉率等超参数的依赖。

7.4 模型的推广

模型可以推广至消防与农业领域，用于灭火或农业无人机的调度，解决多智能体多任务协同处理问题。

八、参考文献

- [1] 陈浩,高欣宝,李天鹏,等.国外烟幕干扰弹发展及关键技术研究[J].飞航导弹,2017,(12):71-74.DOI:10.16338/j.issn.1009-1319.2017.12.16.
- [2] 吴鸿超,梁增友,邓德志,等.烟幕干扰弹新型战斗部结构研究[J].科学技术与工程,2016,16(01):183-187.
- [3] 李为,刘宏程,高晓宽,等.基于 DeepSeek 大模型的军事指挥能力提升路径研究:技术优势与战场应用展望[C].第十三届中国指挥控制大会论文集(上册).第十三届中国指挥控制大会论文集(上册):中国指挥与控制学会,2025:318-322.
- [4] Claude,Claude-3.7-Sonnet-Thinking, Anthropic, 2025-09-04 到 2025-09-07

九、附录

附录 1

介绍：支撑材料的文件列表

1. 1/prob1.py 问题一代码
2. 2/prob2.py 问题二代码
3. 2/all_fine_tuned_solutions.xlsx 问题二精细化搜索后全部方案
4. 3/prob3.py 问题三代码
5. 3/ga_generation_best_records.xlsx 问题三遗传算法中每代最佳个体
6. 4/prob4.py 问题四代码
7. 4/FY1.xlsx 问题四组合时 FY1 的候选方案
8. 4/FY2.xlsx 问题四组合时 FY2 的候选方案
9. 4/FY3.xlsx 问题四组合时 FY3 的候选方案
10. 5/prob5.py 问题五的搜索代码
11. 5/combine.py 问题五的组合代码
12. 5/analysis.py 问题五的详细求解和可视化代码
13. 5/FY*_Missile*_results 问题五搜索方案的文件夹系
14. 5/defense_animation.mp4 问题五防守演示动画
15. 5/shielding_timeline.png 问题五遮蔽时间线
16. result1.xlsx 问题三答案
17. result2.xlsx 问题四答案
18. result3.xlsx 问题五答案

附录 2

问题一代码

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.animation as animation
from matplotlib.patches import Circle
from matplotlib import cm

from scipy.spatial.distance import cdist

import matplotlib.pyplot as plt
from matplotlib import font_manager

plt.rcParams['font.sans-serif'] = ['WenQuanYi Zen Hei']
```

```

# 常量定义

MISSILE_SPEED = 300 # m/s

DRONE_SPEED_RANGE = (70, 140) # m/s

SMOKE_DESCENT_RATE = 3 # m/s

SMOKE_EFFECTIVE_RADIUS = 10 # m

SMOKE_EFFECTIVE_TIME = 20 # s

GRAVITY = 9.8 # m/s^2


# 目标和初始位置信息

FAKE_TARGET = np.array([0, 0, 0])

REAL_TARGET_CENTER = np.array([0, 200, 5]) # 圆柱体中心点

REAL_TARGET_RADIUS = 7 # m

REAL_TARGET_HEIGHT = 10 # m


MISSILE_M1_INIT = np.array([20000, 0, 2000])

DRONE_FY1_INIT = np.array([17800, 0, 1800])


def normalize(v):
    """归一化向量"""
    norm = np.linalg.norm(v)
    if norm == 0:
        return v
    return v / norm


def missile_trajectory(init_pos, target_pos, speed, t):
    """计算导弹在时间 t 的位置"""
    direction = normalize(target_pos - init_pos)
    return init_pos + direction * speed * t


def drone_trajectory(init_pos, direction_deg, speed, t):
    """计算无人机在时间 t 的位置"""
    # 将角度转换为弧度
    direction_rad = np.radians(direction_deg)
    # 计算方向向量 (x 轴正方向为 0 度, 逆时针为正)
    direction_vector = np.array([np.cos(direction_rad), np.sin(direction_rad), 0])
    return init_pos + direction_vector * speed * t


def smoke_trajectory(release_pos, release_time, direction_deg, speed, t):
    """计算烟幕弹在时间 t 的位置 (考虑重力)"""
    # 将角度转换为弧度
    direction_rad = np.radians(direction_deg)
    # 计算方向向量 (x 轴正方向为 0 度, 逆时针为正)
    direction_vector = np.array([np.cos(direction_rad), np.sin(direction_rad), 0])
    if t < release_time:

```

```

        return None # 未释放

    dt = t - release_time

    # 烟幕弹初始速度与无人机相同

    return release_pos + np.array([0, 0, -0.5 * GRAVITY * dt**2]) + direction_vector * speed * dt

def smoke_cloud_trajectory(detonation_pos, detonation_time, t):
    """计算烟幕云团在时间 t 的位置"""

    if t < detonation_time:

        return None # 未爆炸

    dt = t - detonation_time

    return detonation_pos + np.array([0, 0, -SMOKE_DESCENT_RATE * dt])

def distance_point_to_line(point, line_start, line_end):
    """计算点到直线的距离"""

    if np.array_equal(line_start, line_end):

        return np.linalg.norm(point - line_start)

    line_vec = line_end - line_start
    point_vec = point - line_start
    line_len = np.linalg.norm(line_vec)
    line_unitvec = line_vec / line_len
    point_vec_scaled = point_vec / line_len

    t = np.dot(line_unitvec, point_vec_scaled)
    t = max(0, min(1, t)) # 确保 t 在[0,1]范围内

    nearest = line_start + t * line_vec
    return np.linalg.norm(point - nearest)

def is_target_in_shadow_cone(missile_pos, cloud_pos, target_pos, target_radius, target_height):
    """
    检查目标是否在烟幕云团投射的阴影锥体内

    参数:
    missile_pos: 导弹位置
    cloud_pos: 烟幕云团中心位置
    target_pos: 目标底面中心位置
    target_radius: 目标半径
    target_height: 目标高度

    返回:
    bool: 如果目标完全在阴影锥体内, 则返回 True
    """

```

```

# if missile_pos[0]+10 < cloud_pos[0]:
#     return False
# 计算烟雾云团到导弹的方向向量
direction = missile_pos - cloud_pos
distance_missile_cloud = np.linalg.norm(direction)

if distance_missile_cloud < SMOKE_EFFECTIVE_RADIUS:
    # 如果导弹在云团内，肯定遮蔽成功
    return True

# 单位方向向量
unit_direction = direction / distance_missile_cloud

# 计算从导弹到烟雾球体的切线所形成的锥角
#  $\sin(\theta) = r/d$ , 其中  $r$  是球体半径,  $d$  是导弹到球体中心的距离
sin_theta = min(1.0, SMOKE_EFFECTIVE_RADIUS / distance_missile_cloud)
cos_theta = np.sqrt(1 - sin_theta**2)

# 检查目标圆柱体的每个关键点是否在阴影锥体内
# 关键点: 底面圆周上的点和顶面圆周上的点
num_points = 12 # 圆周上的采样点数
angles = np.linspace(0, 2*np.pi, num_points, endpoint=False)

# 底面圆周上的点
bottom_points = []
for angle in angles:
    x = target_pos[0] + target_radius * np.cos(angle)
    y = target_pos[1] + target_radius * np.sin(angle)
    z = target_pos[2]
    bottom_points.append([x, y, z])

# 顶面圆周上的点
top_points = []
for angle in angles:
    x = target_pos[0] + target_radius * np.cos(angle)
    y = target_pos[1] + target_radius * np.sin(angle)
    z = target_pos[2] + target_height
    top_points.append([x, y, z])

# 合并所有点
all_points = np.vstack([bottom_points, top_points])

# 检查每个点是否在阴影锥体内

```



```

for point in all_points:

    # 从导弹到目标点的向量
    missile_to_point = point - missile_pos
    distance = np.linalg.norm(missile_to_point)

    if distance < 1e-10: # 防止除以零
        continue

    # 计算该向量与导弹到云团方向的夹角余弦值
    cos_angle = np.dot(missile_to_point, unit_direction) / distance
    if cos_angle > 0:
        # 如果夹角大于 90 度, 说明点在导弹的背后, 不在阴影锥体内
        return False
    # 取绝对值
    cos_angle = abs(cos_angle)
    cos_theta = abs(cos_theta)

    # 如果夹角余弦值小于锥角余弦值, 说明该点不在阴影锥体内
    if cos_angle < cos_theta:

        #打印相关的全部信息
        if distance_missile_cloud < 700:
            print("导弹位置:", missile_pos)
            print("云团位置:", cloud_pos)
            print("cos:", cos_angle, cos_theta)
            #print("目标点不在阴影锥体内:", point)
        return False

    # 所有点都在阴影锥体内, 目标被完全遮蔽
    return True

def calculate_shielding_effectiveness(missile_pos, cloud_pos, cloud_start_time, current_time):
    """
    计算烟幕对导弹的遮蔽效果

    参数:
    missile_pos: 导弹位置
    cloud_pos: 烟幕云团中心位置
    cloud_start_time: 烟幕云团形成时间
    current_time: 当前时间

    返回:
    int: 1 表示有效遮蔽, 0 表示无效遮蔽

```

```

"""

# 检查烟雾云团是否有效

if cloud_pos is None or current_time - cloud_start_time > SMOKE_EFFECTIVE_TIME:

    return 0 # 未形成云团或云团已失效


# 检查真目标是否在阴影锥体内
target_in_shadow = is_target_in_shadow_cone(

    missile_pos,

    cloud_pos,

    REAL_TARGET_CENTER,

    REAL_TARGET_RADIUS,

    REAL_TARGET_HEIGHT

)


return 1 if target_in_shadow else 0


def visualize_problem1():

    """可视化问题 1 的解决方案"""

    # 问题 1 参数

    drone_speed = 120 # m/s

    # 计算无人机方向 (朝向假目标)

    direction_vec = FAKE_TARGET - DRONE_FY1_INIT

    direction_vec_xy = np.array([direction_vec[0], direction_vec[1], 0])


    print("无人机方向: ", direction_vec_xy)

    # 计算与 x 轴正方向的夹角 (度数)

    angle = np.degrees(np.arctan2(direction_vec_xy[1], direction_vec_xy[0]))

    if angle < 0:

        angle += 360


    # 1.5 秒后投放, 3.6 秒后起爆

    release_time = 1.5

    detonation_time = release_time + 3.6


    # 计算投放点

    drone_release_pos = drone_trajectory(DRONE_FY1_INIT, angle, drone_speed, release_time)

    # 打印投放点坐标

    print(f"问题 1: 投放点坐标: ({drone_release_pos[0]:.2f}, {drone_release_pos[1]:.2f}, {drone_release_pos[2]:.2f})")


    # 计算起爆点

    smoke_bomb_pos = smoke_trajectory(drone_release_pos, release_time, angle, drone_speed, detonation_time)

```

```

# 打印起爆点坐标
print(f'问题 1: 起爆点坐标: ({smoke_bomb_pos[0]:.2f}, {smoke_bomb_pos[1]:.2f}, {smoke_bomb_pos[2]:.2f})')

# 模拟导弹和烟幕的轨迹
simulation_time = np.linspace(0, 15, 100000)
missile_positions = []
drone_positions = []
smoke_bomb_positions = []
smoke_cloud_positions = []
shielding_effectiveness = []

for t in simulation_time:
    # 导弹位置
    missile_pos = missile_trajectory(MISSILE_M1_INIT, FAKE_TARGET, MISSILE_SPEED, t)
    missile_positions.append(missile_pos)

    # 无人机位置
    drone_pos = drone_trajectory(DRONE_FY1_INIT, angle, drone_speed, t)
    drone_positions.append(drone_pos)

    # 烟幕弹位置
    if t >= release_time and t < detonation_time:
        bomb_pos = smoke_trajectory(drone_release_pos, release_time, angle, drone_speed, t)
        smoke_bomb_positions.append(bomb_pos)

    # 烟幕云团位置
    if t >= detonation_time:
        cloud_pos = smoke_cloud_trajectory(smoke_bomb_pos, detonation_time, t)
        smoke_cloud_positions.append((t, cloud_pos))

        # visualize_shadow_cone(missile_pos, cloud_pos) # 可视化阴影锥体

        # 计算有效性
        effectiveness = calculate_shielding_effectiveness(
            missile_pos, cloud_pos, detonation_time, t)
        shielding_effectiveness.append((t, effectiveness))

# 计算有效遮蔽时长
effective_times = [(t, eff) for t, eff in shielding_effectiveness if eff > 0]
if effective_times:
    start_time = effective_times[0][0]
    end_time = effective_times[-1][0]

    print(f'问题 1: 有效遮蔽时间段: {start_time:.5f} 秒 到 {end_time:.5f} 秒')
    effective_duration = end_time - start_time

```

```

        print(f'问题 1: 有效遮蔽时长约为 {effective_duration:.5f} 秒')
    else:
        print("问题 1: 无有效遮蔽")

# 创建 3D 图
fig = plt.figure(figsize=(12, 10))
ax = fig.add_subplot(111, projection='3d')

# 绘制假目标
ax.scatter([0], [0], [0], color='red', s=100, label='假目标')

# 绘制真目标（圆柱体）
theta = np.linspace(0, 2*np.pi, 100)
z = np.linspace(0, REAL_TARGET_HEIGHT, 10)
theta_grid, z_grid = np.meshgrid(theta, z)
x_cylinder = REAL_TARGET_CENTER[0] + REAL_TARGET_RADIUS * np.cos(theta_grid)
y_cylinder = REAL_TARGET_CENTER[1] + REAL_TARGET_RADIUS * np.sin(theta_grid)
ax.plot_surface(x_cylinder, y_cylinder, z_grid, alpha=0.5, color='blue')

# 绘制导弹轨迹
missile_positions = np.array(missile_positions)
ax.plot(missile_positions[:, 0], missile_positions[:, 1], missile_positions[:, 2],
        color='red', label='导弹 M1 轨迹')
ax.scatter(MISSILE_M1_INIT[0], MISSILE_M1_INIT[1], MISSILE_M1_INIT[2],
        color='darkred', label='导弹 M1 初始位置')

# 绘制无人机轨迹
drone_positions = np.array(drone_positions)
ax.plot(drone_positions[:, 0], drone_positions[:, 1], drone_positions[:, 2],
        color='green', label='无人机 FY1 轨迹')
ax.scatter(DRONE_FY1_INIT[0], DRONE_FY1_INIT[1], DRONE_FY1_INIT[2],
        color='darkgreen', label='无人机 FY1 初始位置')

# 绘制烟雾弹投放点
ax.scatter(drone_release_pos[0], drone_release_pos[1], drone_release_pos[2],
        color='orange', s=80, label='烟雾弹投放点')

# 绘制烟雾弹起爆点
ax.scatter(smoke_bomb_pos[0], smoke_bomb_pos[1], smoke_bomb_pos[2],
        color='purple', s=80, label='烟雾弹起爆点')

# 绘制有效烟雾区域（只绘制特定时间点的）
if smoke_cloud_positions:
    # 每 5 个时间点绘制一次云团

```

```

for i in range(0, len(smoke_cloud_positions), 50):

    t, cloud_pos = smoke_cloud_positions[i]

    if t - detonation_time <= SMOKE_EFFECTIVE_TIME: # 只绘制有效期内的云团

        # 绘制球体表示烟雾云团

        u, v = np.mgrid[0:2*np.pi:20j, 0:np.pi:10j]

        x = cloud_pos[0] + SMOKE_EFFECTIVE_RADIUS * np.cos(u) * np.sin(v)

        y = cloud_pos[1] + SMOKE_EFFECTIVE_RADIUS * np.sin(u) * np.sin(v)

        z = cloud_pos[2] + SMOKE_EFFECTIVE_RADIUS * np.cos(v)

        ax.plot_wireframe(x, y, z, color='gray', alpha=0.2)

# 选择一个关键时间点进行阴影锥体可视化

key_time_index = len(smoke_cloud_positions) // 2

key_time, key_cloud_pos = smoke_cloud_positions[key_time_index]

key_missile_pos = missile_trajectory(MISSILE_M1_INIT, FAKE_TARGET, MISSILE_SPEED, key_time)

# 可视化阴影锥体

#visualize_shadow_cone(key_missile_pos, key_cloud_pos, ax)

# 设置坐标轴范围和标签

ax.set_xlim([16000, 20000])

ax.set_ylim([-200, 200])

ax.set_zlim([1500, 2000])

ax.set_xlabel('X 轴 (m)')

ax.set_ylabel('Y 轴 (m)')

ax.set_zlabel('Z 轴 (m)')

ax.set_title('问题 1: 烟雾干扰弹投放策略可视化')

# 添加图例

ax.legend()

plt.tight_layout()

plt.savefig('problem1_visualization.png', dpi=300)

plt.show()

if __name__ == "__main__":

    # 运行问题 1 的可视化

    visualize_problem1()

```

附录 3

问题二代码

```

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.animation as animation
from matplotlib.patches import Circle
from matplotlib import cm

from scipy.spatial.distance import cdist

import matplotlib.pyplot as plt
from matplotlib import font_manager
import time

import pandas as pd
import os
plt.rcParams['font.sans-serif'] = ['WenQuanYi Zen Hei']

# 常量定义
MISSILE_SPEED = 300 # m/s
DRONE_SPEED_RANGE = (70, 140) # m/s
SMOKE_DESCENT_RATE = 3 # m/s
SMOKE_EFFECTIVE_RADIUS = 10 # m
SMOKE_EFFECTIVE_TIME = 20 # s
GRAVITY = 9.8 # m/s^2

# 目标和初始位置信息
FAKE_TARGET = np.array([0, 0, 0])
REAL_TARGET_CENTER = np.array([0, 200, 5]) # 圆柱体中心点
REAL_TARGET_RADIUS = 7 # m
REAL_TARGET_HEIGHT = 10 # m

MISSILE_M1_INIT = np.array([20000, 0, 2000])
DRONE_FY1_INIT = np.array([17800, 0, 1800])

def normalize(v):
    """归一化向量"""
    norm = np.linalg.norm(v)
    if norm == 0:
        return v
    return v / norm

def missile_time_to_target(init_pos, target_pos, speed):

```

```

        """计算导弹到达目标的时间"""

        distance = np.linalg.norm(target_pos - init_pos)

        return distance / speed

def missile_trajectory(init_pos, target_pos, speed, t):
    """计算导弹在时间 t 的位置"""

    direction = normalize(target_pos - init_pos)

    return init_pos + direction * speed * t

def drone_trajectory(init_pos, direction_deg, speed, t):
    """计算无人机在时间 t 的位置"""

    # 将角度转换为弧度

    direction_rad = np.radians(direction_deg)

    # 计算方向向量 (x 轴正方向为 0 度, 逆时针为正)

    direction_vector = np.array([np.cos(direction_rad), np.sin(direction_rad), 0])

    return init_pos + direction_vector * speed * t

def smoke_trajectory(release_pos, release_time, direction_deg, speed, t):
    """计算烟幕弹在时间 t 的位置 (考虑重力)"""

    # 将角度转换为弧度

    direction_rad = np.radians(direction_deg)

    # 计算方向向量 (x 轴正方向为 0 度, 逆时针为正)

    direction_vector = np.array([np.cos(direction_rad), np.sin(direction_rad), 0])

    if t < release_time:

        return None # 未释放

    dt = t - release_time

    # 烟幕弹初始速度与无人机相同

    return release_pos + np.array([0, 0, -0.5 * GRAVITY * dt**2]) + direction_vector * speed * dt

def smoke_cloud_trajectory(detonation_pos, detonation_time, t):
    """计算烟幕云团在时间 t 的位置"""

    if t < detonation_time:

        return None # 未爆炸

    dt = t - detonation_time

    return detonation_pos + np.array([0, 0, -SMOKE_DESCENT_RATE * dt])

def is_target_in_shadow_cone(missile_pos, cloud_pos, target_pos, target_radius, target_height):
    """

    检查目标是否在烟幕云团投射的阴影锥体内

    参数:

    missile_pos: 导弹位置

    cloud_pos: 烟幕云团中心位置

```

```

target_pos: 目标底面中心位置
target_radius: 目标半径
target_height: 目标高度

返回:

bool: 如果目标完全在阴影锥体内, 则返回 True
"""

if missile_pos[0]+10 < cloud_pos[0]:
    return False

# 计算烟幕云团到导弹的方向向量
direction = missile_pos - cloud_pos
distance_missile_cloud = np.linalg.norm(direction)

if distance_missile_cloud < SMOKE_EFFECTIVE_RADIUS:
    # 如果导弹在云团内, 肯定遮蔽成功
    return True

# 单位方向向量
unit_direction = direction / distance_missile_cloud

# 计算从导弹到烟幕球体的切线所形成的锥角
#  $\sin(\theta) = r/d$ , 其中  $r$  是球体半径,  $d$  是导弹到球体中心的距离
sin_theta = min(1.0, SMOKE_EFFECTIVE_RADIUS / distance_missile_cloud)
cos_theta = np.sqrt(1 - sin_theta**2)

# 检查目标圆柱体的每个关键点是否在阴影锥体内
# 关键点: 底面圆周上的点和顶面圆周上的点
num_points = 12 # 圆周上的采样点数
angles = np.linspace(0, 2*np.pi, num_points, endpoint=False)

# 底面圆周上的点
bottom_points = []
for angle in angles:
    x = target_pos[0] + target_radius * np.cos(angle)
    y = target_pos[1] + target_radius * np.sin(angle)
    z = target_pos[2]
    bottom_points.append([x, y, z])

# 顶面圆周上的点
top_points = []
for angle in angles:
    x = target_pos[0] + target_radius * np.cos(angle)
    y = target_pos[1] + target_radius * np.sin(angle)

```



```

        z = target_pos[2] + target_height
        top_points.append([x, y, z])

# 合并所有点
all_points = np.vstack([bottom_points, top_points])

# 检查每个点是否在阴影锥体内
for point in all_points:
    # 从导弹到目标点的向量
    missile_to_point = point - missile_pos
    distance = np.linalg.norm(missile_to_point)

    if distance < 1e-10: # 防止除以零
        continue

    # 计算该向量与导弹到云团方向的夹角余弦值
    cos_angle = np.dot(missile_to_point, unit_direction) / distance
    if cos_angle > 0:
        # 如果夹角大于 90 度, 说明点在导弹的背后, 不在阴影锥体内
        return False
    # 取绝对值
    cos_angle = abs(cos_angle)
    cos_theta = abs(cos_theta)

    # 如果夹角余弦值小于锥角余弦值, 说明该点不在阴影锥体内
    if cos_angle < cos_theta:

        #打印相关的全部信息
        # if distance_missile_cloud < 700:
        #     print("导弹位置:", missile_pos)
        #     print("云团位置:", cloud_pos)
        #     print("cos:", cos_angle, cos_theta)
        #     #print("目标点不在阴影锥体内:", point)
        return False

# 所有点都在阴影锥体内, 目标被完全遮蔽
return True

def calculate_shielding_effectiveness(missile_pos, cloud_pos, cloud_start_time, current_time):
    """计算烟幕对导弹的遮蔽效果"""
    if cloud_pos is None or current_time - cloud_start_time > SMOKE_EFFECTIVE_TIME:
        return 0 # 未形成云团或云团已失效

```

```

# 检查真目标是否被烟幕遮蔽
target_shielded = is_target_in_shadow_cone(
    missile_pos, cloud_pos, REAL_TARGET_CENTER, REAL_TARGET_RADIUS, REAL_TARGET_HEIGHT
)

return 1 if target_shielded else 0

def calculate_effective_shielding_time(drone_direction, drone_speed, release_time, detonation_delay):
    """计算指定参数下的有效遮蔽时长"""
    detonation_time = release_time + detonation_delay

    # 计算投放点
    release_pos = drone_trajectory(DRONE_FY1_INIT, drone_direction, drone_speed, release_time)

    # 计算起爆点
    detonation_pos = smoke_trajectory(release_pos, release_time, drone_direction, drone_speed, detonation_time)

    limit_time = detonation_time + SMOKE_EFFECTIVE_TIME

    # 模拟导弹和烟幕的轨迹
    effective_duration = 0
    in_cloud = False
    start_time = None

    # 时间步长
    time_step = 0.01

    # 遍历导弹飞行的整个过程
    for t in np.arange(0, limit_time, time_step):
        missile_pos = missile_trajectory(MISSILE_M1_INIT, FAKE_TARGET, MISSILE_SPEED, t)

        if t >= detonation_time and t - detonation_time <= SMOKE_EFFECTIVE_TIME:
            cloud_pos = smoke_cloud_trajectory(detonation_pos, detonation_time, t)

            if cloud_pos is not None:
                is_effective = calculate_shielding_effectiveness(
                    missile_pos, cloud_pos, detonation_time, t
                )

                if is_effective and not in_cloud:
                    in_cloud = True
                    start_time = t
                elif not is_effective and in_cloud:
                    in_cloud = False

```

```

        effective_duration += (t - start_time)

# 检查如果结束时仍在云团中
if in_cloud:
    end_time = min(missile_total_time, detonation_time + SMOKE_EFFECTIVE_TIME)
    effective_duration += (end_time - start_time)

return {
    'effective_duration': effective_duration,
    'release_pos': release_pos,
    'detonation_pos': detonation_pos,
    'detonation_time': detonation_time
}

def calculate_effective_shielding_time_2(drone_direction, drone_speed, release_time, detonation_delay):
    """计算指定参数下的有效遮蔽时长"""
    detonation_time = release_time + detonation_delay

    # 计算投放点
    release_pos = drone_trajectory(DRONE_FY1_INIT, drone_direction, drone_speed, release_time)

    # 计算起爆点
    detonation_pos = smoke_trajectory(release_pos, release_time, drone_direction, drone_speed, detonation_time)

    limit_time = detonation_time + SMOKE_EFFECTIVE_TIME

    # 模拟导弹和烟幕的轨迹
    effective_duration = 0
    in_cloud = False
    start_time = None

    # 时间步长
    time_step = 0.001

    # 遍历导弹飞行的整个过程
    for t in np.arange(0, limit_time, time_step):
        missile_pos = missile_trajectory(MISSILE_M1_INIT, FAKE_TARGET, MISSILE_SPEED, t)

        if t >= detonation_time and t - detonation_time <= SMOKE_EFFECTIVE_TIME:
            cloud_pos = smoke_cloud_trajectory(detonation_pos, detonation_time, t)

            if cloud_pos is not None:
                is_effective = calculate_shielding_effectiveness(

```

```

        missile_pos, cloud_pos, detonation_time, t
    )

    if is_effective and not in_cloud:
        in_cloud = True
        start_time = t

    elif not is_effective and in_cloud:
        in_cloud = False
        effective_duration += (t - start_time)

# 检查如果结束时仍在云团中
if in_cloud:
    end_time = min(missile_total_time, detonation_time + SMOKE_EFFECTIVE_TIME)
    effective_duration += (end_time - start_time)

return {
    'effective_duration': effective_duration,
    'release_pos': release_pos,
    'detonation_pos': detonation_pos,
    'detonation_time': detonation_time
}

def optimize_problem2():
    """优化问题 2: 寻找最佳烟幕干扰弹投放策略, 并保存所有解决方案"""
    print("正在优化问题 2...")
    start_time = time.time()

    # 创建存储所有解决方案的列表
    all_solutions = []

    # 搜索参数空间
    best_params = None
    best_duration = 0

    # 创建结果目录
    results_dir = "optimization_results"
    if not os.path.exists(results_dir):
        os.makedirs(results_dir)

    directions = np.linspace(3, 9, 41) #41

    # 搜索空间
    for direction in directions:
        start_direction_time = time.time()
        for speed in np.linspace(90, 140, 26): #26# 70 到 140 m/s, 每 2.5 m/s 搜索一次

```

```

for release_time in np.linspace(0, 1.5, 11): #11
    for detonation_delay in np.linspace(0, 1, 11): #11
        if release_time + detonation_delay > 7:
            continue

        result = calculate_effective_shielding_time(
            direction, speed, release_time, detonation_delay
        )

        # 将当前解决方案添加到列表中(只保存有效时长大于 4 的方案)
        if result['effective_duration'] > 4:
            solution = {
                'direction': direction,
                'speed': speed,
                'release_time': release_time,
                'detonation_delay': detonation_delay,
                'release_pos_x': result['release_pos'][0],
                'release_pos_y': result['release_pos'][1],
                'release_pos_z': result['release_pos'][2],
                'detonation_pos_x': result['detonation_pos'][0],
                'detonation_pos_y': result['detonation_pos'][1],
                'detonation_pos_z': result['detonation_pos'][2],
                'effective_duration': result['effective_duration']
            }
            all_solutions.append(solution)

        if result['effective_duration'] > best_duration:
            best_duration = result['effective_duration']
            best_params = {
                'direction': direction,
                'speed': speed,
                'release_time': release_time,
                'detonation_delay': detonation_delay,
                'release_pos': result['release_pos'],
                'detonation_pos': result['detonation_pos'],
                'detonation_time': result['detonation_time'],
                'effective_duration': best_duration
            }

            print(f"找到更好的参数: 方向={direction}°, 速度={speed}m/s, "
                  f"投放时间={release_time:.2f}s, 起爆延迟={detonation_delay:.2f}s, "
                  f"有效时长={best_duration:.2f}s")

    print(f"方向 {direction}° 搜索完成, 当前最佳有效时长: {best_duration:.2f}s")

# 打印出该方向的搜索耗时

```

```

end_direction_time = time.time()

print(f"方向 {direction} °搜索耗时: {end_direction_time - start_direction_time:.2f} 秒")

# 每完成一个方向的搜索，保存一次中间结果
if all_solutions:
    df_partial = pd.DataFrame(all_solutions)
    df_partial.to_excel(f"{results_dir}/solutions_direction_{direction}.xlsx", index=False)
    print(f"已保存方向 {direction} °的所有解决方案")

# 所有方向搜索完成后，保存完整的解决方案集
if all_solutions:
    df_all = pd.DataFrame(all_solutions)
    df_all.to_excel(f"{results_dir}/all_solutions.xlsx", index=False)
    print(f"已保存所有解决方案到 {results_dir}/all_solutions.xlsx，共 {len(all_solutions)} 个方案")

# 保存前 10 个最佳解决方案
df_best = df_all.sort_values(by='effective_duration', ascending=False).head(10)
df_best.to_excel(f"{results_dir}/top10_solutions.xlsx", index=False)
print(f"已保存前 10 个最佳解决方案到 {results_dir}/top10_solutions.xlsx")

# 对每个解决方案进行精细化搜索
all_fine_tuned_solutions = []
best_solutions = []

for i, row in df_best.iterrows():
    solution_id = i + 1
    base_solution = row.to_dict()

    # 进行精细化搜索
    best_params, fine_tuned_solutions = fine_tune_solution(base_solution, solution_id, results_dir)

    # 添加到所有解决方案列表
    all_fine_tuned_solutions.extend(fine_tuned_solutions)
    best_solutions.append(best_params)

# 保存所有精细化搜索的解决方案
if all_fine_tuned_solutions:
    df_all = pd.DataFrame(all_fine_tuned_solutions)
    df_all.to_excel(f"{results_dir}/all_fine_tuned_solutions.xlsx", index=False)
    print(f"已保存所有精细化搜索的解决方案，共 {len(all_fine_tuned_solutions)} 个方案")

# 比较各个起点的最佳解决方案
if best_solutions:
    df_best = pd.DataFrame([

```

```

        {
            'solution_id': params['solution_id'],
            'direction': params['direction'],
            'speed': params['speed'],
            'release_time': params['release_time'],
            'detonation_delay': params['detonation_delay'],
            'effective_duration': params['effective_duration']
        }
        for params in best_solutions
    ])

# 按有效时长排序
df_best = df_best.sort_values(by='effective_duration', ascending=False)
df_best.to_excel(f'{results_dir}/best_solutions_comparison.xlsx', index=False)
print("\n 各起点精细化搜索的最佳解决方案比较:")
print(df_best[['solution_id', 'direction', 'speed', 'release_time', 'detonation_delay', 'effective_duration']])

# 找出全局最优解
global_best = df_best.iloc[0]
print("\n 全局最优解决方案:")
print(f"来自解决方案 #{global_best['solution_id']}")
print(f"方向: {global_best['direction']:.2f} °")
print(f"速度: {global_best['speed']:.2f} m/s")
print(f"投放时间: {global_best['release_time']:.2f} s")
print(f"起爆延迟: {global_best['detonation_delay']:.2f} s")
print(f"有效遮蔽时长: {global_best['effective_duration']:.2f} s")

# 将最优解单独保存
best_solution_id = global_best['solution_id']
best_solution_data = df_all[df_all['base_solution_id'] == best_solution_id]
best_solution_data = best_solution_data.sort_values(by='effective_duration', ascending=False)
best_solution_data.head(10).to_excel(f'{results_dir}/global_best_solution_details.xlsx', index=False)
print(f"已保存全局最优解决方案详情到 {results_dir}/global_best_solution_details.xlsx")

end_time = time.time()
print(f"优化完成, 总耗时: {end_time - start_time:.2f} 秒")

# 输出最优参数
if best_params:
    print("\n 最优烟幕干扰弹投放策略:")
    print(f"飞行方向: {best_params['direction']} °")
    print(f"飞行速度: {best_params['speed']} m/s")

```

```

print(f"投放时间: {best_params['release_time']:.2f} s")
print(f"起爆延迟: {best_params['detonation_delay']:.2f} s")
print(f"投放点坐标: ({best_params['release_pos'][0]:.2f}, {best_params['release_pos'][1]:.2f}, {best_params['release_pos'][2]:.2f}) m")
print(f"起爆点坐标: ({best_params['detonation_pos'][0]:.2f}, {best_params['detonation_pos'][1]:.2f}, {best_params['detonation_pos'][2]:.2f})
m")

print(f"有效遮蔽时长: {best_params['effective_duration']:.2f} s")

# 将最优解单独保存
best_solution_df = pd.DataFrame([ {
    'direction': best_params['direction'],
    'speed': best_params['speed'],
    'release_time': best_params['release_time'],
    'detonation_delay': best_params['detonation_delay'],
    'release_pos_x': best_params['release_pos'][0],
    'release_pos_y': best_params['release_pos'][1],
    'release_pos_z': best_params['release_pos'][2],
    'detonation_pos_x': best_params['detonation_pos'][0],
    'detonation_pos_y': best_params['detonation_pos'][1],
    'detonation_pos_z': best_params['detonation_pos'][2],
    'effective_duration': best_params['effective_duration']
}])

best_solution_df.to_excel(f"{results_dir}/best_solution.xlsx", index=False)
print(f"已保存最优解决方案到 {results_dir}/best_solution.xlsx")

# 可视化最优结果
visualize_solution(best_params)

return best_params, all_solutions

def fine_tune_solution(base_solution, solution_id, results_dir):
    """对一个基础解决方案进行精细化搜索"""
    print(f"\n 开始对解决方案 #{solution_id} 进行精细化搜索:")
    print(f"基础参数: 方向={base_solution['direction']}°, 速度={base_solution['speed']}m/s, "
          f"投放时间={base_solution['release_time']:.2f}s, 起爆延迟={base_solution['detonation_delay']:.2f}s, "
          f"有效时长={base_solution['effective_duration']:.2f}s")

    start_time = time.time()

    # 提取基础参数
    direction = base_solution['direction']
    speed = base_solution['speed']

```



```

release_time = base_solution['release_time']
detonation_delay = base_solution['detonation_delay']
best_duration = base_solution['effective_duration']

# 创建精细化搜索结果列表
fine_tuning_solutions = []
best_params = {
    'direction': direction,
    'speed': speed,
    'release_time': release_time,
    'detonation_delay': detonation_delay,
    'effective_duration': best_duration,
    'solution_id': solution_id
}

# 定义搜索范围
direction_range = np.linspace(direction - 0.075, direction + 0.075, 5)
speed_range = np.linspace(max(70, speed - 0.5), min(140, speed + 0.5), 5)
release_time_range = np.linspace(max(0.1, release_time - 0.25), release_time + 0.25, 5)
detonation_delay_range = np.linspace(max(0.1, detonation_delay - 0.25), detonation_delay + 0.25, 5)

total_combinations = len(direction_range) * len(speed_range) * len(release_time_range) * len(detonation_delay_range)
print(f"搜索范围: 方向={direction-0.5}°到{direction+0.5}°, 速度={max(70, speed-1.5)}m/s 到 {min(140, speed+1.5)}m/s")
print(f"      投放时间={max(0.1, release_time-0.25)}s 到 {release_time+0.25}s, 起爆延迟={max(0.1, detonation_delay-0.25)}s 到
{detonation_delay+0.25}s")
print(f"总搜索组合数: {total_combinations}")

# 进行精细化搜索
completed = 0
for d in direction_range:
    for s in speed_range:
        for rt in release_time_range:
            for dd in detonation_delay_range:
                # 计算有效遮蔽时长
                result = calculate_effective_shielding_time_2(d, s, rt, dd)

                # 只保存有效的解决方案
                if result['effective_duration'] > 0:
                    solution = {
                        'direction': d,
                        'speed': s,
                        'release_time': rt,
                        'detonation_delay': dd,
                        'release_pos_x': result['release_pos'][0],

```

```

        'release_pos_y': result['release_pos'][1],
        'release_pos_z': result['release_pos'][2],
        'detonation_pos_x': result['detonation_pos'][0],
        'detonation_pos_y': result['detonation_pos'][1],
        'detonation_pos_z': result['detonation_pos'][2],
        'effective_duration': result['effective_duration'],
        'base_solution_id': solution_id
    }
    fine_tuning_solutions.append(solution)

# 更新最佳解决方案
if result['effective_duration'] > best_params['effective_duration']:
    best_params = {
        'direction': d,
        'speed': s,
        'release_time': rt,
        'detonation_delay': dd,
        'release_pos': result['release_pos'],
        'detonation_pos': result['detonation_pos'],
        'detonation_time': result['detonation_time'],
        'effective_duration': result['effective_duration'],
        'solution_id': solution_id
    }

    print(f"发现更优解: 方向={d:.2f}°, 速度={s:.2f}m/s, "
          f"投放时间={rt:.2f}s, 起爆延迟={dd:.2f}s, "
          f"有效时长={result['effective_duration']:.4f}s")

# 更新进度
completed += 1
if completed % 500 == 0:
    progress = (completed / total_combinations) * 100
    print(f"完成: {completed}/{total_combinations} ({progress:.1f}%)")

# 保存精细化搜索结果
if fine_tuning_solutions:
    df_fine = pd.DataFrame(fine_tuning_solutions)
    file_path = f"{results_dir}/fine_tuning_solution_{solution_id}.xlsx"
    df_fine.to_excel(file_path, index=False)
    print(f"已保存精细化搜索结果到 {file_path}, 共 {len(fine_tuning_solutions)} 个方案")

# 输出统计信息
end_time = time.time()
print(f"精细化搜索耗时: {end_time - start_time:.2f}秒")

```

```

print(f'改进幅度: {(best_params['effective_duration'] - best_duration) / best_duration * 100:.2f}%')

return best_params, fine_tuning_solutions

def visualize_solution(params):
    """可视化最优解决方案"""
    # 提取参数
    direction = params['direction']
    speed = params['speed']
    release_time = params['release_time']
    detonation_time = params['detonation_time']
    release_pos = params['release_pos']
    detonation_pos = params['detonation_pos']

    # 导弹到达假目标的时间
    missile_total_time = missile_time_to_target(MISSILE_M1_INIT, FAKE_TARGET, MISSILE_SPEED)

    # 创建 3D 图
    fig = plt.figure(figsize=(12, 10))
    ax = fig.add_subplot(111, projection='3d')

    # 绘制假目标
    ax.scatter([0], [0], [0], color='red', s=100, label='假目标')

    # 绘制真目标（圆柱体）
    theta = np.linspace(0, 2*np.pi, 100)
    z = np.linspace(0, REAL_TARGET_HEIGHT, 10)
    theta_grid, z_grid = np.meshgrid(theta, z)
    x_cylinder = REAL_TARGET_CENTER[0] + REAL_TARGET_RADIUS * np.cos(theta_grid)
    y_cylinder = REAL_TARGET_CENTER[1] + REAL_TARGET_RADIUS * np.sin(theta_grid)
    ax.plot_surface(x_cylinder, y_cylinder, z_grid, alpha=0.5, color='blue', label='真目标')

    # 绘制导弹轨迹
    missile_times = np.linspace(0, missile_total_time, 1000)
    missile_positions = []
    for t in missile_times:
        missile_positions.append(missile_trajectory(MISSILE_M1_INIT, FAKE_TARGET, MISSILE_SPEED, t))
    missile_positions = np.array(missile_positions)
    ax.plot(missile_positions[:, 0], missile_positions[:, 1], missile_positions[:, 2],
            color='red', label='导弹 M1 轨迹')
    ax.scatter(MISSILE_M1_INIT[0], MISSILE_M1_INIT[1], MISSILE_M1_INIT[2],
            color='darkred', label='导弹 M1 初始位置')

```

```

# 绘制无人机轨迹

drone_times = np.linspace(0, release_time, 100)
drone_positions = []
for t in drone_times:
    drone_positions.append(drone_trajectory(DRONE_FY1_INIT, direction, speed, t))
drone_positions = np.array(drone_positions)
ax.plot(drone_positions[:, 0], drone_positions[:, 1], drone_positions[:, 2],
        color='green', label='无人机 FY1 轨迹')
ax.scatter(DRONE_FY1_INIT[0], DRONE_FY1_INIT[1], DRONE_FY1_INIT[2],
        color='darkgreen', label='无人机 FY1 初始位置')

# 绘制烟雾弹投放点
ax.scatter(release_pos[0], release_pos[1], release_pos[2],
        color='orange', s=100, label='烟雾弹投放点')

# 绘制烟雾弹起爆点
ax.scatter(detonation_pos[0], detonation_pos[1], detonation_pos[2],
        color='purple', s=100, label='烟雾弹起爆点')

# 寻找有效遮蔽时间段
effective_periods = []
is_effective = False
start_time = None

for t in np.arange(0, missile_total_time, 0.1):
    if t >= detonation_time and t - detonation_time <= SMOKE_EFFECTIVE_TIME:
        missile_pos = missile_trajectory(MISSILE_M1_INIT, FAKE_TARGET, MISSILE_SPEED, t)
        cloud_pos = smoke_cloud_trajectory(detonation_pos, detonation_time, t)

        if cloud_pos is not None:
            effectiveness = calculate_shielding_effectiveness(
                missile_pos, cloud_pos, detonation_time, t
            )

            if effectiveness and not is_effective:
                is_effective = True
                start_time = t
            elif not effectiveness and is_effective:
                is_effective = False
                effective_periods.append((start_time, t))

if is_effective and start_time is not None:
    effective_periods.append((start_time, min(missile_total_time, detonation_time + SMOKE_EFFECTIVE_TIME)))

```

```

# 绘制有效遮蔽时间段的烟幕云团

for period_start, period_end in effective_periods:
    # 选择几个时间点绘制云团
    for t in np.linspace(period_start, period_end, 5):
        cloud_pos = smoke_cloud_trajectory(detonation_pos, detonation_time, t)
        if cloud_pos is not None:
            # 绘制烟幕云团
            u, v = np.mgrid[0:2*np.pi:15j, 0:np.pi:15j]
            x_sphere = cloud_pos[0] + SMOKE_EFFECTIVE_RADIUS * np.cos(u) * np.sin(v)
            y_sphere = cloud_pos[1] + SMOKE_EFFECTIVE_RADIUS * np.sin(u) * np.sin(v)
            z_sphere = cloud_pos[2] + SMOKE_EFFECTIVE_RADIUS * np.cos(v)
            ax.plot_wireframe(x_sphere, y_sphere, z_sphere, color='gray', alpha=0.3, linewidth=0.5)

            # 绘制遮蔽锥体示意
            missile_pos = missile_trajectory(MISSILE_M1_INIT, FAKE_TARGET, MISSILE_SPEED, t)
            # 简化处理：只绘制从导弹到真目标中心的线
            ax.plot([missile_pos[0], REAL_TARGET_CENTER[0]],
                    [missile_pos[1], REAL_TARGET_CENTER[1]],
                    [missile_pos[2], REAL_TARGET_CENTER[2]], 'r--', alpha=0.3)

# 添加遮蔽时长信息
ax.text2D(0.05, 0.95, f"有效遮蔽时长: {params['effective_duration']:.2f} 秒",
          transform=ax.transAxes, fontsize=12)

# 设置坐标轴范围和标签
ax.set_xlim([0, 20000])
ax.set_ylim([-500, 500])
ax.set_zlim([0, 2500])
ax.set_xlabel('X 轴 (m)')
ax.set_ylabel('Y 轴 (m)')
ax.set_zlabel('Z 轴 (m)')
ax.set_title('问题 2: 最优烟幕干扰弹投放策略')

# 添加图例
ax.legend(loc='upper right')

# 保存图片
plt.tight_layout()
plt.savefig('problem2_optimal_solution.png', dpi=300)
# plt.show()

if __name__ == "__main__":
    # 运行问题 2 的优化

```

```
best_params, all_solutions = optimize_problem2()
```

附录 4

问题三代码

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import time
import os
import multiprocessing
from joblib import Parallel, delayed
from tqdm import tqdm
import random

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['WenQuanYi Zen Hei']
plt.rcParams['axes.unicode_minus'] = False

# 常量定义
MISSILE_SPEED = 300 # m/s
DRONE_SPEED_RANGE = (70, 140) # m/s
SMOKE_DESCENT_RATE = 3 # m/s
SMOKE_EFFECTIVE_RADIUS = 10 # m
SMOKE_EFFECTIVE_TIME = 20 # s
GRAVITY = 9.8 # m/s^2

# 目标和初始位置信息
FAKE_TARGET = np.array([0, 0, 0])
REAL_TARGET_CENTER = np.array([0, 200, 5]) # 圆柱体中心点
REAL_TARGET_RADIUS = 7 # m
REAL_TARGET_HEIGHT = 10 # m

MISSILE_M1_INIT = np.array([20000, 0, 2000])
DRONE_FY1_INIT = np.array([17800, 0, 1800])

# 创建结果目录
RESULTS_DIR = "optimization_results_q3"
```

```

if not os.path.exists(RESULTS_DIR):
    os.makedirs(RESULTS_DIR)

def normalize(v):
    """归一化向量"""
    norm = np.linalg.norm(v)
    if norm == 0:
        return v
    return v / norm

def missile_time_to_target(init_pos, target_pos, speed):
    """计算导弹到达目标的时间"""
    distance = np.linalg.norm(target_pos - init_pos)
    return distance / speed

def missile_trajectory(init_pos, target_pos, speed, t):
    """计算导弹在时间 t 的位置"""
    direction = normalize(target_pos - init_pos)
    return init_pos + direction * speed * t

def drone_trajectory(init_pos, direction_deg, speed, t):
    """计算无人机在时间 t 的位置"""
    # 将角度转换为弧度
    direction_rad = np.radians(direction_deg)
    # 计算方向向量 (x 轴正方向为 0 度, 逆时针为正)
    direction_vector = np.array([np.cos(direction_rad), np.sin(direction_rad), 0])
    return init_pos + direction_vector * speed * t

def smoke_trajectory(release_pos, release_time, direction_deg, speed, t):
    """计算烟雾弹在时间 t 的位置 (考虑重力)"""
    # 将角度转换为弧度
    direction_rad = np.radians(direction_deg)
    # 计算方向向量
    direction_vector = np.array([np.cos(direction_rad), np.sin(direction_rad), 0])
    if t < release_time:
        return None # 未释放
    dt = t - release_time
    # 烟雾弹初始速度与无人机相同, 并在重力作用下下落
    return release_pos + np.array([0, 0, -0.5 * GRAVITY * dt**2]) + direction_vector * speed * dt

def smoke_cloud_trajectory(detonation_pos, detonation_time, t):
    """计算烟雾云团在时间 t 的位置"""
    if t < detonation_time:
        return None # 未爆炸

```

```

dt = t - detonation_time

return detonation_pos + np.array([0, 0, -SMOKE_DESCENT_RATE * dt])

def is_target_in_shadow_cone(missile_pos, cloud_pos, target_pos, target_radius, target_height):
    """检查目标是否在烟幕云团投射的阴影锥体内"""

    # 计算烟幕云团到导弹的方向向量
    direction = missile_pos - cloud_pos
    distance_missile_cloud = np.linalg.norm(direction)

    if distance_missile_cloud < SMOKE_EFFECTIVE_RADIUS:
        # 如果导弹在云团内，肯定遮蔽成功
        return True

    # 单位方向向量
    unit_direction = direction / distance_missile_cloud

    # 计算从导弹到烟幕球体的切线所形成的锥角
    sin_theta = min(1.0, SMOKE_EFFECTIVE_RADIUS / distance_missile_cloud)
    cos_theta = np.sqrt(1 - sin_theta**2)

    # 检查目标圆柱体的每个关键点是否在阴影锥体内
    num_points = 12 # 圆周上的采样点数
    angles = np.linspace(0, 2*np.pi, num_points, endpoint=False)

    # 底面圆周上的点
    bottom_points = []
    for angle in angles:
        x = target_pos[0] + target_radius * np.cos(angle)
        y = target_pos[1] + target_radius * np.sin(angle)
        z = target_pos[2]
        bottom_points.append([x, y, z])

    # 顶面圆周上的点
    top_points = []
    for angle in angles:
        x = target_pos[0] + target_radius * np.cos(angle)
        y = target_pos[1] + target_radius * np.sin(angle)
        z = target_pos[2] + target_height
        top_points.append([x, y, z])

    # 合并所有点
    all_points = np.vstack([bottom_points, top_points])

```



```

# 检查每个点是否在阴影锥体内

for point in all_points:
    # 从导弹到目标点的向量
    missile_to_point = point - missile_pos
    distance = np.linalg.norm(missile_to_point)

    if distance < 1e-10: # 防止除以零
        continue

    # 计算该向量与导弹到云团方向的夹角余弦值
    cos_angle = np.dot(missile_to_point, unit_direction) / distance

    if cos_angle > 0:
        # 如果夹角大于 90 度, 说明点在导弹的背后, 不在阴影锥体内
        return False

    # 取绝对值
    cos_angle = abs(cos_angle)
    cos_theta = abs(cos_theta)

    # 如果夹角余弦值小于锥角余弦值, 说明该点不在阴影锥体内
    if cos_angle < cos_theta:
        return False

# 所有点都在阴影锥体内, 目标被完全遮蔽
return True

def calculate_shielding_effectiveness(missile_pos, cloud_pos, cloud_start_time, current_time):
    """计算烟幕对导弹的遮蔽效果"""
    if cloud_pos is None or current_time - cloud_start_time > SMOKE_EFFECTIVE_TIME:
        return 0 # 未形成云团或云团已失效

    # 检查真目标是否被烟幕遮蔽
    target_shielded = is_target_in_shadow_cone(
        missile_pos, cloud_pos, REAL_TARGET_CENTER, REAL_TARGET_RADIUS, REAL_TARGET_HEIGHT
    )

    return 1 if target_shielded else 0

def calculate_single_smoke_effect(drone_direction, drone_speed, release_time, detonation_delay, time_step=0.01):
    """计算单个烟幕干扰弹的效果, 返回有效遮蔽的时间区间列表"""
    detonation_time = release_time + detonation_delay

    # 计算投放点

```

```

release_pos = drone_trajectory(DRONE_FY1_INIT, drone_direction, drone_speed, release_time)

# 计算起爆点
detonation_pos = smoke_trajectory(release_pos, release_time, drone_direction, drone_speed, detonation_time)

if detonation_pos is None:
    return []

# 导弹到达假目标的时间
missile_total_time = missile_time_to_target(MISSILE_M1_INIT, FAKE_TARGET, MISSILE_SPEED)
limit_time = min(missile_total_time, detonation_time + SMOKE_EFFECTIVE_TIME + 5)

# 模拟导弹和烟幕的轨迹
effective_intervals = []
in_cloud = False
start_time = None

# 遍历导弹飞行的整个过程
for t in np.arange(0, limit_time, time_step):
    missile_pos = missile_trajectory(MISSILE_M1_INIT, FAKE_TARGET, MISSILE_SPEED, t)

    if t >= detonation_time and t - detonation_time <= SMOKE_EFFECTIVE_TIME:
        cloud_pos = smoke_cloud_trajectory(detonation_pos, detonation_time, t)

        if cloud_pos is not None:
            is_effective = calculate_shielding_effectiveness(
                missile_pos, cloud_pos, detonation_time, t
            )

            if is_effective and not in_cloud:
                in_cloud = True
                start_time = t
            elif not is_effective and in_cloud:
                in_cloud = False
                effective_intervals.append((start_time, t))

# 检查如果结束时仍在云团中
if in_cloud and start_time is not None:
    end_time = min(limit_time, detonation_time + SMOKE_EFFECTIVE_TIME)
    effective_intervals.append((start_time, end_time))

return effective_intervals

def calculate_total_effective_time(time_intervals):

```

```

"""计算多个时间区间的总有效时长，处理重叠问题"""

if not time_intervals:
    return 0

# 展平列表
flat_intervals = []
for intervals in time_intervals:
    flat_intervals.extend(intervals)

# 排序时间区间
sorted_intervals = sorted(flat_intervals, key=lambda x: x[0])

merged = []
for interval in sorted_intervals:
    # 如果 merged 为空或当前区间与前一个不重叠
    if not merged or merged[-1][1] < interval[0]:
        merged.append(interval)
    else:
        # 合并重叠区间
        merged[-1] = (merged[-1][0], max(merged[-1][1], interval[1]))

# 计算总时长
total_time = sum(end - start for start, end in merged)
return total_time

def calculate_multi_smoke_effect(params):
    """计算多个烟雾干扰弹的总效果

    参数格式:
    params = [direction, speed, t_r1, dt_d1, t_r2, dt_d2, t_r3, dt_d3]
    """
    direction, speed, t_r1, dt_d1, t_r2, dt_d2, t_r3, dt_d3 = params

    # 计算三个烟雾弹的有效遮蔽时间区间
    intervals1 = calculate_single_smoke_effect(direction, speed, t_r1, dt_d1)
    intervals2 = calculate_single_smoke_effect(direction, speed, t_r2, dt_d2)
    intervals3 = calculate_single_smoke_effect(direction, speed, t_r3, dt_d3)

    # 计算总有效遮蔽时长
    total_time = calculate_total_effective_time([intervals1, intervals2, intervals3])

    return {
        'params': params,
        'effective_duration': total_time,
    }

```

```

        'intervals': [intervals1, intervals2, intervals3]
    }

def validate_params(params):
    """验证参数是否合法"""
    direction, speed, t_r1, dt_d1, t_r2, dt_d2, t_r3, dt_d3 = params

    # 检查方向范围
    if not (0 <= direction < 360):
        return False

    # 检查速度范围
    if not (70 <= speed <= 140):
        return False

    # 检查投放时间顺序
    if not (t_r1 < t_r2 < t_r3):
        return False

    # 检查起爆延迟范围
    if not (0.0 <= dt_d1 <= 2.0 and 0.0 <= dt_d2 <= 2.0 and 0.0 <= dt_d3 <= 2.0):
        return False

    # 检查总投放时间约束
    if t_r3 + dt_d3 > 12.0:
        return False

    return True

def constrain_params(params):
    """约束参数在有效范围内"""
    direction, speed, t_r1, dt_d1, t_r2, dt_d2, t_r3, dt_d3 = params

    # 约束方向
    direction = direction % 360

    # 约束速度
    speed = max(70, min(140, speed))

    # 约束起爆延迟
    dt_d1 = max(0.0, min(2.0, dt_d1))
    dt_d2 = max(0.0, min(2.0, dt_d2))
    dt_d3 = max(0.0, min(2.0, dt_d3))

```

```

# 约束投放时间顺序和总时间

if t_r1 < 0: t_r1 = 0

t_r2 = max(t_r1 + 1, t_r2)

t_r3 = max(t_r2 + 1, t_r3)


# 确保总投放时间不超过约束

if t_r3 + dt_d3 > 12.0:

    scale = 12.0 / (t_r3 + dt_d3)

    t_r1 *= scale

    t_r2 *= scale

    t_r3 *= scale


return [direction, speed, t_r1, dt_d1, t_r2, dt_d2, t_r3, dt_d3]


def adaptive_local_search(initial_params, iterations=50, initial_step_sizes=None):
    """自适应局部搜索算法"""
    if initial_step_sizes is None:
        initial_step_sizes = [1.0, 2.0, 0.1, 0.05, 0.1, 0.05, 0.1, 0.05]

    best_params = initial_params
    best_result = calculate_multi_smoke_effect(best_params)
    best_fitness = best_result['effective_duration']

    step_sizes = initial_step_sizes.copy()

    for iteration in range(iterations):
        improved = False

        # 对每个参数进行扰动
        for i in range(len(best_params)):
            for direction in [1, -1]:
                new_params = best_params.copy()
                new_params[i] += direction * step_sizes[i]

                # 确保参数在有效范围内
                new_params = constrain_params(new_params)

                # 检查是否为有效参数
                if not validate_params(new_params):
                    continue

                result = calculate_multi_smoke_effect(new_params)
                fitness = result['effective_duration']

```

```

        if fitness > best_fitness:
            best_fitness = fitness
            best_params = new_params
            best_result = result
            improved = True
            print(f'迭代 {iteration}: 找到更好的解, 有效时长: {best_fitness:.4f}')

# 自适应调整步长
if not improved:
    step_sizes = [s * 0.7 for s in step_sizes] # 缩小搜索步长
    print(f'迭代 {iteration}: 无改进, 缩小步长至 {step_sizes}')
else:
    print(f'迭代 {iteration}: 当前最佳参数: {best_params}')

# 步长过小时终止
if max(step_sizes) < 0.01:
    print(f'步长过小, 提前终止搜索, 迭代次数: {iteration+1}/{iterations}')
    break

return best_result

def genetic_algorithm(population_size=80, generations=40, elite_size=8, mutation_rate=0.2):
    """遗传算法优化"""
    # 从问题 2 的结果初始化方向和速度范围
    direction_range = (1, 10)
    speed_range = (90, 140)

    # 创建初始种群
    population = []
    for _ in range(population_size):
        # 创建一个随机个体
        direction = random.uniform(*direction_range)
        speed = random.uniform(*speed_range)

        # 随机生成三个投放时间和起爆延迟
        t_r1 = random.uniform(0.0, 2.0)
        dt_d1 = random.uniform(0.0, 2.0)

        t_r2 = random.uniform(t_r1 + 1, 5.0)
        dt_d2 = random.uniform(0.0, 2.0)

        t_r3 = random.uniform(t_r2 + 1, 8.0)
        dt_d3 = random.uniform(0.0, 2.0)

```

```

# 确保总投放时间不超过约束
if t_r3 + dt_d3 > 12.0:
    scale = 12.0 / (t_r3 + dt_d3)
    t_r1 *= scale
    t_r2 *= scale
    t_r3 *= scale

params = [direction, speed, t_r1, dt_d1, t_r2, dt_d2, t_r3, dt_d3]
population.append(params)

# 初始种群加入一些手动优化的起点
# 起点 1: 从问题 2 的最优解扩展
population[0] = [6.0, 138, 0.325, 0.5, 2.5, 0.5, 4.7, 0.6]
# 起点 2: 均匀间隔的投放
population[1] = [5.0, 135.0, 0.2, 0.3, 2.7, 0.4, 5.2, 0.5]
# 起点 3: 注重早期遮蔽
population[2] = [5.0, 135.0, 0.1, 0.5, 1.5, 0.6, 3.0, 0.7]
# 起点 4: 注重后期遮蔽
population[3] = [5.0, 135.0, 1.0, 0.3, 3.0, 0.4, 5.0, 0.5]

best_individual = None
best_fitness = 0

# 创建列表存储每代最佳个体
generation_best_records = []

# 进化迭代
for generation in range(generations):
    print(f"\n 开始第 {generation+1}/{generations} 代进化")

    # 评估适应度
    fitness_results = []
    for individual in population:
        if not validate_params(individual):
            fitness_results.append(0)
            continue

        result = calculate_multi_smoke_effect(individual)
        fitness = result['effective_duration']
        fitness_results.append(fitness)

    # 更新全局最佳解
    if fitness > best_fitness:
        best_fitness = fitness

```

```

        best_individual = individual.copy()

        print(f"第 {generation+1} 代: 找到更好的解, 有效时长: {best_fitness:.4f}")
        print(f"参数: {best_individual}")

# 找出本代最佳个体
gen_best_idx = np.argmax(fitness_results)
gen_best_fitness = fitness_results[gen_best_idx]
gen_best_individual = population[gen_best_idx]

# 记录本代最佳个体
record = {
    'generation': generation + 1,
    'fitness': round(gen_best_fitness, 4),
    'direction': round(gen_best_individual[0], 4),
    'speed': round(gen_best_individual[1], 4),
    't_r1': round(gen_best_individual[2], 4),
    'dt_d1': round(gen_best_individual[3], 4),
    't_r2': round(gen_best_individual[4], 4),
    'dt_d2': round(gen_best_individual[5], 4),
    't_r3': round(gen_best_individual[6], 4),
    'dt_d3': round(gen_best_individual[7], 4)
}
generation_best_records.append(record)

print(f"第 {generation+1} 代最佳适应度: {gen_best_fitness:.4f}")
print(f"参数: {gen_best_individual}")

# 如果已经到最后一代, 跳出循环
if generation == generations - 1:

    # 保存最后一代 population 记录到 Excel
    df_population = pd.DataFrame(population)
    df_population.columns = ['direction', 'speed', 't_r1', 'dt_d1', 't_r2', 'dt_d2', 't_r3', 'dt_d3']
    df_population['fitness'] = fitness_results
    df_population.to_excel(f'{RESULTS_DIR}/ga_final_population.xlsx', index=False)
    print(f"已保存遗传算法最后一代种群到 {RESULTS_DIR}/ga_final_population.xlsx")

    break

# 选择精英
elite_indices = np.argsort(fitness_results)[-elite_size:]
elite = [population[i] for i in elite_indices]

# 创建新一代

```



```

new_population = elite.copy()

# 通过选择、交叉和变异生成其余个体
while len(new_population) < population_size:
    # 选择两个父代 (锦标赛选择)
    tournament_size = 3
    parent1_idx = max(random.sample(range(population_size), tournament_size),
                      key=lambda i: fitness_results[i])
    parent2_idx = max(random.sample(range(population_size), tournament_size),
                      key=lambda i: fitness_results[i])

    parent1 = population[parent1_idx]
    parent2 = population[parent2_idx]

    # 交叉
    if random.random() < 0.8: # 80%概率进行交叉
        # 均匀交叉
        child = []
        for i in range(len(parent1)):
            if random.random() < 0.5:
                child.append(parent1[i])
            else:
                child.append(parent2[i])
        else:
            # 直接继承父代 1
            child = parent1.copy()

    # 变异
    for i in range(len(child)):
        if random.random() < mutation_rate:
            # 根据不同参数使用不同的变异幅度
            if i == 0: # 方向
                child[i] += random.uniform(-2, 2)
            elif i == 1: # 速度
                child[i] += random.uniform(-5, 5)
            else: # 时间参数
                child[i] += random.uniform(-0.3, 0.3)

    # 约束参数
    child = constrain_params(child)

    # 确保参数有效
    if validate_params(child):
        new_population.append(child)

```

```

        # 更新种群
        population = new_population

    # 保存每代最佳个体记录到 Excel
    df_records = pd.DataFrame(generation_best_records)
    df_records.to_excel(f'{RESULTS_DIR}/ga_generation_best_records.xlsx', index=False)
    print(f'已保存遗传算法每代最佳个体记录到 {RESULTS_DIR}/ga_generation_best_records.xlsx')

    # 对最佳个体进行局部搜索微调
    print("\n 对全局最佳解进行局部搜索微调...")
    best_result = adaptive_local_search(best_individual, iterations=20)

    return best_result

def hybrid_optimization():
    """混合优化策略：结合多种算法"""
    print("开始混合优化...")

    # 2. 遗传算法
    print("\n===== 执行遗传算法 =====")
    ga_result = genetic_algorithm()

    # 比较三种算法的结果
    results = [
        ("遗传算法", ga_result)
    ]

    best_method = max(results, key=lambda x: x[1]['effective_duration'])

    print("\n===== 优化结果比较 =====")
    for method, result in results:
        print(f'{method} 有效遮蔽时长: {result["effective_duration"]:.4f}')

    print(f'\n 最佳方法: {best_method[0]}')
    print(f'最终有效遮蔽时长: {best_method[1]["effective_duration"]:.4f}')
    print(f'最优参数: {best_method[1]["params"]}')

    return best_method[1]

def visualize_multi_smoke_effect(result):

```

```

"""可视化多烟雾干扰弹效果"""

params = result['params']
direction, speed, t_r1, dt_d1, t_r2, dt_d2, t_r3, dt_d3 = params

# 计算投放点和起爆点
release_pos1 = drone_trajectory(DRONE_FY1_INIT, direction, speed, t_r1)
detonation_time1 = t_r1 + dt_d1
detonation_pos1 = smoke_trajectory(release_pos1, t_r1, direction, speed, detonation_time1)

release_pos2 = drone_trajectory(DRONE_FY1_INIT, direction, speed, t_r2)
detonation_time2 = t_r2 + dt_d2
detonation_pos2 = smoke_trajectory(release_pos2, t_r2, direction, speed, detonation_time2)

release_pos3 = drone_trajectory(DRONE_FY1_INIT, direction, speed, t_r3)
detonation_time3 = t_r3 + dt_d3
detonation_pos3 = smoke_trajectory(release_pos3, t_r3, direction, speed, detonation_time3)

# 导弹到达假目标的时间
missile_total_time = missile_time_to_target(MISSILE_M1_INIT, FAKE_TARGET, MISSILE_SPEED)

# 创建 3D 图
fig = plt.figure(figsize=(14, 10))
ax = fig.add_subplot(111, projection='3d')

# 绘制假目标
ax.scatter([0], [0], [0], color='red', s=100, label='假目标')

# 绘制真目标（圆柱体）
theta = np.linspace(0, 2*np.pi, 100)
z = np.linspace(0, REAL_TARGET_HEIGHT, 10)
theta_grid, z_grid = np.meshgrid(theta, z)
x_cylinder = REAL_TARGET_CENTER[0] + REAL_TARGET_RADIUS * np.cos(theta_grid)
y_cylinder = REAL_TARGET_CENTER[1] + REAL_TARGET_RADIUS * np.sin(theta_grid)
ax.plot_surface(x_cylinder, y_cylinder, z_grid, alpha=0.5, color='blue', label='真目标')

# 绘制导弹轨迹
missile_times = np.linspace(0, missile_total_time, 1000)
missile_positions = []
for t in missile_times:
    missile_positions.append(missile_trajectory(MISSILE_M1_INIT, FAKE_TARGET, MISSILE_SPEED, t))
missile_positions = np.array(missile_positions)
ax.plot(missile_positions[:, 0], missile_positions[:, 1], missile_positions[:, 2],
        color='red', label='导弹 M1 轨迹')
ax.scatter(MISSILE_M1_INIT[0], MISSILE_M1_INIT[1], MISSILE_M1_INIT[2],

```

```

        color='darkred', label='导弹 M1 初始位置')

# 绘制无人机轨迹
drone_times = np.linspace(0, t_r3, 100)
drone_positions = []
for t in drone_times:
    drone_positions.append(drone_trajectory(DRONE_FY1_INIT, direction, speed, t))
drone_positions = np.array(drone_positions)
ax.plot(drone_positions[:, 0], drone_positions[:, 1], drone_positions[:, 2],
        color='green', label='无人机 FY1 轨迹')
ax.scatter(DRONE_FY1_INIT[0], DRONE_FY1_INIT[1], DRONE_FY1_INIT[2],
        color='darkgreen', label='无人机 FY1 初始位置')

# 绘制三个烟雾弹投放点和起爆点
ax.scatter(release_pos1[0], release_pos1[1], release_pos1[2],
        color='orange', s=80, label='烟雾弹 1 投放点')
ax.scatter(detonation_pos1[0], detonation_pos1[1], detonation_pos1[2],
        color='darkorange', s=80, label='烟雾弹 1 起爆点')

ax.scatter(release_pos2[0], release_pos2[1], release_pos2[2],
        color='purple', s=80, label='烟雾弹 2 投放点')
ax.scatter(detonation_pos2[0], detonation_pos2[1], detonation_pos2[2],
        color='darkviolet', s=80, label='烟雾弹 2 起爆点')

ax.scatter(release_pos3[0], release_pos3[1], release_pos3[2],
        color='cyan', s=80, label='烟雾弹 3 投放点')
ax.scatter(detonation_pos3[0], detonation_pos3[1], detonation_pos3[2],
        color='darkcyan', s=80, label='烟雾弹 3 起爆点')

# 绘制有效遮蔽时间区间的烟雾云团
colors = ['orange', 'purple', 'cyan']
for i, (detonation_time, detonation_pos) in enumerate([
    (detonation_time1, detonation_pos1),
    (detonation_time2, detonation_pos2),
    (detonation_time3, detonation_pos3)
]):
    # 获取有效遮蔽时间区间
    intervals = result['intervals'][i] if 'intervals' in result else []

    # 对每个有效区间，绘制多个时间点的云团
    for interval_start, interval_end in intervals:
        for t in np.linspace(interval_start, interval_end, 3):
            cloud_pos = smoke_cloud_trajectory(detonation_pos, detonation_time, t)
            if cloud_pos is not None:

```

```

# 绘制烟幕云团
u, v = np.mgrid[0:2*np.pi:15j, 0:np.pi:15j]
x_sphere = cloud_pos[0] + SMOKE_EFFECTIVE_RADIUS * np.cos(u) * np.sin(v)
y_sphere = cloud_pos[1] + SMOKE_EFFECTIVE_RADIUS * np.sin(u) * np.sin(v)
z_sphere = cloud_pos[2] + SMOKE_EFFECTIVE_RADIUS * np.cos(v)
ax.plot_wireframe(x_sphere, y_sphere, z_sphere, color=colors[i], alpha=0.3, linewidth=0.5)

# 绘制从导弹到真目标中心的线，表示遮蔽效果
missile_pos = missile_trajectory(MISSILE_M1_INIT, FAKE_TARGET, MISSILE_SPEED, t)
ax.plot([missile_pos[0], REAL_TARGET_CENTER[0]],
        [missile_pos[1], REAL_TARGET_CENTER[1]],
        [missile_pos[2], REAL_TARGET_CENTER[2]],
        'r--', alpha=0.3)

# 添加遮蔽时长信息
ax.text2D(0.05, 0.95, f'总有效遮蔽时长: {result['effective_duration']:.2f} 秒",
          transform=ax.transAxes, fontsize=12)

# 添加每个烟幕弹的参数信息
ax.text2D(0.05, 0.90, f'烟幕弹 1: 投放时间={t_r1:.2f}s, 起爆延迟={dt_d1:.2f}s",
          transform=ax.transAxes, fontsize=10, color='orange')
ax.text2D(0.05, 0.87, f'烟幕弹 2: 投放时间={t_r2:.2f}s, 起爆延迟={dt_d2:.2f}s",
          transform=ax.transAxes, fontsize=10, color='purple')
ax.text2D(0.05, 0.84, f'烟幕弹 3: 投放时间={t_r3:.2f}s, 起爆延迟={dt_d3:.2f}s",
          transform=ax.transAxes, fontsize=10, color='cyan')

# 设置坐标轴范围和标签
ax.set_xlim([0, 20000])
ax.set_ylim([-500, 500])
ax.set_zlim([0, 2500])
ax.set_xlabel('X 轴 (m)')
ax.set_ylabel('Y 轴 (m)')
ax.set_zlabel('Z 轴 (m)')
ax.set_title('问题 3: 三枚烟幕干扰弹投放策略')

# 添加图例
handles, labels = ax.get_legend_handles_labels()
unique_labels = []
unique_handles = []
for handle, label in zip(handles, labels):
    if label not in unique_labels:
        unique_labels.append(label)
        unique_handles.append(handle)
ax.legend(unique_handles, unique_labels, loc='upper right')

```

```

# 保存图片
plt.tight_layout()
plt.savefig(f'{RESULTS_DIR}/problem3_optimal_solution.png', dpi=300)
plt.close()

# 绘制时间线图，显示各烟雾弹的有效遮蔽区间
plt.figure(figsize=(12, 6))

colors = ['orange', 'purple', 'cyan']
labels = ['烟雾弹 1', '烟雾弹 2', '烟雾弹 3']

for i, intervals in enumerate(result['intervals']):
    for start, end in intervals:
        plt.barh(i, end - start, left=start, height=0.6, color=colors[i], alpha=0.7)
        plt.text(start + (end - start) / 2, i, f'{end - start:.1f}s',
                 ha='center', va='center', fontweight='bold')

plt.yticks([0, 1, 2], labels)
plt.xlabel('时间 (秒)')
plt.title('三枚烟雾干扰弹的有效遮蔽时间区间')
plt.grid(True, alpha=0.3)

# 添加总有效遮蔽时长信息
plt.figtext(0.5, 0.01, f'总有效遮蔽时长: {result["effective_duration"]:.2f}秒',
           ha='center', fontsize=12, bbox=dict(facecolor='white', alpha=0.8))

plt.tight_layout()
plt.savefig(f'{RESULTS_DIR}/problem3_shielding_timeline.png', dpi=300)
plt.close()

def save_result_to_excel(result):
    """保存结果到 Excel 文件"""
    direction, speed, t_r1, dt_d1, t_r2, dt_d2, t_r3, dt_d3 = result['params']

    # 计算投放点和起爆点
    release_pos1 = drone_trajectory(DRONE_FY1_INIT, direction, speed, t_r1)
    detonation_time1 = t_r1 + dt_d1
    detonation_pos1 = smoke_trajectory(release_pos1, t_r1, direction, speed, detonation_time1)

    release_pos2 = drone_trajectory(DRONE_FY1_INIT, direction, speed, t_r2)
    detonation_time2 = t_r2 + dt_d2
    detonation_pos2 = smoke_trajectory(release_pos2, t_r2, direction, speed, detonation_time2)

```

```

release_pos3 = drone_trajectory(DRONE_FY1_INIT, direction, speed, t_r3)

detonation_time3 = t_r3 + dt_d3

detonation_pos3 = smoke_trajectory(release_pos3, t_r3, direction, speed, detonation_time3)

# 创建结果 DataFrame
result_data = {
    '参数': [
        '飞行方向', '飞行速度',
        '烟雾弹 1 投放时间', '烟雾弹 1 起爆延迟',
        '烟雾弹 2 投放时间', '烟雾弹 2 起爆延迟',
        '烟雾弹 3 投放时间', '烟雾弹 3 起爆延迟',
        '总有效遮蔽时长'
    ],
    '值': [
        f"{direction:.2f}°",
        f"{speed:.2f} m/s",
        f"{t_r1:.2f} s",
        f"{dt_d1:.2f} s",
        f"{t_r2:.2f} s",
        f"{dt_d2:.2f} s",
        f"{t_r3:.2f} s",
        f"{dt_d3:.2f} s",
        f"{result['effective_duration']:.2f} s"
    ]
}

df_params = pd.DataFrame(result_data)

# 创建投放点和起爆点 DataFrame
positions_data = {
    '点位': [
        '烟雾弹 1 投放点', '烟雾弹 1 起爆点',
        '烟雾弹 2 投放点', '烟雾弹 2 起爆点',
        '烟雾弹 3 投放点', '烟雾弹 3 起爆点'
    ],
    'X 坐标 (m)': [
        f"{release_pos1[0]:.2f}",
        f"{detonation_pos1[0]:.2f}",
        f"{release_pos2[0]:.2f}",
        f"{detonation_pos2[0]:.2f}",
        f"{release_pos3[0]:.2f}",
        f"{detonation_pos3[0]:.2f}"
    ],
    'Y 坐标 (m)': [

```

```

        f"{release_pos1[1]:.2f}",
        f"{detonation_pos1[1]:.2f}",
        f"{release_pos2[1]:.2f}",
        f"{detonation_pos2[1]:.2f}",
        f"{release_pos3[1]:.2f}",
        f"{detonation_pos3[1]:.2f}"
    ],
    'Z 坐标 (m)': [
        f"{release_pos1[2]:.2f}",
        f"{detonation_pos1[2]:.2f}",
        f"{release_pos2[2]:.2f}",
        f"{detonation_pos2[2]:.2f}",
        f"{release_pos3[2]:.2f}",
        f"{detonation_pos3[2]:.2f}"
    ]
}

df_positions = pd.DataFrame(positions_data)

# 创建有效遮蔽时间区间 DataFrame
intervals_data = []
for i, intervals in enumerate(result['intervals']):
    for j, (start, end) in enumerate(intervals):
        intervals_data.append({
            '烟雾弹': f"烟雾弹 {i+1}",
            '区间序号': j+1,
            '开始时间 (s)': f"{start:.2f}",
            '结束时间 (s)': f"{end:.2f}",
            '时长 (s)': f"{end - start:.2f}"
        })

df_intervals = pd.DataFrame(intervals_data)

# 保存到 Excel 文件，不同表放在不同 sheet
with pd.ExcelWriter(f"{RESULTS_DIR}/problem3_optimal_solution.xlsx") as writer:
    df_params.to_excel(writer, sheet_name='最优参数', index=False)
    df_positions.to_excel(writer, sheet_name='投放点和起爆点', index=False)
    df_intervals.to_excel(writer, sheet_name='有效遮蔽时间区间', index=False)

print(f"已保存结果到 {RESULTS_DIR}/problem3_optimal_solution.xlsx")

def main():
    """主函数"""
    start_time = time.time()

```



```

print("开始优化问题 3: 三枚烟幕干扰弹投放策略...")

# 使用混合优化策略
result = hybrid_optimization()

# 可视化结果
print("\n 开始可视化最优结果...")
visualize_multi_smoke_effect(result)

# 保存结果到 Excel
save_result_to_excel(result)

end_time = time.time()
print(f"\n 优化完成, 总耗时: {end_time - start_time:.2f} 秒")
print(f"最终有效遮蔽时长: {result['effective_duration']:.4f} 秒")

direction, speed, t_r1, dt_d1, t_r2, dt_d2, t_r3, dt_d3 = result['params']
print("\n 最优烟幕干扰弹投放策略:")
print(f"飞行方向: {direction:.2f}°")
print(f"飞行速度: {speed:.2f} m/s")
print(f"烟幕弹 1: 投放时间={t_r1:.2f}s, 起爆延迟={dt_d1:.2f}s")
print(f"烟幕弹 2: 投放时间={t_r2:.2f}s, 起爆延迟={dt_d2:.2f}s")
print(f"烟幕弹 3: 投放时间={t_r3:.2f}s, 起爆延迟={dt_d3:.2f}s")

return result

if __name__ == "__main__":
    main()

```

附录 5

问题四代码

```

import pandas as pd
import numpy as np
import itertools
import concurrent.futures
import os
import time
from tqdm import tqdm

import numpy as np
import matplotlib.pyplot as plt

```

```

from mpl_toolkits.mplot3d import Axes3D

import matplotlib.animation as animation

from matplotlib.patches import Circle

from matplotlib import cm

from scipy.spatial.distance import cdist

import matplotlib.pyplot as plt
from matplotlib import font_manager

plt.rcParams['font.sans-serif'] = ['WenQuanYi Zen Hei']

# 常量定义
MISSILE_SPEED = 300 # m/s
DRONE_SPEED_RANGE = (70, 140) # m/s
SMOKE_DESCENT_RATE = 3 # m/s
SMOKE_EFFECTIVE_RADIUS = 10 # m
SMOKE_EFFECTIVE_TIME = 20 # s
GRAVITY = 9.8 # m/s^2

# 目标和初始位置信息
FAKE_TARGET = np.array([0, 0, 0])
REAL_TARGET_CENTER = np.array([0, 200, 5]) # 圆柱体中心点
REAL_TARGET_RADIUS = 7 # m
REAL_TARGET_HEIGHT = 10 # m

MISSILE_M1_INIT = np.array([20000, 0, 2000])
DRONE_FY1_INIT = np.array([17800, 0, 1800])
DRONE_FY2_INIT = np.array([12000, 1400, 1400])
DRONE_FY3_INIT = np.array([6000, -3000, 700])

DRONE_LOCATIONS=[DRONE_FY1_INIT,DRONE_FY2_INIT,DRONE_FY3_INIT]

def normalize(v):
    """归一化向量"""
    norm = np.linalg.norm(v)
    if norm == 0:
        return v
    return v / norm

def missile_time_to_target(init_pos, target_pos, speed):
    """计算导弹到达目标的时间"""

```

```

distance = np.linalg.norm(target_pos - init_pos)

return distance / speed

def missile_trajectory(init_pos, target_pos, speed, t):
    """计算导弹在时间 t 的位置"""

    direction = normalize(target_pos - init_pos)

    return init_pos + direction * speed * t

def drone_trajectory(init_pos, direction_deg, speed, t):
    """计算无人机在时间 t 的位置"""

    # 将角度转换为弧度

    direction_rad = np.radians(direction_deg)

    # 计算方向向量 (x 轴正方向为 0 度, 逆时针为正)

    direction_vector = np.array([np.cos(direction_rad), np.sin(direction_rad), 0])

    return init_pos + direction_vector * speed * t

def smoke_trajectory(release_pos, release_time, direction_deg, speed, t):
    """计算烟雾弹在时间 t 的位置 (考虑重力) """

    # 将角度转换为弧度

    direction_rad = np.radians(direction_deg)

    # 计算方向向量 (x 轴正方向为 0 度, 逆时针为正)

    direction_vector = np.array([np.cos(direction_rad), np.sin(direction_rad), 0])

    if t < release_time:
        return None # 未释放

    dt = t - release_time

    # 烟雾弹初始速度与无人机相同

    return release_pos + np.array([0, 0, -0.5 * GRAVITY * dt**2]) + direction_vector * speed * dt

def smoke_cloud_trajectory(detonation_pos, detonation_time, t):
    """计算烟雾云团在时间 t 的位置"""

    if t < detonation_time:
        return None # 未爆炸

    dt = t - detonation_time

    return detonation_pos + np.array([0, 0, -SMOKE_DESCENT_RATE * dt])

def is_target_in_shadow_cone(missile_pos, cloud_pos, target_pos, target_radius, target_height):
    """

    检查目标是否在烟雾云团投射的阴影锥体内

    参数:

    missile_pos: 导弹位置

    cloud_pos: 烟雾云团中心位置

    target_pos: 目标底面中心位置

```

```

target_radius: 目标半径
target_height: 目标高度

返回:

bool: 如果目标完全在阴影锥体内, 则返回 True
"""
if missile_pos[0]+10 < cloud_pos[0]:
    return False
# 计算烟幕云团到导弹的方向向量
direction = missile_pos - cloud_pos
distance_missile_cloud = np.linalg.norm(direction)

if distance_missile_cloud < SMOKE_EFFECTIVE_RADIUS:
    # 如果导弹在云团内, 肯定遮蔽成功
    return True

# 单位方向向量
unit_direction = direction / distance_missile_cloud

# 计算从导弹到烟幕球体的切线所形成的锥角
#  $\sin(\theta) = r/d$ , 其中  $r$  是球体半径,  $d$  是导弹到球体中心的距离
sin_theta = min(1.0, SMOKE_EFFECTIVE_RADIUS / distance_missile_cloud)
cos_theta = np.sqrt(1 - sin_theta**2)

# 检查目标圆柱体的每个关键点是否在阴影锥体内
# 关键点: 底面圆周上的点和顶面圆周上的点
num_points = 12 # 圆周上的采样点数
angles = np.linspace(0, 2*np.pi, num_points, endpoint=False)

# 底面圆周上的点
bottom_points = []
for angle in angles:
    x = target_pos[0] + target_radius * np.cos(angle)
    y = target_pos[1] + target_radius * np.sin(angle)
    z = target_pos[2]
    bottom_points.append([x, y, z])

# 顶面圆周上的点
top_points = []
for angle in angles:
    x = target_pos[0] + target_radius * np.cos(angle)
    y = target_pos[1] + target_radius * np.sin(angle)
    z = target_pos[2] + target_height

```

```

        top_points.append([x, y, z])

# 合并所有点
all_points = np.vstack([bottom_points, top_points])

# 检查每个点是否在阴影锥体内
for point in all_points:
    # 从导弹到目标点的向量
    missile_to_point = point - missile_pos
    distance = np.linalg.norm(missile_to_point)

    if distance < 1e-10: # 防止除以零
        continue

    # 计算该向量与导弹到云团方向的夹角余弦值
    cos_angle = np.dot(missile_to_point, unit_direction) / distance
    if cos_angle > 0:
        # 如果夹角大于 90 度, 说明点在导弹的背后, 不在阴影锥体内
        return False
    # 取绝对值
    cos_angle = abs(cos_angle)
    cos_theta = abs(cos_theta)

    # 如果夹角余弦值小于锥角余弦值, 说明该点不在阴影锥体内
    if cos_angle < cos_theta:

        #打印相关的全部信息
        # if distance_missile_cloud < 700:
        #     print("导弹位置:", missile_pos)
        #     print("云团位置:", cloud_pos)
        #     print("cos:", cos_angle, cos_theta)
        #     #print("目标点不在阴影锥体内:", point)
        return False

# 所有点都在阴影锥体内, 目标被完全遮蔽
return True

def calculate_shielding_effectiveness(missile_pos, cloud_pos, cloud_start_time, current_time):
    """计算烟幕对导弹的遮蔽效果"""
    if cloud_pos is None or current_time - cloud_start_time > SMOKE_EFFECTIVE_TIME:
        return 0 # 未形成云团或云团已失效

# 检查真目标是否被烟幕遮蔽

```

```

target_shielded = is_target_in_shadow_cone(
    missile_pos, cloud_pos, REAL_TARGET_CENTER, REAL_TARGET_RADIUS, REAL_TARGET_HEIGHT
)

return 1 if target_shielded else 0

def calculate_effective_shielding_time(drone_idx, drone_direction, drone_speed, release_time, detonation_delay):
    """计算指定参数下的有效遮蔽时长"""
    detonation_time = release_time + detonation_delay

    # 计算投放点
    release_pos = drone_trajectory(DRONE_LOCATIONS[drone_idx], drone_direction, drone_speed, release_time)

    # 计算起爆点
    detonation_pos = smoke_trajectory(release_pos, release_time, drone_direction, drone_speed, detonation_time)

    if detonation_pos is None:
        return {
            'effective_duration': 0,
            'release_pos': release_pos,
            'detonation_pos': None,
            'detonation_time': detonation_time,
            'start_time': None,
            'end_time': None,
            'shielding_intervals': []
        }

    limit_time = detonation_time + SMOKE_EFFECTIVE_TIME

    # 模拟导弹和烟幕的轨迹
    effective_duration = 0
    in_cloud = False
    start_time = None
    shielding_intervals = []

    # 时间步长
    time_step = 0.01

    # 遍历导弹飞行的整个过程
    for t in np.arange(0, limit_time, time_step):
        missile_pos = missile_trajectory(MISSILE_M1_INIT, FAKE_TARGET, MISSILE_SPEED, t)

        if t >= detonation_time and t - detonation_time <= SMOKE_EFFECTIVE_TIME:

```

```

cloud_pos = smoke_cloud_trajectory(detonation_pos, detonation_time, t)

if cloud_pos is not None:
    is_effective = calculate_shielding_effectiveness(
        missile_pos, cloud_pos, detonation_time, t
    )

    if is_effective and not in_cloud:
        in_cloud = True
        start_time = t
    elif not is_effective and in_cloud:
        in_cloud = False
        effective_duration += (t - start_time)
        shielding_intervals.append((start_time, t))

# 检查如果结束时仍在云团中
end_time = None
if in_cloud and start_time is not None:
    end_time = min(limit_time, detonation_time + SMOKE_EFFECTIVE_TIME)
    effective_duration += (end_time - start_time)
    shielding_intervals.append((start_time, end_time))

# 如果没有遮蔽区间，设置默认的开始和结束时间
if not shielding_intervals:
    first_start_time = None
    last_end_time = None
else:
    first_start_time = shielding_intervals[0][0]
    last_end_time = shielding_intervals[-1][1]

return {
    'effective_duration': effective_duration,
    'release_pos': release_pos,
    'detonation_pos': detonation_pos,
    'detonation_time': detonation_time,
    'start_time': first_start_time,
    'end_time': last_end_time,
    'shielding_intervals': shielding_intervals
}

def calculate_shielding_times(df, idx):
    """计算每个方案的遮蔽开始和结束时间"""
    drone_idx = idx - 1

```

```

# 创建新列
df['shielding_start'] = None
df['shielding_end'] = None
df['shielding_intervals'] = None
df['release_pos_x'] = None
df['release_pos_y'] = None
df['release_pos_z'] = None
df['detonation_pos_x'] = None
df['detonation_pos_y'] = None
df['detonation_pos_z'] = None

print(f"计算 FY {idx} 的遮蔽时间...")
for i, row in tqdm(df.iterrows(), total=len(df)):
    direction = row['direction']
    speed = row['speed']
    release_time = row['release_time']
    detonation_delay = row['detonation_delay']

    result = calculate_effective_shielding_time(drone_idx, direction, speed, release_time, detonation_delay)

    # 更新 DataFrame
    df.at[i, 'shielding_start'] = result['start_time']
    df.at[i, 'shielding_end'] = result['end_time']
    df.at[i, 'shielding_intervals'] = str(result['shielding_intervals'])

    # 添加投放点和起爆点坐标
    if result['release_pos'] is not None:
        df.at[i, 'release_pos_x'] = result['release_pos'][0]
        df.at[i, 'release_pos_y'] = result['release_pos'][1]
        df.at[i, 'release_pos_z'] = result['release_pos'][2]

    if result['detonation_pos'] is not None:
        df.at[i, 'detonation_pos_x'] = result['detonation_pos'][0]
        df.at[i, 'detonation_pos_y'] = result['detonation_pos'][1]
        df.at[i, 'detonation_pos_z'] = result['detonation_pos'][2]

    # 验证计算的 effective_duration 是否与 Excel 中的一致
    calculated_duration = result['effective_duration']
    original_duration = row['effective_duration']

    # 如果有显著差异，使用计算的值
    if abs(calculated_duration - original_duration) > 0.1:
        print(f"警告: FY {idx} 行 {i} 的计算时长({calculated_duration:.2f})与原始时长({original_duration:.2f})不同")
        df.at[i, 'effective_duration'] = calculated_duration

```



```

    return df

def read_excel_file(file_path):
    """读取 Excel 文件数据"""
    return pd.read_excel(file_path)

def select_top_solutions(df, n=500):
    """选择有效遮蔽时长最长的前 n 个方案"""
    # 确保有效遮蔽时长列为数值类型
    df['effective_duration'] = pd.to_numeric(df['effective_duration'])
    return df.nlargest(min(n, len(df)), 'effective_duration')

def calculate_total_shielding_time(intervals):
    """计算多个时间区间的总有效时长，处理重叠问题"""
    if not intervals:
        return 0

    # 按开始时间排序区间
    sorted_intervals = sorted(intervals)

    # 合并重叠区间
    merged = [sorted_intervals[0]]
    for current in sorted_intervals[1:]:
        previous = merged[-1]
        if current[0] <= previous[1]: # 重叠区间
            # 更新前一个区间的结束时间(如果当前区间结束时间更晚)
            merged[-1] = (previous[0], max(previous[1], current[1]))
        else: # 不重叠区间
            merged.append(current)

    # 计算总时长
    total_time = sum(end - start for start, end in merged)
    return total_time

def evaluate_combination(combo):
    """评估一个三无人机组合的总遮蔽时长"""
    sol1, sol2, sol3 = combo

    # 确保遮蔽时间有效
    intervals = []

```

```

if sol1['shielding_start'] is not None and sol1['shielding_end'] is not None:
    intervals.append((sol1['shielding_start'], sol1['shielding_end']))

if sol2['shielding_start'] is not None and sol2['shielding_end'] is not None:
    intervals.append((sol2['shielding_start'], sol2['shielding_end']))

if sol3['shielding_start'] is not None and sol3['shielding_end'] is not None:
    intervals.append((sol3['shielding_start'], sol3['shielding_end']))

total_time = calculate_total_shielding_time(intervals)
return total_time, combo

def save_results_to_excel(max_time, best_combo, output_file):
    """保存最优组合结果到 Excel 文件"""
    # 创建结果数据结构
    results_data = []

    # 提取合并后的遮蔽区间
    sol1, sol2, sol3 = best_combo
    intervals = []

    for sol in [sol1, sol2, sol3]:
        if sol['shielding_start'] is not None and sol['shielding_end'] is not None:
            intervals.append((sol['shielding_start'], sol['shielding_end']))

    # 合并区间
    sorted_intervals = sorted(intervals)
    merged = []
    for interval in sorted_intervals:
        if not merged or merged[-1][1] < interval[0]:
            merged.append(interval)
        else:
            merged[-1] = (merged[-1][0], max(merged[-1][1], interval[1]))

    # 添加总遮蔽时长
    results_data.append({
        '无人机': '总计',
        '遮蔽时长(秒)': max_time,
        '方向(度)': "",
        '速度(m/s)': "",
        '投放时间(秒)': "",
        '起爆延迟(秒)': "",
        '遮蔽开始时间(秒)': merged[0][0] if merged else None,
        '遮蔽结束时间(秒)': merged[-1][1] if merged else None,
    })

```

```

        '投放点 x 坐标(m)': ",
        '投放点 y 坐标(m)': ",
        '投放点 z 坐标(m)': ",
        '起爆点 x 坐标(m)': ",
        '起爆点 y 坐标(m)': ",
        '起爆点 z 坐标(m)': "
    })

# 添加每个无人机的详细信息
for i, solution in enumerate(best_combo):

    drone_name = f"FY {i+1}"

    # 获取坐标信息
    release_pos_x = solution.get('release_pos_x', solution.get('release_pos', [None])[0] if isinstance(solution.get('release_pos', None), (list, np.ndarray)) else None)
    release_pos_y = solution.get('release_pos_y', solution.get('release_pos', [None, None])[1] if isinstance(solution.get('release_pos', None), (list, np.ndarray)) else None)
    release_pos_z = solution.get('release_pos_z', solution.get('release_pos', [None, None, None])[2] if isinstance(solution.get('release_pos', None), (list, np.ndarray)) else None)

    detonation_pos_x = solution.get('detonation_pos_x', solution.get('detonation_pos', [None])[0] if isinstance(solution.get('detonation_pos', None), (list, np.ndarray)) else None)
    detonation_pos_y = solution.get('detonation_pos_y', solution.get('detonation_pos', [None, None])[1] if isinstance(solution.get('detonation_pos', None), (list, np.ndarray)) else None)
    detonation_pos_z = solution.get('detonation_pos_z', solution.get('detonation_pos', [None, None, None])[2] if isinstance(solution.get('detonation_pos', None), (list, np.ndarray)) else None)

    results_data.append({
        '无人机': drone_name,
        '遮蔽时长(秒)': solution['effective_duration'],
        '方向(度)': solution['direction'],
        '速度(m/s)': solution['speed'],
        '投放时间(秒)': solution['release_time'],
        '起爆延迟(秒)': solution['detonation_delay'],
        '遮蔽开始时间(秒)': solution['shielding_start'],
        '遮蔽结束时间(秒)': solution['shielding_end'],
        '投放点 x 坐标(m)': release_pos_x,
        '投放点 y 坐标(m)': release_pos_y,
        '投放点 z 坐标(m)': release_pos_z,
        '起爆点 x 坐标(m)': detonation_pos_x,
        '起爆点 y 坐标(m)': detonation_pos_y,
        '起爆点 z 坐标(m)': detonation_pos_z
    })

```

```

# 添加合并后的时间区间信息

for i, (start, end) in enumerate(merged):

    results_data.append({

        '无人机': f'合并区间 {i+1}',

        '遮蔽时长(秒)': end - start,

        '方向(度)': "",

        '速度(m/s)': "",

        '投放时间(秒)': "",

        '起爆延迟(秒)': "",

        '遮蔽开始时间(秒)': start,

        '遮蔽结束时间(秒)': end,

        '投放点 x 坐标(m)': "",

        '投放点 y 坐标(m)': "",

        '投放点 z 坐标(m)': "",

        '起爆点 x 坐标(m)': "",

        '起爆点 y 坐标(m)': "",

        '起爆点 z 坐标(m)': ""

    })

# 创建 DataFrame 并保存

results_df = pd.DataFrame(results_data)

results_df.to_excel(output_file, index=False)

print(f'已保存优化结果到 {output_file}')

def save_top_solutions(df, output_file, drone_idx):

    """保存前 100 个最佳方案的详细信息"""

    # 确保所有需要的列都存在

    columns_to_check = [

        'effective_duration', 'direction', 'speed', 'release_time',

        'detonation_delay', 'shielding_start', 'shielding_end',

        'release_pos_x', 'release_pos_y', 'release_pos_z',

        'detonation_pos_x', 'detonation_pos_y', 'detonation_pos_z'

    ]

    for col in columns_to_check:

        if col not in df.columns:

            df[col] = None

    # 添加无人机标识

    df['drone_id'] = f'FY{drone_idx}'

    # 保存数据

    df.to_excel(output_file, index=False)

    print(f'已保存 FY{drone_idx} 前 100 个最佳方案到 {output_file}')

```

```

def find_optimal_combination_parallel(df1, df2, df3, num_workers=8):
    """使用并行处理找出最优组合"""
    # 转换 DataFrame 为字典列表以便处理
    solutions1 = df1.to_dict('records')
    solutions2 = df2.to_dict('records')
    solutions3 = df3.to_dict('records')

    # 生成所有组合
    all_combinations = list(itertools.product(solutions1, solutions2, solutions3))
    total_combinations = len(all_combinations)
    print(f"总组合数: {total_combinations}")

    # 并行评估所有组合
    best_overall_time = 0
    best_overall_combo = None

    with concurrent.futures.ProcessPoolExecutor(max_workers=num_workers) as executor:
        # 提交所有组合进行评估
        futures = [executor.submit(evaluate_combination, combo) for combo in all_combinations]

        # 处理结果
        for future in tqdm(concurrent.futures.as_completed(futures), total=len(futures), desc="评估组合"):
            try:
                time_result, combo = future.result()

                if time_result > best_overall_time:
                    best_overall_time = time_result
                    best_overall_combo = combo
                    print(f"找到更好的组合, 遮蔽时长: {best_overall_time:.4f} 秒")

            except Exception as e:
                print(f"评估组合时出错: {e}")

    return best_overall_time, best_overall_combo

def main():
    # 设置输出目录
    output_dir = "multi_drone_results"
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    # 文件路径
    fyl_file = "FY1.xlsx"

```

```

fy2_file = "FY2.xlsx"
fy3_file = "FY3.xlsx"

start_time = time.time()

# 读取每个无人机的数据
print("读取无人机数据...")
fy1_df = read_excel_file(fy1_file)
fy2_df = read_excel_file(fy2_file)
fy3_df = read_excel_file(fy3_file)

# 选择每个无人机的前 100 个最佳方案
print("选择前 100 个最佳方案...")
fy1_top = select_top_solutions(fy1_df)
fy2_top = select_top_solutions(fy2_df)
fy3_top = select_top_solutions(fy3_df)

print(f"FY1: {len(fy1_top)} 个方案")
print(f"FY2: {len(fy2_top)} 个方案")
print(f"FY3: {len(fy3_top)} 个方案")

# 计算遮蔽开始和结束时间
print("计算遮蔽时间...")
fy1_top = calculate_shielding_times(fy1_top, 1)
fy2_top = calculate_shielding_times(fy2_top, 2)
fy3_top = calculate_shielding_times(fy3_top, 3)

# 保存每个无人机的 Top100 方案详细信息
save_top_solutions(fy1_top, f"{output_dir}/FY1_top100_detailed.xlsx", 1)
save_top_solutions(fy2_top, f"{output_dir}/FY2_top100_detailed.xlsx", 2)
save_top_solutions(fy3_top, f"{output_dir}/FY3_top100_detailed.xlsx", 3)

# 找出最优组合
print("寻找最优组合...")
max_time, best_combo = find_optimal_combination_parallel(fy1_top, fy2_top, fy3_top)

# 保存最优组合结果
output_file = f"{output_dir}/optimal_combination_detailed.xlsx"
save_results_to_excel(max_time, best_combo, output_file)

# 创建合并结果概要文件
summary_file = f"{output_dir}/optimization_summary.xlsx"
summary_data = [{
    '总有效遮蔽时长(秒)': max_time,

```

```

        '计算时间(秒)': time.time() - start_time,

        'FY1 方向(度)': best_combo[0]['direction'],

        'FY1 速度(m/s)': best_combo[0]['speed'],

        'FY1 投放时间(秒)': best_combo[0]['release_time'],

        'FY1 起爆延迟(秒)': best_combo[0]['detonation_delay'],

        'FY2 方向(度)': best_combo[1]['direction'],

        'FY2 速度(m/s)': best_combo[1]['speed'],

        'FY2 投放时间(秒)': best_combo[1]['release_time'],

        'FY2 起爆延迟(秒)': best_combo[1]['detonation_delay'],

        'FY3 方向(度)': best_combo[2]['direction'],

        'FY3 速度(m/s)': best_combo[2]['speed'],

        'FY3 投放时间(秒)': best_combo[2]['release_time'],

        'FY3 起爆延迟(秒)': best_combo[2]['detonation_delay']

    }}

pd.DataFrame(summary_data).to_excel(summary_file, index=False)

end_time = time.time()
elapsed_time = end_time - start_time

# 打印结果
print("\n 优化结果:")

print(f"最大总遮蔽时长: {max_time:.4f} 秒")
print(f"优化完成, 耗时: {elapsed_time:.2f} 秒")
print(f"详细结果已保存到: {output_file}")
print(f"结果概要已保存到: {summary_file}")

print("\n 最优组合详情:")
for i, solution in enumerate(best_combo):

    drone_name = f"FY {i+1}"
    print(f"\n {drone_name}:")

    print(f"  方向: {solution['direction']}°")
    print(f"  速度: {solution['speed']} m/s")
    print(f"  投放时间: {solution['release_time']} 秒")
    print(f"  起爆延迟: {solution['detonation_delay']} 秒")
    print(f"  有效遮蔽时长: {solution['effective_duration']} 秒")

    if solution['shielding_start'] is not None and solution['shielding_end'] is not None:
        print(f"    遮蔽时间区间: {solution['shielding_start']:.4f} 秒 - {solution['shielding_end']:.4f} 秒")
    else:
        print(f"    遮蔽时间区间: 无有效遮蔽")

if __name__ == "__main__":
    main()

```

附录 6

问题五搜索代码

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.animation as animation
from matplotlib.patches import Circle
from matplotlib import cm

from scipy.spatial.distance import cdist

import matplotlib.pyplot as plt
from matplotlib import font_manager
import time

import pandas as pd
import os
plt.rcParams['font.sans-serif'] = ['WenQuanYi Zen Hei']

# 常量定义
MISSILE_SPEED = 300 # m/s
DRONE_SPEED_RANGE = (70, 140) # m/s
SMOKE_DESCENT_RATE = 3 # m/s
SMOKE_EFFECTIVE_RADIUS = 10 # m
SMOKE_EFFECTIVE_TIME = 20 # s
GRAVITY = 9.8 # m/s^2

# 目标和初始位置信息
FAKE_TARGET = np.array([0, 0, 0])
REAL_TARGET_CENTER = np.array([0, 200, 5]) # 圆柱体中心点
REAL_TARGET_RADIUS = 7 # m
REAL_TARGET_HEIGHT = 10 # m

# 导弹初始位置
MISSILE_POSITIONS = [
    np.array([20000, 0, 2000]), # M1
    np.array([19000, 600, 2100]), # M2
    np.array([18000, -600, 1900]) # M3
]
```



```

# 无人机初始位置
DRONE_POSITIONS = [
    np.array([17800, 0, 1800]),    # FY1
    np.array([12000, 1400, 1400]), # FY2
    np.array([6000, -3000, 700]),  # FY3
    np.array([11000, 2000, 1800]), # FY4
    np.array([13000, -2000, 1300]) # FY5
]

MISSILE_M1_INIT = np.array([20000, 0, 2000])
DRONE_FY1_INIT = np.array([17800, 0, 1800])

def normalize(v):
    """归一化向量"""
    norm = np.linalg.norm(v)
    if norm == 0:
        return v
    return v / norm

def missile_time_to_target(init_pos, target_pos, speed):
    """计算导弹到达目标的时间"""
    distance = np.linalg.norm(target_pos - init_pos)
    return distance / speed

def missile_trajectory(init_pos, target_pos, speed, t):
    """计算导弹在时间 t 的位置"""
    direction = normalize(target_pos - init_pos)
    return init_pos + direction * speed * t

def drone_trajectory(init_pos, direction_deg, speed, t):
    """计算无人机在时间 t 的位置"""
    # 将角度转换为弧度
    direction_rad = np.radians(direction_deg)
    # 计算方向向量 (x 轴正方向为 0 度, 逆时针为正)
    direction_vector = np.array([np.cos(direction_rad), np.sin(direction_rad), 0])
    return init_pos + direction_vector * speed * t

def smoke_trajectory(release_pos, release_time, direction_deg, speed, t):
    """计算烟幕弹在时间 t 的位置 (考虑重力) """
    # 将角度转换为弧度
    direction_rad = np.radians(direction_deg)
    # 计算方向向量 (x 轴正方向为 0 度, 逆时针为正)
    direction_vector = np.array([np.cos(direction_rad), np.sin(direction_rad), 0])

```

```

    if t < release_time:
        return None # 未释放

    dt = t - release_time

    # 烟雾弹初始速度与无人机相同

    return release_pos + np.array([0, 0, -0.5 * GRAVITY * dt**2]) + direction_vector * speed * dt

def smoke_cloud_trajectory(detonation_pos, detonation_time, t):
    """计算烟雾云团在时间 t 的位置"""

    if t < detonation_time:
        return None # 未爆炸

    dt = t - detonation_time

    return detonation_pos + np.array([0, 0, -SMOKE_DESCENT_RATE * dt])

def is_target_in_shadow_cone(missile_pos, cloud_pos, target_pos, target_radius, target_height):
    """
    检查目标是否在烟雾云团投射的阴影锥体内

    参数:
    missile_pos: 导弹位置
    cloud_pos: 烟雾云团中心位置
    target_pos: 目标底面中心位置
    target_radius: 目标半径
    target_height: 目标高度

    返回:
    bool: 如果目标完全在阴影锥体内, 则返回 True
    """

    if missile_pos[0] + 10 < cloud_pos[0]:
        return False

    # 计算烟雾云团到导弹的方向向量
    direction = missile_pos - cloud_pos
    distance_missile_cloud = np.linalg.norm(direction)

    if distance_missile_cloud < SMOKE_EFFECTIVE_RADIUS:
        # 如果导弹在云团内, 肯定遮蔽成功
        return True

    # 单位方向向量
    unit_direction = direction / distance_missile_cloud

    # 计算从导弹到烟雾球体的切线所形成的锥角
    #  $\sin(\theta) = r/d$ , 其中  $r$  是球体半径,  $d$  是导弹到球体中心的距离

```

```

sin_theta = min(1.0, SMOKE_EFFECTIVE_RADIUS / distance_missile_cloud)
cos_theta = np.sqrt(1 - sin_theta**2)

# 检查目标圆柱体的每个关键点是否在阴影锥体内
# 关键点：底面圆周上的点和顶面圆周上的点
num_points = 12 # 圆周上的采样点数
angles = np.linspace(0, 2*np.pi, num_points, endpoint=False)

# 底面圆周上的点
bottom_points = []
for angle in angles:
    x = target_pos[0] + target_radius * np.cos(angle)
    y = target_pos[1] + target_radius * np.sin(angle)
    z = target_pos[2]
    bottom_points.append([x, y, z])

# 顶面圆周上的点
top_points = []
for angle in angles:
    x = target_pos[0] + target_radius * np.cos(angle)
    y = target_pos[1] + target_radius * np.sin(angle)
    z = target_pos[2] + target_height
    top_points.append([x, y, z])

# 合并所有点
all_points = np.vstack([bottom_points, top_points])

# 检查每个点是否在阴影锥体内
for point in all_points:
    # 从导弹到目标点的向量
    missile_to_point = point - missile_pos
    distance = np.linalg.norm(missile_to_point)

    if distance < 1e-10: # 防止除以零
        continue

    # 计算该向量与导弹到云团方向的夹角余弦值
    cos_angle = np.dot(missile_to_point, unit_direction) / distance
    if cos_angle > 0:
        # 如果夹角大于 90 度，说明点在导弹的背后，不在阴影锥体内
        return False

    # 取绝对值
    cos_angle = abs(cos_angle)
    cos_theta = abs(cos_theta)

```

```

# 如果夹角余弦值小于锥角余弦值, 说明该点不在阴影锥体内
if cos_angle < cos_theta:

    #打印相关的全部信息

    # if distance_missile_cloud < 700:

    #     print("导弹位置:", missile_pos)

    #     print("云团位置:", cloud_pos)

    #     print("cos:", cos_angle, cos_theta)

    #     #print("目标点不在阴影锥体内:", point)

    return False

# 所有点都在阴影锥体内, 目标被完全遮蔽
return True


def calculate_shielding_effectiveness(missile_pos, cloud_pos, cloud_start_time, current_time):
    """计算烟幕对导弹的遮蔽效果"""

    if cloud_pos is None or current_time - cloud_start_time > SMOKE_EFFECTIVE_TIME:
        return 0 # 未形成云团或云团已失效

    # 检查真目标是否被烟幕遮蔽
    target_shielded = is_target_in_shadow_cone(
        missile_pos, cloud_pos, REAL_TARGET_CENTER, REAL_TARGET_RADIUS, REAL_TARGET_HEIGHT
    )

    return 1 if target_shielded else 0


def calculate_effective_shielding_time(drone_idx, missile_idx, drone_direction, drone_speed, release_time, detonation_delay):
    """计算指定参数下的有效遮蔽时长"""

    detonation_time = release_time + detonation_delay

    # 计算投放点
    release_pos = drone_trajectory(DRONE_POSITIONS[drone_idx], drone_direction, drone_speed, release_time)

    # 计算起爆点
    detonation_pos = smoke_trajectory(release_pos, release_time, drone_direction, drone_speed, detonation_time)

    limit_time = detonation_time + SMOKE_EFFECTIVE_TIME

    # 模拟导弹和烟幕的轨迹
    effective_duration = 0
    in_cloud = False
    start_time = None

```

```

# 时间步长
time_step = 1

# 遍历导弹飞行的整个过程
for t in np.arange(0, limit_time, time_step):
    missile_pos = missile_trajectory(MISSILE_POSITIONS[missile_idx], FAKE_TARGET, MISSILE_SPEED, t)

    if t >= detonation_time and t - detonation_time <= SMOKE_EFFECTIVE_TIME:
        cloud_pos = smoke_cloud_trajectory(detonation_pos, detonation_time, t)

        if cloud_pos is not None:
            is_effective = calculate_shielding_effectiveness(
                missile_pos, cloud_pos, detonation_time, t
            )

            if is_effective and not in_cloud:
                in_cloud = True
                start_time = t
            elif not is_effective and in_cloud:
                in_cloud = False
                effective_duration += (t - start_time)

# 检查如果结束时仍在云团中
if in_cloud:
    end_time = detonation_time + SMOKE_EFFECTIVE_TIME
    effective_duration += (end_time - start_time)

return {
    'effective_duration': effective_duration,
    'release_pos': release_pos,
    'detonation_pos': detonation_pos,
    'detonation_time': detonation_time
}

def optimize_problem(drone_idx, missile_idx):
    print("正在优化问题...")
    start_time = time.time()

    # 创建存储所有解决方案的列表
    all_solutions = []

    # 搜索参数空间
    best_params = None

```

```

best_duration = 0

# 创建结果目录
results_dir = f'FY{drone_idx+1}_Missile{missile_idx+1}_results'
if not os.path.exists(results_dir):
    os.makedirs(results_dir)

if DRONE_POSITIONS[drone_idx][1] - MISSILE_POSITIONS[missile_idx][1] < 0:
    directions = np.linspace(0, 180, 41)
else:
    directions = np.linspace(180, 360, 41)

# 搜索空间
for direction in directions:
    start_direction_time = time.time()
    for speed in np.linspace(100, 140, 21): #26# 70 到 140 m/s, 每 2.5 m/s 搜索一次
        for release_time in np.linspace(0, 40, 11): #11
            for detonation_delay in np.linspace(0, 20, 21): #11

                result = calculate_effective_shielding_time(
                    drone_idx, missile_idx, direction, speed, release_time, detonation_delay
                )

                # 将当前解决方案添加到列表中(只保存有效时长大于 4 的方案)
                if result['effective_duration'] > 0:
                    solution = {
                        'direction': direction,
                        'speed': speed,
                        'release_time': release_time,
                        'detonation_delay': detonation_delay,
                        'release_pos_x': result['release_pos'][0],
                        'release_pos_y': result['release_pos'][1],
                        'release_pos_z': result['release_pos'][2],
                        'detonation_pos_x': result['detonation_pos'][0],
                        'detonation_pos_y': result['detonation_pos'][1],
                        'detonation_pos_z': result['detonation_pos'][2],
                        'effective_duration': result['effective_duration']
                    }
                    all_solutions.append(solution)

            if result['effective_duration'] > best_duration:

```

```

        best_duration = result['effective_duration']

        best_params = {
            'direction': direction,
            'speed': speed,
            'release_time': release_time,
            'detonation_delay': detonation_delay,
            'release_pos': result['release_pos'],
            'detonation_pos': result['detonation_pos'],
            'detonation_time': result['detonation_time'],
            'effective_duration': best_duration
        }

        print(f"找到更好的参数: 方向={direction}°, 速度={speed}m/s, "
              f"投放时间={release_time:.2f}s, 起爆延迟={detonation_delay:.2f}s, "
              f"有效时长={best_duration:.2f}s")

    print(f"方向 {direction}° 搜索完成, 当前最佳有效时长: {best_duration:.2f}s")
    # 打印出该方向的搜索耗时
    end_direction_time = time.time()
    print(f"方向 {direction}° 搜索耗时: {end_direction_time - start_direction_time:.2f} 秒")

    # 每完成一个方向的搜索, 保存一次中间结果
    if all_solutions:
        df_partial = pd.DataFrame(all_solutions)
        df_partial.to_excel(f"{results_dir}/solutions_direction_{direction}.xlsx", index=False)
        print(f"已保存方向 {direction}° 的所有解决方案")

# 所有方向搜索完成后, 保存完整的解决方案集
if all_solutions:
    df_all = pd.DataFrame(all_solutions)
    df_all.to_excel(f"{results_dir}/all_solutions.xlsx", index=False)
    print(f"已保存所有解决方案到 {results_dir}/all_solutions.xlsx, 共 {len(all_solutions)} 个方案")

    # 保存前 10 个最佳解决方案
    df_best = df_all.sort_values(by='effective_duration', ascending=False).head(10)
    df_best.to_excel(f"{results_dir}/top10_solutions.xlsx", index=False)
    print(f"已保存前 10 个最佳解决方案到 {results_dir}/top10_solutions.xlsx")

end_time = time.time()
print(f"优化完成, 总耗时: {end_time - start_time:.2f} 秒")

# 输出最优参数
if best_params:
    print("\n 最优烟幕干扰弹投放策略:")

```

```

print(f"飞行方向: {best_params['direction']}°")
print(f"飞行速度: {best_params['speed']} m/s")
print(f"投放时间: {best_params['release_time']:.2f} s")
print(f"起爆延迟: {best_params['detonation_delay']:.2f} s")
print(f"投放点坐标: ({best_params['release_pos'][0]:.2f}, {best_params['release_pos'][1]:.2f}, {best_params['release_pos'][2]:.2f}) m")
print(f"起爆点坐标: ({best_params['detonation_pos'][0]:.2f}, {best_params['detonation_pos'][1]:.2f}, {best_params['detonation_pos'][2]:.2f}) m")

print(f"有效遮蔽时长: {best_params['effective_duration']:.2f} s")

# 将最优解单独保存
best_solution_df = pd.DataFrame([
    'direction': best_params['direction'],
    'speed': best_params['speed'],
    'release_time': best_params['release_time'],
    'detonation_delay': best_params['detonation_delay'],
    'release_pos_x': best_params['release_pos'][0],
    'release_pos_y': best_params['release_pos'][1],
    'release_pos_z': best_params['release_pos'][2],
    'detonation_pos_x': best_params['detonation_pos'][0],
    'detonation_pos_y': best_params['detonation_pos'][1],
    'detonation_pos_z': best_params['detonation_pos'][2],
    'effective_duration': best_params['effective_duration']
])

best_solution_df.to_excel(f'{results_dir}/best_solution.xlsx', index=False)
print(f"已保存最优解决方案到 {results_dir}/best_solution.xlsx")

return best_params, all_solutions

if __name__ == "__main__":
    for i in range(5):
        for j in range(3):
            if i==0 :
                continue
            best_params, all_solutions = optimize_problem(i,j)

```


问题五组合代码

```
import numpy as np

import pandas as pd

import os

import time

from itertools import product

from tqdm import tqdm

import multiprocessing

from joblib import Parallel, delayed

import itertools


# 常量定义

MISSILE_SPEED = 300 # m/s

DRONE_SPEED_RANGE = (70, 140) # m/s

SMOKE_DESCENT_RATE = 3 # m/s

SMOKE_EFFECTIVE_RADIUS = 10 # m

SMOKE_EFFECTIVE_TIME = 20 # s

GRAVITY = 9.8 # m/s^2


# 目标和初始位置信息

FAKE_TARGET = np.array([0, 0, 0])

REAL_TARGET_CENTER = np.array([0, 200, 5]) # 圆柱体中心点

REAL_TARGET_RADIUS = 7 # m

REAL_TARGET_HEIGHT = 10 # m


# 导弹初始位置

MISSILE_POSITIONS = [

    np.array([20000, 0, 2000]), # M1

    np.array([19000, 600, 2100]), # M2

    np.array([18000, -600, 1900]) # M3

]


# 无人机初始位置

DRONE_POSITIONS = [

    np.array([17800, 0, 1800]), # FY1

    np.array([12000, 1400, 1400]), # FY2

    np.array([6000, -3000, 700]), # FY3

    np.array([11000, 2000, 1800]), # FY4

    np.array([13000, -2000, 1300]) # FY5

]


def normalize(v):

    """归一化向量"""

    norm = np.linalg.norm(v)
```

```

    if norm == 0:
        return v
    return v / norm

def missile_trajectory(init_pos, target_pos, speed, t):
    """计算导弹在时间 t 的位置"""
    direction = normalize(target_pos - init_pos)
    return init_pos + direction * speed * t

def drone_trajectory(init_pos, direction_deg, speed, t):
    """计算无人机在时间 t 的位置"""
    direction_rad = np.radians(direction_deg)
    direction_vector = np.array([np.cos(direction_rad), np.sin(direction_rad), 0])
    return init_pos + direction_vector * speed * t

def smoke_trajectory(release_pos, release_time, direction_deg, speed, t):
    """计算烟雾弹在时间 t 的位置"""
    direction_rad = np.radians(direction_deg)
    direction_vector = np.array([np.cos(direction_rad), np.sin(direction_rad), 0])
    if t < release_time:
        return None
    dt = t - release_time
    return release_pos + np.array([0, 0, -0.5 * GRAVITY * dt**2]) + direction_vector * speed * dt

def smoke_cloud_trajectory(detonation_pos, detonation_time, t):
    """计算烟雾云团在时间 t 的位置"""
    if t < detonation_time:
        return None
    dt = t - detonation_time
    return detonation_pos + np.array([0, 0, -SMOKE_DESCENT_RATE * dt])

def is_target_in_shadow_cone(missile_pos, cloud_pos, target_pos, target_radius, target_height):
    """检查目标是否在烟雾云团投射的阴影锥体内"""
    if missile_pos[0] + 10 < cloud_pos[0]:
        return False

    direction = missile_pos - cloud_pos
    distance_missile_cloud = np.linalg.norm(direction)

    if distance_missile_cloud < SMOKE_EFFECTIVE_RADIUS:
        return True

    unit_direction = direction / distance_missile_cloud
    sin_theta = min(1.0, SMOKE_EFFECTIVE_RADIUS / distance_missile_cloud)

```

```

cos_theta = np.sqrt(1 - sin_theta**2)

num_points = 12
angles = np.linspace(0, 2*np.pi, num_points, endpoint=False)

bottom_points = []
for angle in angles:
    x = target_pos[0] + target_radius * np.cos(angle)
    y = target_pos[1] + target_radius * np.sin(angle)
    z = target_pos[2]
    bottom_points.append([x, y, z])

top_points = []
for angle in angles:
    x = target_pos[0] + target_radius * np.cos(angle)
    y = target_pos[1] + target_radius * np.sin(angle)
    z = target_pos[2] + target_height
    top_points.append([x, y, z])

all_points = np.vstack([bottom_points, top_points])

for point in all_points:
    missile_to_point = point - missile_pos
    distance = np.linalg.norm(missile_to_point)

    if distance < 1e-10:
        continue

    cos_angle = np.dot(missile_to_point, unit_direction) / distance
    if cos_angle > 0:
        return False

    cos_angle = abs(cos_angle)
    cos_theta = abs(cos_theta)

    if cos_angle < cos_theta:
        return False

return True

def calculate_shielding_effectiveness(missile_pos, cloud_pos, cloud_start_time, current_time):
    """计算烟雾对导弹的遮蔽效果"""
    if cloud_pos is None or current_time - cloud_start_time > SMOKE_EFFECTIVE_TIME:
        return 0

```

```

target_shielded = is_target_in_shadow_cone(
    missile_pos, cloud_pos, REAL_TARGET_CENTER, REAL_TARGET_RADIUS, REAL_TARGET_HEIGHT
)

return 1 if target_shielded else 0

def calculate_missile_defense_effect(missile_idx, defense_plan, time_step=0.01):
    """计算特定导弹的防御效果（有效遮蔽时间区间）"""
    # 导弹到达假目标的时间
    missile_init_pos = MISSILE_POSITIONS[missile_idx]
    distance = np.linalg.norm(FAKE_TARGET - missile_init_pos)
    missile_total_time = distance / MISSILE_SPEED

    # 按时间排序防御计划
    sorted_defense = sorted(defense_plan, key=lambda x: x['release_time'] + x['detonation_delay'])

    if not sorted_defense:
        return []

    earliest_defense = sorted_defense[0]['release_time'] + sorted_defense[0]['detonation_delay']
    latest_defense = sorted_defense[-1]['release_time'] + sorted_defense[-1]['detonation_delay'] + SMOKE_EFFECTIVE_TIME

    # 确定模拟的时间范围
    start_time = max(0, earliest_defense - 1)
    end_time = min(missile_total_time, latest_defense + 1)

    # 模拟导弹和烟幕的轨迹
    effective_intervals = []
    in_cloud = False
    start_time_record = None

    # 为每个烟幕弹计算并缓存起爆点和有效时间窗口
    defense_cache = []
    for defense in sorted_defense:
        drone_idx = defense['drone_idx']
        direction = defense['direction']
        speed = defense['speed']
        release_time = defense['release_time']
        detonation_delay = defense['detonation_delay']
        detonation_time = release_time + detonation_delay

        # 计算投放点和起爆点
        drone_init_pos = DRONE_POSITIONS[drone_idx]

```

```

release_pos = drone_trajectory(drone_init_pos, direction, speed, release_time)
detonation_pos = smoke_trajectory(release_pos, release_time, direction, speed, detonation_time)

if detonation_pos is not None:
    defense_cache.append({
        'detonation_pos': detonation_pos,
        'detonation_time': detonation_time,
        'end_time': detonation_time + SMOKE_EFFECTIVE_TIME
    })

# 遍历导弹飞行的时间步长
for t in np.arange(start_time, end_time, time_step):
    missile_pos = missile_trajectory(missile_init_pos, FAKE_TARGET, MISSILE_SPEED, t)

    # 检查所有烟幕云团对该导弹的遮蔽效果
    is_effective = 0

    # 只考虑当前时间有效的烟幕弹
    active_defenses = [d for d in defense_cache if d['detonation_time'] <= t <= d['end_time']]

    for defense in active_defenses:
        detonation_pos = defense['detonation_pos']
        detonation_time = defense['detonation_time']

        # 计算云团位置
        cloud_pos = smoke_cloud_trajectory(detonation_pos, detonation_time, t)

        if cloud_pos is not None:
            # 计算遮蔽效果
            effect = calculate_shielding_effectiveness(
                missile_pos, cloud_pos, detonation_time, t
            )
            is_effective = max(is_effective, effect)

    # 记录有效遮蔽区间
    if is_effective and not in_cloud:
        in_cloud = True
        start_time_record = t
    elif not is_effective and in_cloud:
        in_cloud = False
        effective_intervals.append((start_time_record, t))

# 检查如果结束时仍在云团中
if in_cloud and start_time_record is not None:

```

```

        effective_intervals.append((start_time_record, end_time))

    return effective_intervals

def calculate_single_bomb_intervals(drone_idx, missile_idx, direction, speed, release_time, detonation_delay):
    """计算单个烟雾弹的有效遮蔽区间"""
    defense_plan = [{
        'drone_idx': drone_idx,
        'direction': direction,
        'speed': speed,
        'release_time': release_time,
        'detonation_delay': detonation_delay
    }]

    return calculate_missile_defense_effect(missile_idx, defense_plan)

def calculate_total_effective_time(time_intervals):
    """计算多个时间区间的总有效时长，处理重叠问题"""
    if not time_intervals:
        return 0

    sorted_intervals = sorted(time_intervals)
    merged = [sorted_intervals[0]]

    for current in sorted_intervals[1:]:
        previous = merged[-1]
        if current[0] <= previous[1]: # 重叠区间
            merged[-1] = (previous[0], max(previous[1], current[1]))
        else: # 不重叠区间
            merged.append(current)

    total_time = sum(end - start for start, end in merged)
    return total_time

def get_top_solutions_with_intervals(drone_idx, missile_idx, num_solutions=3):
    """获取指定无人机-导弹组合的前几个最佳解决方案并预计算其遮蔽区间"""
    solutions = []

    # 无人机 1 使用固定参数
    if drone_idx == 0:
        if missile_idx == 0: # 只对 M1 有效
            # FY1 对 M1 的三个烟雾弹参数
            bomb_params = [
                {

```

```

        'direction': 4.87985955499065,
        'speed': 140,
        'release_time': 0,
        'detonation_delay': 0
    },
    {
        'direction': 4.87985955499065,
        'speed': 140,
        'release_time': 1,
        'detonation_delay': 0
    },
    {
        'direction': 4.87985955499065,
        'speed': 140,
        'release_time': 7.00425122294099,
        'detonation_delay': 0.203560474384496
    }
]

# 计算每个烟雾弹的有效遮蔽区间
for i, params in enumerate(bomb_params):
    intervals = calculate_single_bomb_intervals(
        drone_idx, missile_idx,
        params['direction'], params['speed'],
        params['release_time'], params['detonation_delay']
    )

    effective_duration = calculate_total_effective_time(intervals)

    solutions.append({
        'direction': params['direction'],
        'speed': params['speed'],
        'release_time': params['release_time'],
        'detonation_delay': params['detonation_delay'],
        'effective_duration': effective_duration,
        'intervals': intervals
    })

print(f'已预计算无人机 FY{drone_idx+1} 对导弹 M{missile_idx+1} 的 {len(solutions)} 个烟雾弹遮蔽区间')
return solutions

# 其他无人机从文件加载解决方案
try:
    fine_tune_dir = f'FY{drone_idx+1}_Missile{missile_idx+1}_results/fine_tuned'

```

```

# 优先使用精细化的解决方案

if os.path.exists(f'{fine_tune_dir}/top10_fine_tuned_solutions.xlsx'):
    df = pd.read_excel(f'{fine_tune_dir}/top10_fine_tuned_solutions.xlsx')
    if not df.empty:
        top_df = df.sort_values(by='effective_duration', ascending=False).head(num_solutions)
        for _, row in top_df.iterrows():
            # 计算该方案的有效遮蔽区间
            intervals = calculate_single_bomb_intervals(
                drone_idx, missile_idx,
                row['direction'], row['speed'],
                row['release_time'], row['detonation_delay']
            )

            solutions.append({
                'direction': row['direction'],
                'speed': row['speed'],
                'release_time': row['release_time'],
                'detonation_delay': row['detonation_delay'],
                'effective_duration': row['effective_duration'],
                'intervals': intervals
            })

# 如果精细化解决方案不足，使用原始解决方案补充
if len(solutions) < num_solutions:
    results_dir = f'FY {drone_idx+1}_Missile {missile_idx+1}_results'
    if os.path.exists(f'{results_dir}/top10_solutions.xlsx'):
        df = pd.read_excel(f'{results_dir}/top10_solutions.xlsx')
        if not df.empty:
            top_df = df.sort_values(by='effective_duration', ascending=False).head(num_solutions - len(solutions))
            for _, row in top_df.iterrows():
                # 计算该方案的有效遮蔽区间
                intervals = calculate_single_bomb_intervals(
                    drone_idx, missile_idx,
                    row['direction'], row['speed'],
                    row['release_time'], row['detonation_delay']
                )

                solutions.append({
                    'direction': row['direction'],
                    'speed': row['speed'],
                    'release_time': row['release_time'],
                    'detonation_delay': row['detonation_delay'],
                    'effective_duration': row['effective_duration'],

```



```

        'intervals': intervals

    })

    print(f'已预计算无人机 FY {drone_idx+1} 对导弹 M {missile_idx+1} 的 {len(solutions)} 个解决方案遮蔽区间')
except Exception as e:
    print(f'加载解决方案时出错: {e}')

return solutions

def optimize_defense_strategy(num_combinations_per_drone=2):

    """优化整体防御策略，考虑不同无人机和导弹的组合，使用预计算的遮蔽区间"""
    print("开始优化整体防御策略...")

    # 创建结果目录
    results_dir = "combined_defense_results"
    if not os.path.exists(results_dir):
        os.makedirs(results_dir)

    # 获取每个无人机-导弹组合的前几个最佳解决方案和遮蔽区间
    print("预计算每个无人机-导弹组合的遮蔽区间...")
    drone_missile_solutions = {}

    for drone_idx in range(5):
        drone_missile_solutions[drone_idx] = {}
        for missile_idx in range(3):
            solutions = get_top_solutions_with_intervals(drone_idx, missile_idx, num_combinations_per_drone)
            if solutions:
                drone_missile_solutions[drone_idx][missile_idx] = solutions

    # 为每个无人机创建可能的烟幕弹组合
    drone_combinations = {}

    # 无人机 1 固定参数
    drone_combinations[0] = [{
        'direction': 4.87985955499065,
        'speed': 140,
        'bombs': [
            {
                'release_time': 0,
                'detonation_delay': 0,
                'target_missile': 0,
                'intervals': drone_missile_solutions[0][0]['intervals']
            },

```

```

        {
            'release_time': 1,
            'detonation_delay': 0,
            'target_missile': 0,
            'intervals': drone_missile_solutions[0][0][1]['intervals']
        },
        {
            'release_time': 7.00425122294099,
            'detonation_delay': 0.203560474384496,
            'target_missile': 0,
            'intervals': drone_missile_solutions[0][0][2]['intervals']
        }
    ]
}]

# 为其他无人机创建组合 - 改进的方法
for drone_idx in range(1, 5):
    drone_combinations[drone_idx] = []

    # 1. 收集该无人机所有可能的方向和速度组合
    direction_speed_options = set()
    for missile_idx in range(3):
        if missile_idx in drone_missile_solutions[drone_idx]:
            for solution in drone_missile_solutions[drone_idx][missile_idx]:
                direction_speed_options.add((solution['direction'], solution['speed']))

    # 将集合转换为列表以便迭代
    direction_speed_options = list(direction_speed_options)

    # 2. 对每个方向和速度组合，找到针对每个导弹的最佳参数
    for direction, speed in direction_speed_options:
        # 为每个导弹找到在给定方向和速度下的最佳解决方案
        missile_best_params = {}

        for missile_idx in range(3):
            if missile_idx in drone_missile_solutions[drone_idx]:
                # 筛选具有相同方向和速度的解决方案
                matching_solutions = [
                    s for s in drone_missile_solutions[drone_idx][missile_idx]
                    if abs(s['direction'] - direction) < 0.01 and abs(s['speed'] - speed) < 0.01
                ]

                if matching_solutions:
                    # 按有效时长排序，选择最佳的

```

```

best_solution = max(matching_solutions, key=lambda x: x['effective_duration'])

missile_best_params[missile_idx] = {
    'release_time': best_solution['release_time'],
    'detonation_delay': best_solution['detonation_delay'],
    'effective_duration': best_solution['effective_duration'],
    'intervals': best_solution['intervals']
}

# 3. 创建所有可能的组合（最多 3 枚烟雾弹）
if missile_best_params: # 如果有任何匹配的解决方案
    # 生成所有可能的导弹目标组合（最多 3 个）
    for r in range(1, min(4, len(missile_best_params) + 1)):
        for missile_targets in itertools.combinations(missile_best_params.keys(), r):
            bombs = []
            for target in missile_targets:
                bombs.append({
                    'release_time': missile_best_params[target]['release_time'],
                    'detonation_delay': missile_best_params[target]['detonation_delay'],
                    'target_missile': target,
                    'intervals': missile_best_params[target]['intervals']
                })

            # 确保投放时间顺序合理（间隔至少 1 秒）
            if len(bombs) > 1:
                bombs.sort(key=lambda x: x['release_time'])
                for i in range(1, len(bombs)):
                    if bombs[i]['release_time'] - bombs[i-1]['release_time'] < 1.0:
                        bombs[i]['release_time'] = bombs[i-1]['release_time'] + 1.0

            drone_combinations[drone_idx].append({
                'direction': direction,
                'speed': speed,
                'bombs': bombs
            })

# 创建所有无人机组合的列表
print("生成所有可能的防御策略...")
all_drone_combinations = list(product(
    drone_combinations[0],
    drone_combinations[1],
    drone_combinations[2],
    drone_combinations[3],
    drone_combinations[4]

```

```

))

print(f'共生成 {len(all_drone_combinations)} 种可能的防御策略组合，开始评估...')

# 定义评估函数 - 使用预计算的区间
def evaluate_combination(combo):
    # 每个导弹的所有烟幕弹遮蔽区间
    missile_intervals = {0: [], 1: [], 2: []}

    # 收集每个无人机对每个导弹的遮蔽区间
    for drone_idx, drone_solution in enumerate(combo):
        for bomb in drone_solution['bombs']:
            target_missile = bomb['target_missile']
            missile_intervals[target_missile].extend(bomb['intervals'])

    # 计算每个导弹的总有效遮蔽时长
    missile_times = []
    for missile_idx in range(3):
        total_time = calculate_total_effective_time(missile_intervals[missile_idx])
        missile_times.append(total_time)

    # 创建结果对象
    return {
        'drone_solutions': {i: combo[i] for i in range(5)},
        'total_time': sum(missile_times),
        'times_by_missile': missile_times,
        'intervals_by_missile': missile_intervals
    }

# 并行评估所有组合
num_cores = multiprocessing.cpu_count()
print(f'使用 {num_cores} 个 CPU 核心并行评估...')

all_results = Parallel(n_jobs=num_cores)(
    delayed(evaluate_combination)(combo)
    for combo in tqdm(all_drone_combinations)
)

# 按总时长排序
sorted_results = sorted(all_results, key=lambda x: x['total_time'], reverse=True)

# 保存前 100 个结果
top_results = sorted_results[:100]

```

```

# 将结果转换为 DataFrame
result_data = []

for idx, result in enumerate(top_results):
    row = {
        'rank': idx + 1,
        'total_time': result['total_time'],
        'M1_time': result['times_by_missile'][0],
        'M2_time': result['times_by_missile'][1],
        'M3_time': result['times_by_missile'][2]
    }

    # 添加每个无人机的参数
    for drone_idx, solution in result['drone_solutions'].items():
        row[f'FY {drone_idx+1}_direction'] = solution['direction']
        row[f'FY {drone_idx+1}_speed'] = solution['speed']

        for i, bomb in enumerate(solution['bombs']):
            row[f'FY {drone_idx+1}_bomb {i+1}_missile'] = f'M{bomb['target_missile']+1}'
            row[f'FY {drone_idx+1}_bomb {i+1}_release_time'] = bomb['release_time']
            row[f'FY {drone_idx+1}_bomb {i+1}_detonation_delay'] = bomb['detonation_delay']

    result_data.append(row)

# 创建 DataFrame 并保存
df_results = pd.DataFrame(result_data)
df_results.to_excel(f'{results_dir}/top100_defense_strategies.xlsx', index=False)
print(f'已保存前 100 个最佳防御策略到 {results_dir}/top100_defense_strategies.xlsx')

# 输出最佳策略
# 如果其中一个遮蔽时长为 0 则跳过，选择下一个

best_result = sorted_results[0]
for best_result in sorted_results:
    if all(t > 0 for t in best_result['times_by_missile']):
        break

print("\n 最佳防御策略:")
print(f'总有效遮蔽时长: {best_result['total_time']:.2f} 秒')
print(f'导弹 M1 有效遮蔽时长: {best_result['times_by_missile'][0]:.2f} 秒')
print(f'导弹 M2 有效遮蔽时长: {best_result['times_by_missile'][1]:.2f} 秒')
print(f'导弹 M3 有效遮蔽时长: {best_result['times_by_missile'][2]:.2f} 秒')

print("\n 无人机参数:")
for drone_idx, solution in best_result['drone_solutions'].items():
    print(f'无人机 FY {drone_idx+1}: 方向={solution['direction']:.2f}°, 速度={solution['speed']:.2f} m/s')

```

```

        for i, bomb in enumerate(solution["bombs"]):
            print(f"    烟雾弹 {i+1}: 目标=M{bomb['target_missile']+1}, "
                  f"    投放时间={bomb['release_time']:.2f}s, 起爆延迟={bomb['detonation_delay']:.2f}s")

# 将最佳策略保存为详细格式
best_df = pd.DataFrame([ {
    'total_time': best_result['total_time'],
    'M1_time': best_result['times_by_missile'][0],
    'M2_time': best_result['times_by_missile'][1],
    'M3_time': best_result['times_by_missile'][2]
}])

best_df.to_excel(f'{results_dir}/best_strategy_summary.xlsx", index=False)

# 保存详细的无人机参数
drone_data = []
for drone_idx, solution in best_result['drone_solutions'].items():
    for i, bomb in enumerate(solution["bombs"]):
        drone_data.append( {
            '无人机': f'FY {drone_idx+1}',
            '飞行方向(°)': solution['direction'],
            '飞行速度(m/s)': solution['speed'],
            '烟雾弹编号': i+1,
            '目标导弹': f'M{bomb["target_missile"]+1}',
            '投放时间(s)': bomb['release_time'],
            '起爆延迟(s)': bomb['detonation_delay']
        })

pd.DataFrame(drone_data).to_excel(f'{results_dir}/best_strategy_details.xlsx", index=False)
print(f'已保存最佳策略详细信息到 {results_dir}/best_strategy_details.xlsx")

return sorted_results[0]

if __name__ == "__main__":
    start_time = time.time()
    optimize_defense_strategy(num_combinations_per_drone=2)
    end_time = time.time()
    print(f'总耗时: {end_time - start_time:.2f} 秒")

```